

Chapter 6

Graphs

6.1 The Graph Abstract Data Type

6.1.1 introduction

Graphs have been **widely used** in:

analysis of electrical circuits

finding shortest routes

project planning

identification of chemical compounds

statistical mechanics

genetics

cybernetics

linguistics

social science

...

6.1.2 Definitions

Graph $G = (V, E)$

V : set of vertices, finite, nonempty

E : set of edges $E(G)$

undirected graph: $(u, v) = (v, u)$

directed graph: $\langle u, v \rangle$, u ---**tail**, v ---**head**,
 $\langle u, v \rangle \neq \langle v, u \rangle$

Three graphs:

G_1 :

$$V(G_1) = \{0, 1, 2, 3\}$$

$$E(G_1) = \{(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)\}$$

G_2 :

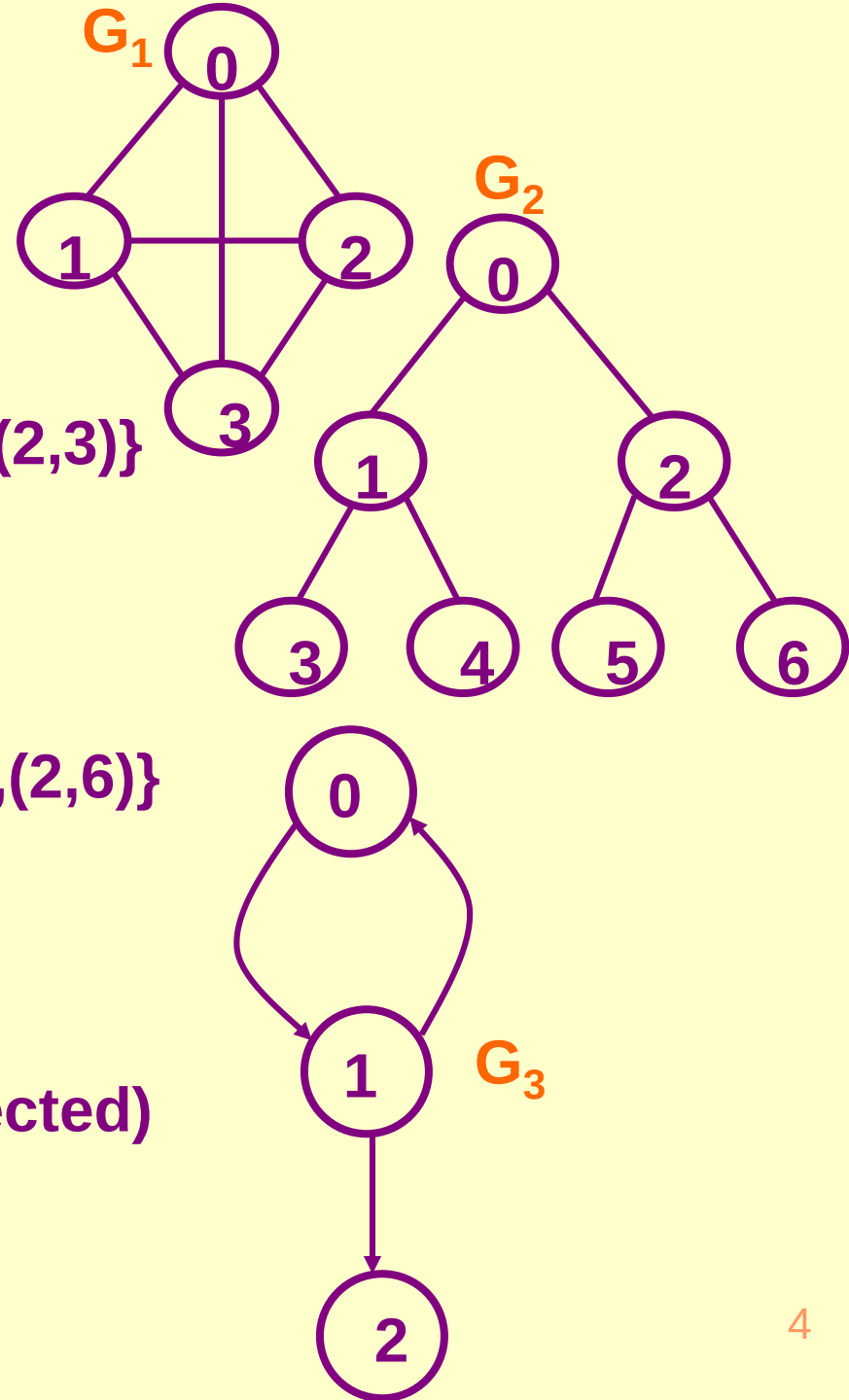
$$V(G_2) = \{0, 1, 2, 3, 4, 5, 6\}$$

$$E(G_2) = \{(0, 1), (0, 2), (1, 3), (1, 4), (2, 5), (2, 6)\}$$

G_3 :

$$V(G_3) = \{0, 1, 2\}$$

$$E(G_3) = \{\langle 0, 1 \rangle, \langle 1, 0 \rangle, \langle 1, 2 \rangle\} \text{ (directed)}$$



■ Restrictions on graphs:

(1) (v, v) or $\langle v, v \rangle$ is not legal, such edges are known as self edges or self loops. If we permit self edges, we obtain a data object referred to as a graph with self edges.

(2) multiple occurrences of the same edges are not allowed. If allowed, we get a multigraph.

■ Thus we have:

- The maximum number of edges in any n -vertex undirected graph is $n(n-1)/2$. ---undirected complete graph
- The maximum number of edges in any n -vertex directed graph is $n(n-1)$. ---directed complete graph

- If (u, v) is a undirected edge, we say u and v are **adjacent** and edge (u, v) is **incident on vertices u and v** . If $\langle u, v \rangle$ is a directed edge, then vertex u is **adjacent to v** , and v is **adjacent from u** , $\langle u, v \rangle$ is **incident to u and v** .
- A **subgraph** of G is a graph G' such that $V(G') \subseteq V(G)$ and $E(G') \subseteq E(G)$.
- A **path** from u to v in G is a sequence of vertices $u, i_1, i_2, \dots, i_k, v$ such that $(u, i_1), (i_1, i_2), \dots, (i_k, v)$ are edges in $E(G)$. If G' is directed, then the path consists of $\langle u, i_1 \rangle, \langle i_1, i_2 \rangle, \dots, \langle i_k, v \rangle$ edges in $E(G')$.
- The **length of a path** is the number of edges on it.
- A **simple path** is a path in which all vertices except possibly the first and last are distinct.

- A **cycle** is a simple path in which the first and last vertices are the same.
- In an undirected G , **u and v are connected** iff there is a path in G from u to v (this means there must be a path from v to u).
- An undirected **G is connected** iff for every pair of distinct u and v in $V(G)$, there is a path from u to v .
- A **tree** is a connected acyclic (i.e., has no cycles) graph.

- A **connected component** H of an undirected graph G is a **maximal connected subgraph**. By maximal, we mean that G contains **no other subgraph that is both connected and properly contains H** .

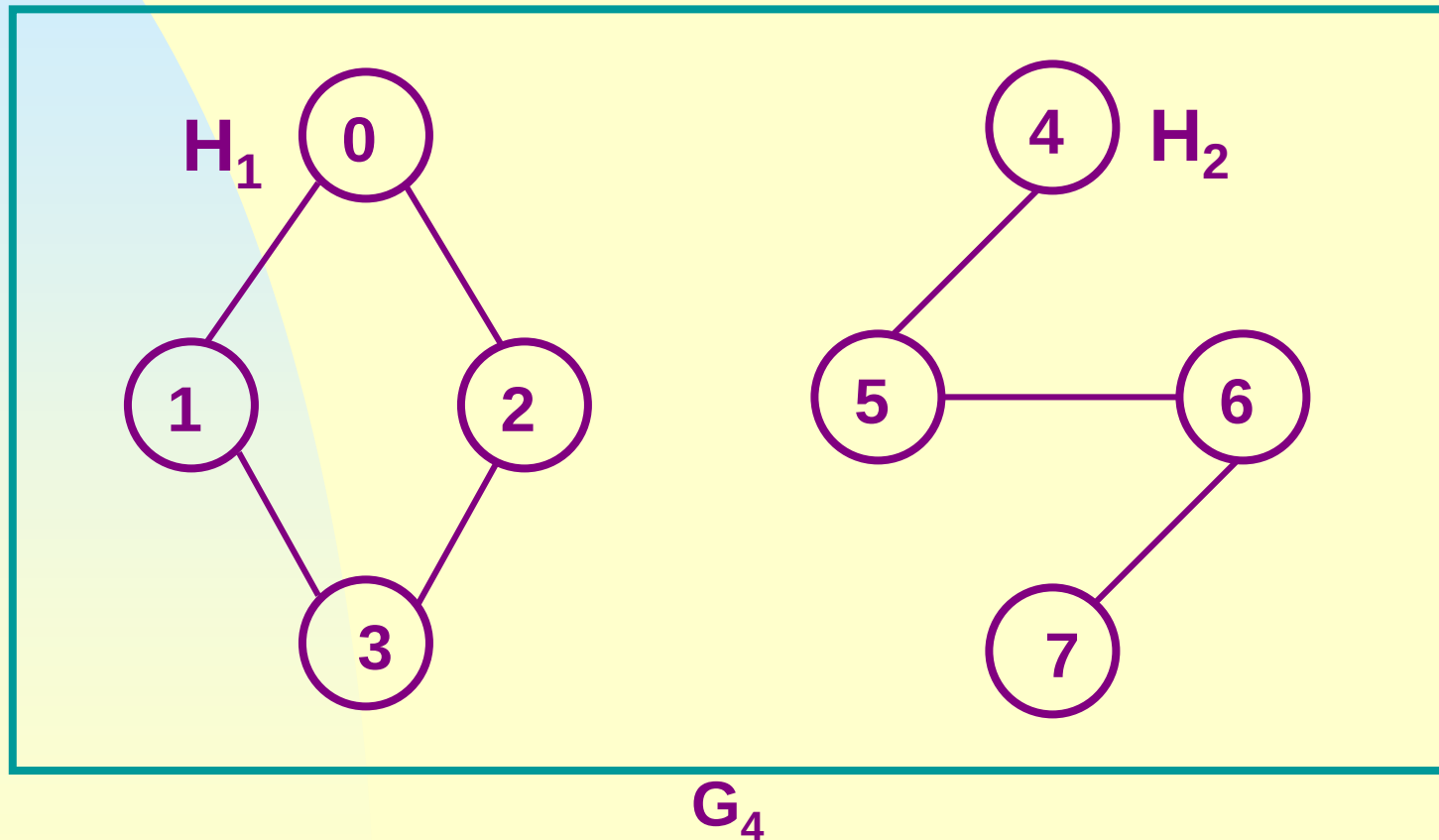


Fig 6.5 A graph with two connected components

- A **directed G** is **strongly connected** iff for every pair of distinct u and v in $V(G)$, there is a directed path from u to v and also from v to u .
- A **strongly connected component** is a maximal subgraph that is strongly connected.
- The **degree of a vertex in undirected graph or in directed graph** is the number of edges incident to that vertex.
- For directed graph G , the **in-degree of $v \in V(G)$** is the number of edges for which v is the **head**. The **out-degree** is the number of edges for which v is the **tail**.

- If d_i is the degree of vertex i in a undirected or directed graph G with n vertices and e edges, then the number of edges is

$$e = \left(\sum_{i=0}^{n-1} d_i \right) / 2$$

- In the following, we'll refer to a directed graph as **digraph**, and a undirected graph as **graph**.

ADT 6.1 Graph

class Graph

{ // A nonempty set of vertices and a set of undirected
// edges, where each edge is a pair of vertices.

public:

virtual ~Graph(){ };

// virtual destructor

bool IsEmpty() **const** {return n==0;};

// return **true** iff graph has no vertices

int NumberOfVertices() **const** {return n;};

// return the number of vertices in the graph

int NumberofEdges() **const** {return e;};

// return number of edges in the graph

virtual int Degree(int u) **const** =0;

// return number of edges incident to vertex u

```
virtual bool ExistsEdge(int u, int v) const =0;  
    // return true iff graph has edge (u, v)  
virtual void InsertVertex (int v) =0;  
    // insert vertex v into graph, v has no incident edges  
virtual void InsertEdge (int u, int v) =0;  
    // insert edge (u, v) into graph  
virtual void DeleteVertex (int v);  
    // delete v and all edges incident to it  
virtual void DeleteEdge (int u, int v) =0;  
    // delete edge (u, v) from the graph  
private:  
    int n;    // number of vertices  
    int e;    // number of edges  
};
```

The graph data structure is specified as an abstract class in ADT 6.1:

- ✓ We use an abstract class, because many different representations for a graph are possible.
- ✓ Classes that actually implement a graph using a specific graph representation can derive publicly from the abstract class Graph.

6.1.3 Graph Representations

Although several representations for graphs are possible, we shall study only the three most commonly used:

- (1) Adjacency matrices
- (2) Adjacency lists
- (3) Adjacency multilists

The choice of a particular representation will depend upon the application and the functions one expects to perform on the graph.

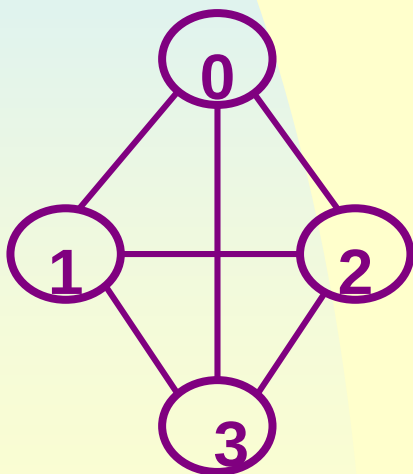
6.1.3.1 Adjacency Matrix

The adjacency matrix of G is a **two-dimensional $n \times n$ array**, say a , such that:

$$a[i][j] = \begin{cases} 1 & \text{iff } (i, j) \in E(G) \text{ (or } \langle i, j \rangle \in E(G) \text{)} \\ 0 & \text{otherwise} \end{cases}$$

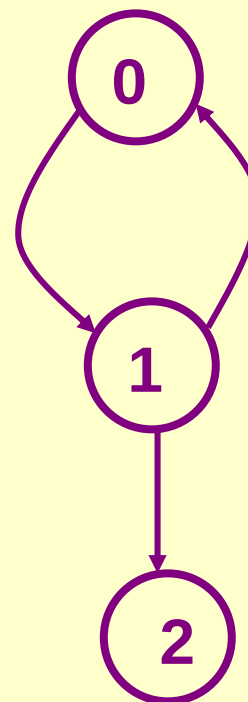
The next slide show adjacency matrices of G_1 , G_3 and G_4 .

	0	1	2	3
0	0	1	1	1
1	1	0	1	1
2	1	1	0	1
3	1	1	1	0



(a) G_1

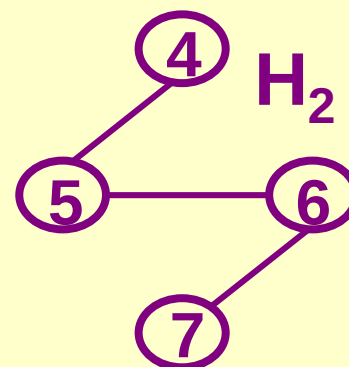
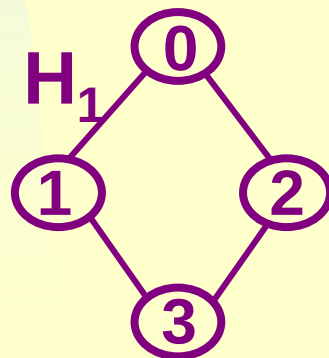
	0	1	2
0	0	1	0
1	1	0	1
2	0	0	0



(b) G_3

	0	1	2	3	4	5	6	7
0	0	1	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0
2	1	0	0	1	0	0	0	0
3	0	1	1	0	0	0	0	0
4	0	0	0	0	0	1	0	0
5	0	0	0	0	1	0	1	0
6	0	0	0	0	0	1	0	1
7	0	0	0	0	0	0	1	0

G_4



For an **undirected graph**, the adjacency matrix is **symmetric**, and

$$d_i = \sum_{j=0}^{n-1} a[i][j]$$

For a **digraph**, the adjacency matrix **may not be symmetric**, and

$$\text{out-}d_i = \sum_{j=0}^{n-1} a[i][j]$$

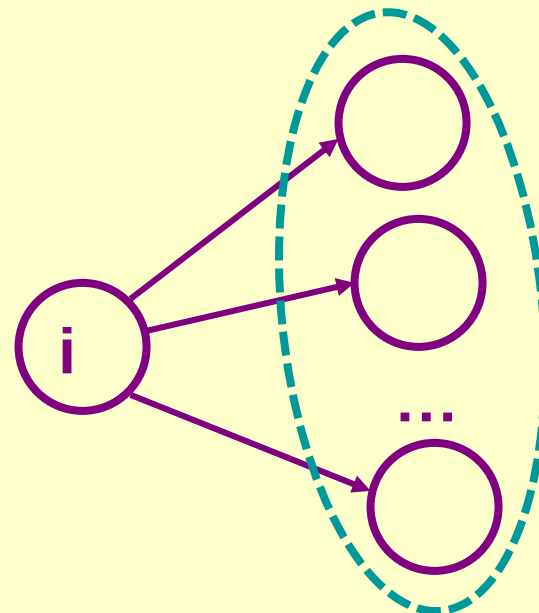
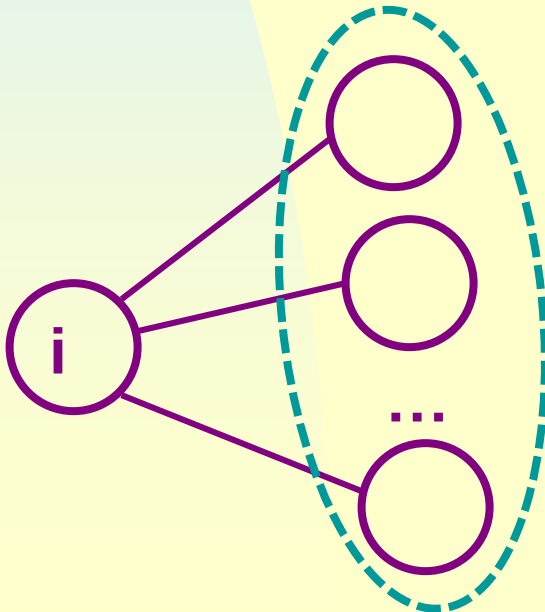
$$\text{in-}d_j = \sum_{i=0}^{n-1} a[i][j]$$

Question: How to get the number of edges in G ?

- ✓ Using an **adjacency matrix representation**, we need $O(n^2)$ time.
- ✓ Because $n^2 - n$ entries of the matrix (diagonal entries are zero) have to be examined.
- ✓ When graphs are **sparse** ($e \ll n^2 / 2$), and if we **explicitly store the edges** in G , then we can solve the above problem in $O(e + n)$.
- ✓ This leads to the next representation for graph, **adjacency lists**.

6.1.3.2 Adjacency Lists

- In this representation, the n rows of the adjacency matrix are represented as n chains (n adjacency lists).
- There is one chain for each vertex in G .
- The nodes in chain i represent the vertices adjacent from i .



- In chain i , the data field of a chain node stores the index of an adjacent vertex of vertex i .
- The vertices in each chain are not required to be ordered.
- An array `adjLists` is used to store the pointers of the first nodes in the adjacency lists. This array may be declared as

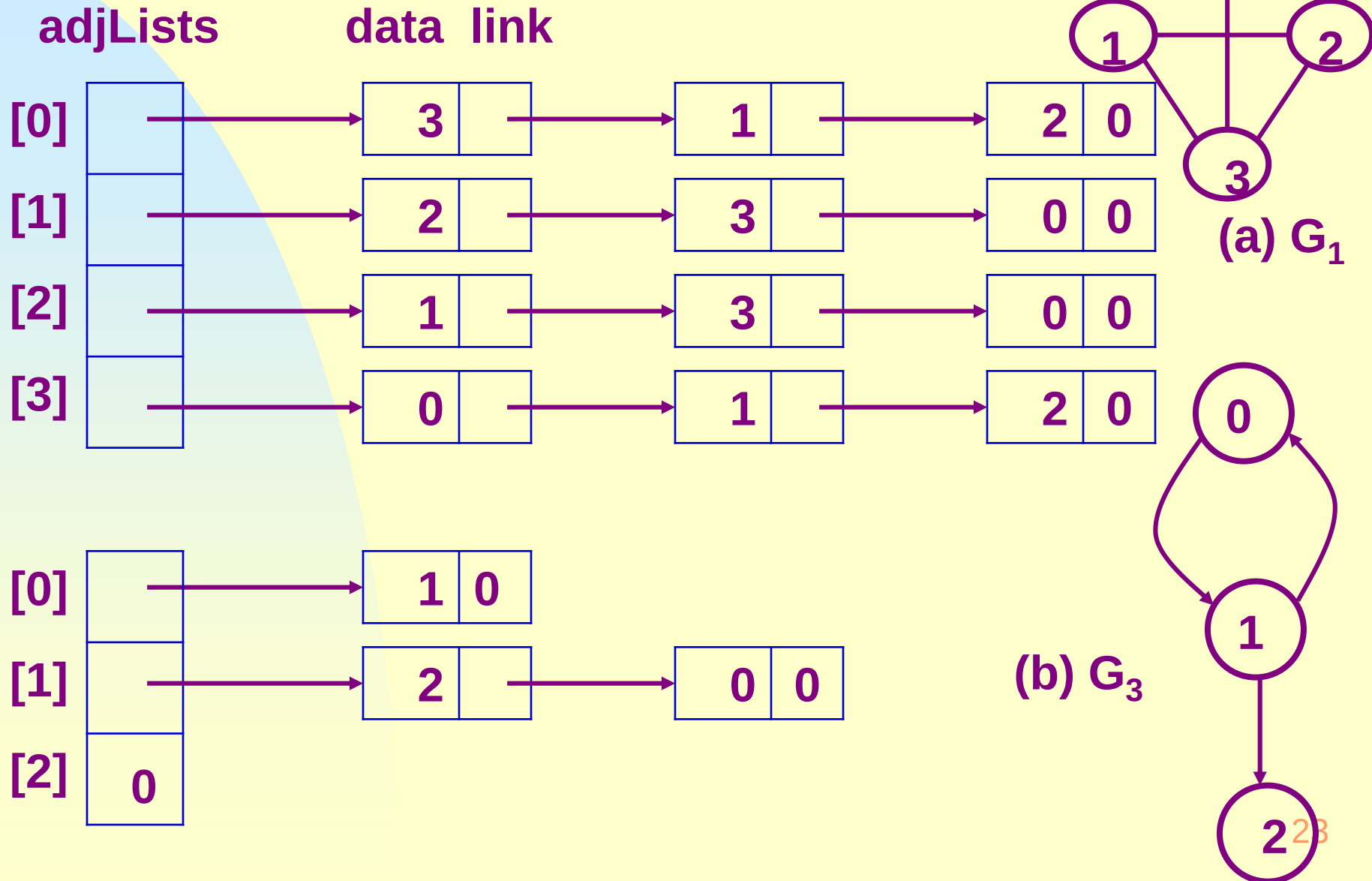
`Chain<int> *adjLists;`

- In the following, `LinkedGraph` is the class for undirected graphs represented as linked adjacency lists, and `adjLists` is a private data member of this class.

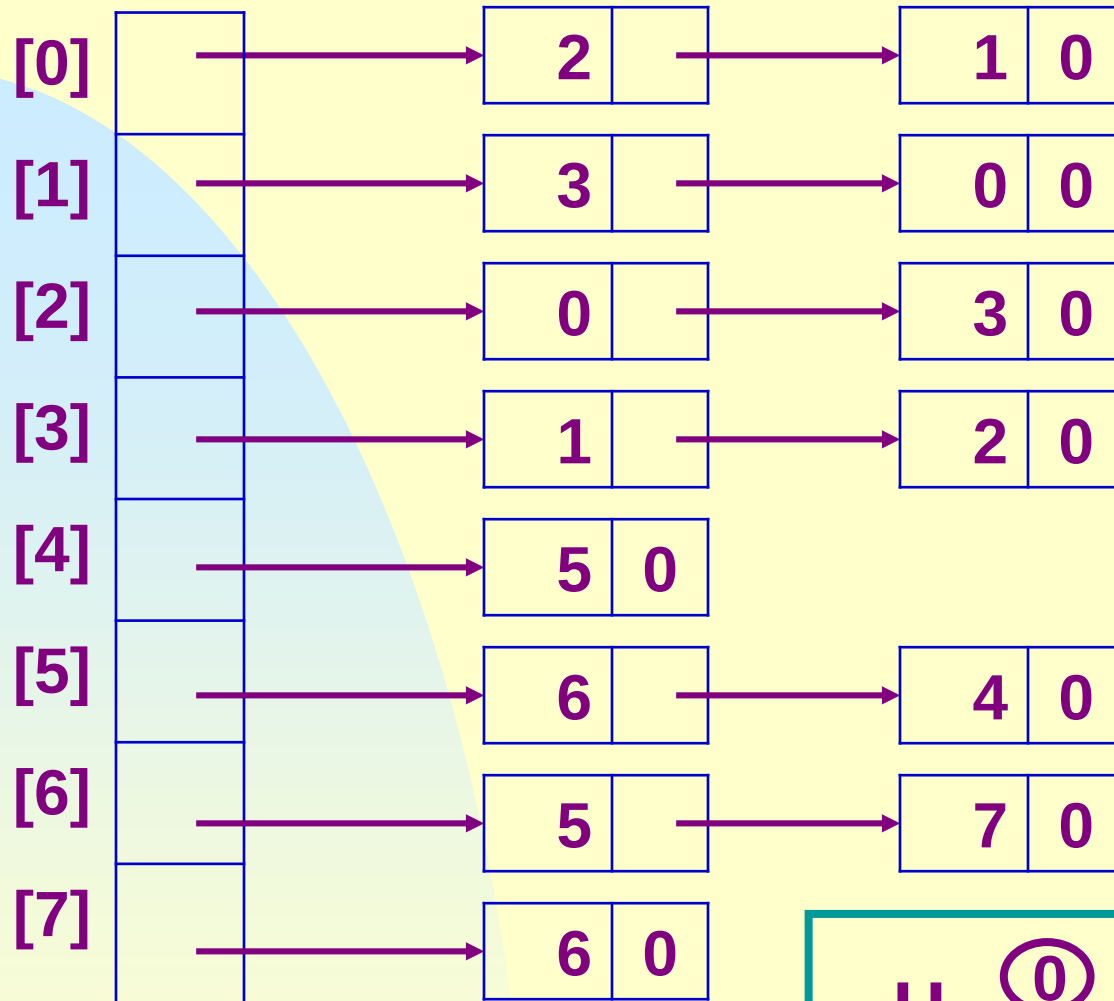
```
class LinkedGraph
{
public:
    LinkedGraph (const int vertices): e(0)
    {
        if (vertices < 1) throw "Number of vertices must be > 0";
        n = vertices;
        adjLists = new Chain<int>[n];
    };
private:
    Chain<int>* adjLists;
    int n;
    int e;
};

//Note: Chain<int> has the private data member
ChainNode<int> *first
```

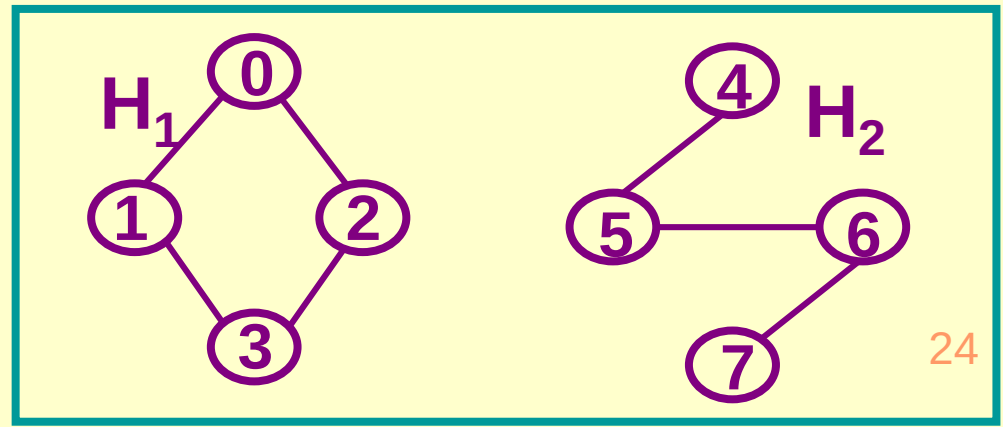
The adjacency list for G_1, G_3 and G_4



adjLists



(c) G_4



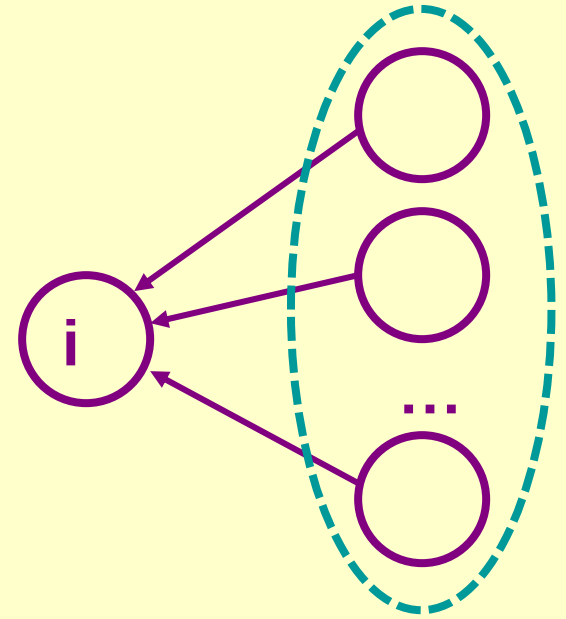
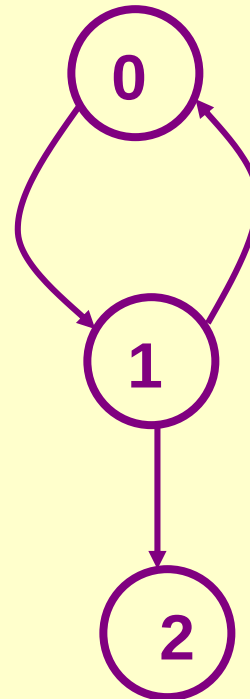
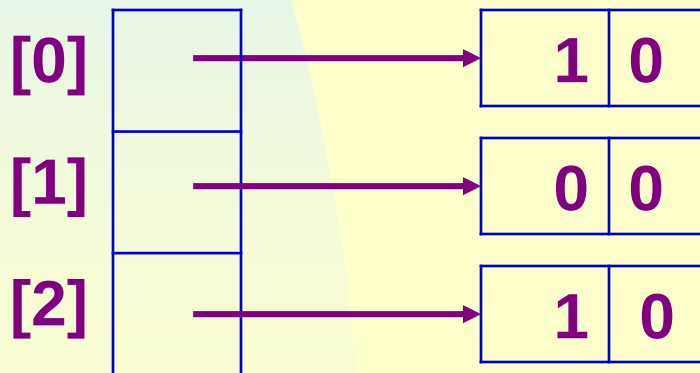
- For an **undirected graph** with n vertices and e edges:
 - It requires an array of size n , and the number of chain nodes is $2e$.
 - The **degree of any vertex** can be determined by just counting the number of nodes in its adjacency list (chain).
- For a **digraph** with n vertices and e edges:
 - It requires an array of size n , and the number of chain nodes is e .
 - **The out-degree of any vertex** can be determined by counting the number of nodes on its adjacency list.
 - To determine the **in-degree of a vertex**, we may keep another set of lists, **inverse adjacency lists**.

Inverse adjacency lists:

one list for each vertex

the nodes in list i represent the vertices adjacent to vertex i .

The following is the inverse adjacency lists for G_3 :



Orthogonal list representation for digraph

For digraph, combining adjacency lists and inverse adjacency lists forms the **orthogonal list representation** (i.e., adjacency multilists for digraph, which will be introduced in the next).

Edge node:

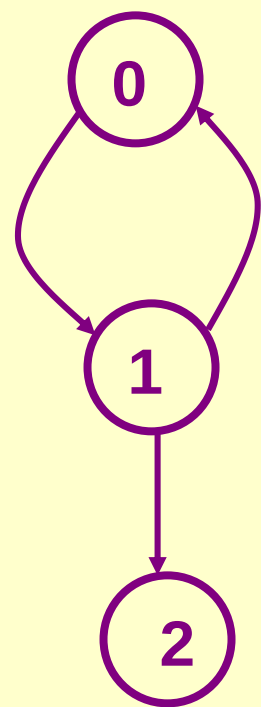
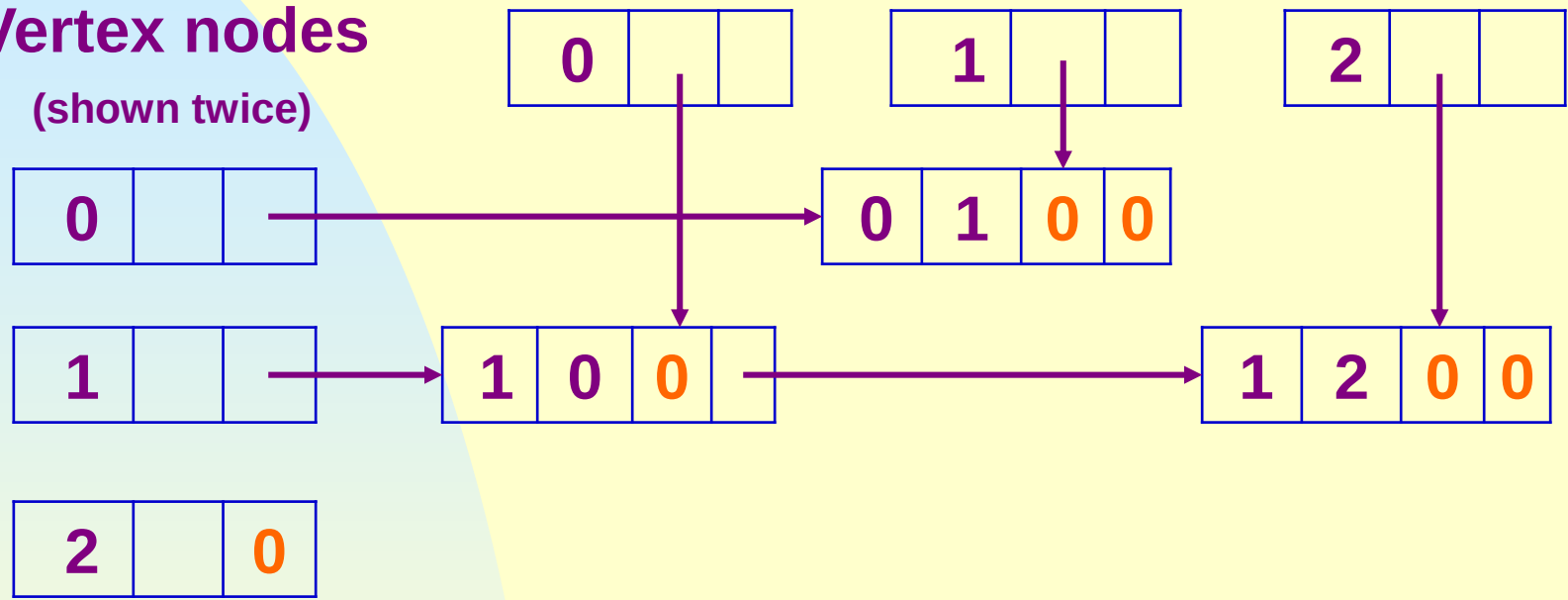
m	tail	head	column link	row link
---	------	------	-------------	----------

Vertex node:

data	firstin	firstout
------	---------	----------

Example: The orthogonal list representation for G_3 :

Vertex nodes
(shown twice)



Vertex node:

data	firstin	firstout
------	---------	----------

Edge node:

m	tail	head	column link	row link
---	------	------	-------------	----------

Note: In the edge node, **m is not shown** in the graph.

6.1.3.3 Adjacency Multilists

- ✓ In the adjacency lists of an undirected graph, each edge (u, v) is represented by two entries, one on the list for u and the other on the list for v .
- ✓ In some situations, it is not convenient for the adjacency list representation. For example, if we wish to mark an edge that has been visited, we should find this two entries.
- ✓ This is easier to accomplish if the graph is represented by adjacency multilists.
- ✓ Multilists are lists in which nodes may be shared among several lists).

- ✓ In the **adjacency multilists** representation, there will be **exactly one node for each edge**, but this node will be **shared in two lists**.
- ✓ The edge node structure is

m	vertex1	vertex2	link1	link2
---	---------	---------	-------	-------

m is a boolean mark field that may be used to indicate **whether the edge has been examined**.

- ✓ The **storage is the same** as for adjacency lists, except for the addition of the mark bit m.
- ✓ For digraph, the adjacency multilists representation is actually the orthogonal list representation.

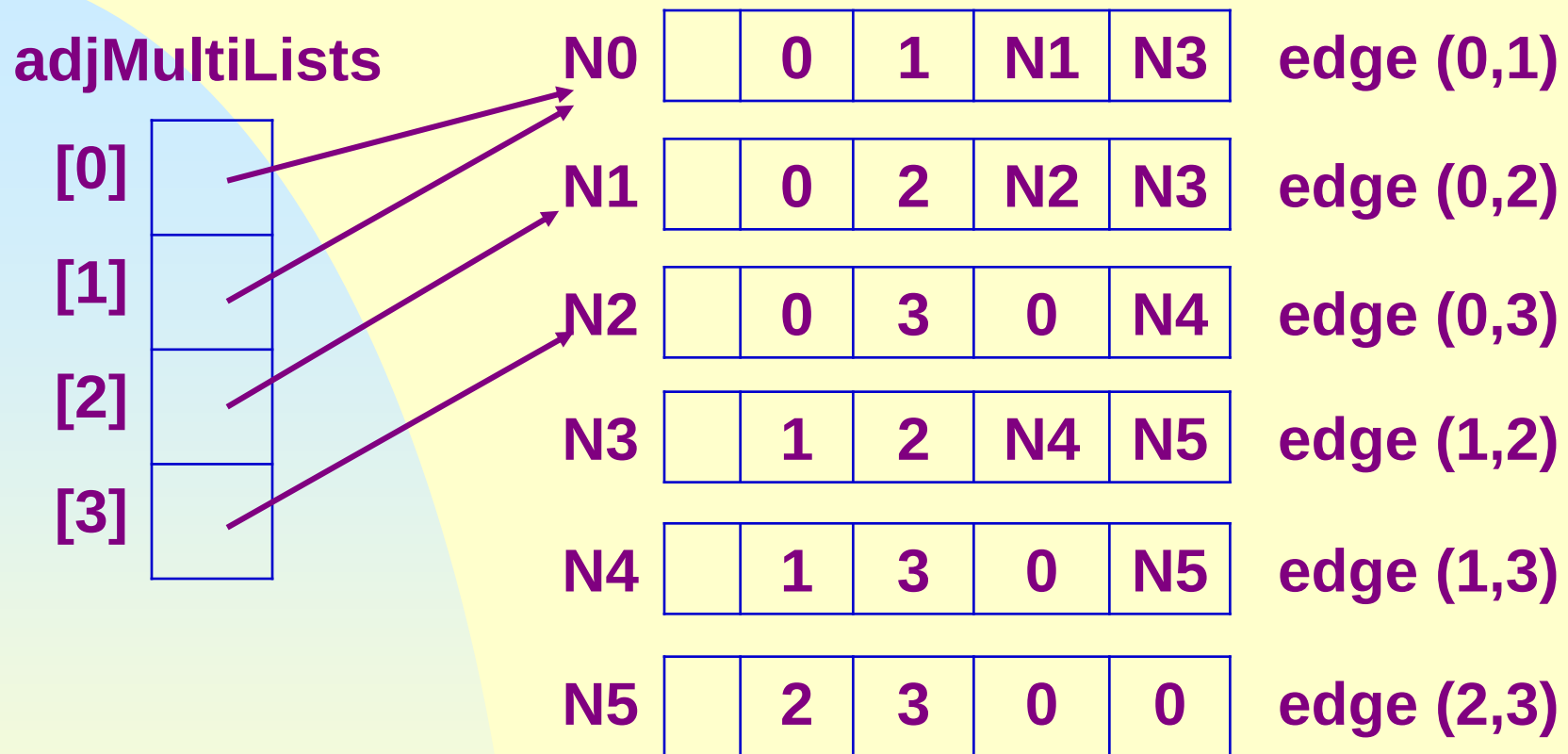
Class definitions for MgraphEdge and MGraph

```
class MGraph;  
class MGraphEdge {  
friend class MGraph;  
public:  
    MGraphEdge(int u, int v){  
        m=false;  
        vertex1=u, vertex2=v;  
        link1=link2=0;  
    }  
private:  
    bool m;  
    int vertex1, vertex2;  
    MGraphEdge *link1, *link2;  
};  
typedef MGraphEdge *EdgePtr ;
```

m	vertex1	vertex2	link1	link2
---	---------	---------	-------	-------

```
class MGraph {  
public:  
    MGraph(const int);  
private:  
    EdgePtr *adjMultiLists;  
    int n;  
    int e;  
};  
  
MGraph::MGraph(const int vertices) : e(0)  
{  
    if (vertices < 1) throw "Number of vertices must be > 0";  
    n = vertices;  
    adjMultiLists = new EdgePtr[n];  
    fill(adjMultiLists, adjMultiLists+n, (EdgePtr)0);  
}
```


Example: The adjacency multilists for G_1 :



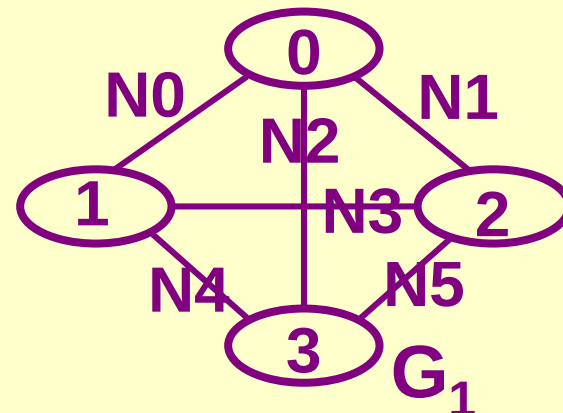
The lists are:

vertex 0: N0->N1->N2

vertex 1: N0->N3->N4

vertex 2: N1->N3->N5

vertex 3: N2->N4->N5



Some operations of MGraph

- If p points to an MGraphEdge, and one of its vertex u is given, to get the other vertex (v) :

if ($p \rightarrow \text{vertex1} == u$) $v = p \rightarrow \text{vertex2}$; else $v = p \rightarrow \text{vertex1}$;

- we can insert an edge in $O(1)$:

```
void MGraph::InsertEdge(int u, int v) {
```

```
    EdgePtr p = new MGraphEdge(u,v);
```

```
    p→m = false;
```

m	vertex1	vertex2	link1	link2
---	---------	---------	-------	-------

```
    p→link1 = adjMultiLists[u];
```

```
    p→link2 = adjMultiLists[v];
```

```
    adjMultiLists[u] = adjMultiLists[v] = p;
```

```
}
```

- **Thinking:** How to delete an edge whose two vertices are u and v ?

6.1.3.4 Weighted Edges

Edges may have weight.

- In the case of adjacency matrix, $A[i][j]$ may keep this information.
- In the case of adjacency lists, we need a weight field in the list node.
- A graph with weighted edges is called a **network**.

6.2 Elementary Graph Operations

The following discuss assumes the graphs are **undirected**, although these methods work on the directed graphs as well.

One of the frequently used operation-traversal:

- ✓ Given $G = (V, E)$, and v in $V(G)$, we wish to **visit all vertices** in G that are **reachable from v** .
- ✓ Usually, there are two ways to do this: **depth-first search** and **breadth-first search**.

6.2.1 Depth-First Search

Idea:

- ✓ First, visit the start vertex v .
- ✓ Next, an unvisited vertex w adjacent to v is selected, and a depth-first search from w is initiated.
- ✓ When all adjacent nodes of the current node have been visited, return to the previous node and continue to look for unvisited adjacent nodes.
- ✓ The search terminates when no unvisited vertex can be reached from any of the visited vertices.

```
virtual void Graph::DFS() // Driver
{
    visited=new bool[n];
    //visited is declared as a bool* data member of Graph
    fill(visited, visited + n, false);
    DFS (0); // start search at vertex 0.
    delete [ ] visited;
}
```

```
virtual void Graph::DFS (const int v) // workhorse
{
    visited[v] = true;
    for (each vertex w adjacent to v)
        if ( !visited[w] ) DFS (w);
}
```

Example 6.1

DFS: 0, 1, (0), 3, (1), 7, (3), 4, (1), (7), (back to 7), 5, 2, (0), (5), 6, (2), (7), (back to 2), (back to 5), (7), (back to 7), (6), (back to 3), (back to 1), (4), (back to 0), (2)

adjLists

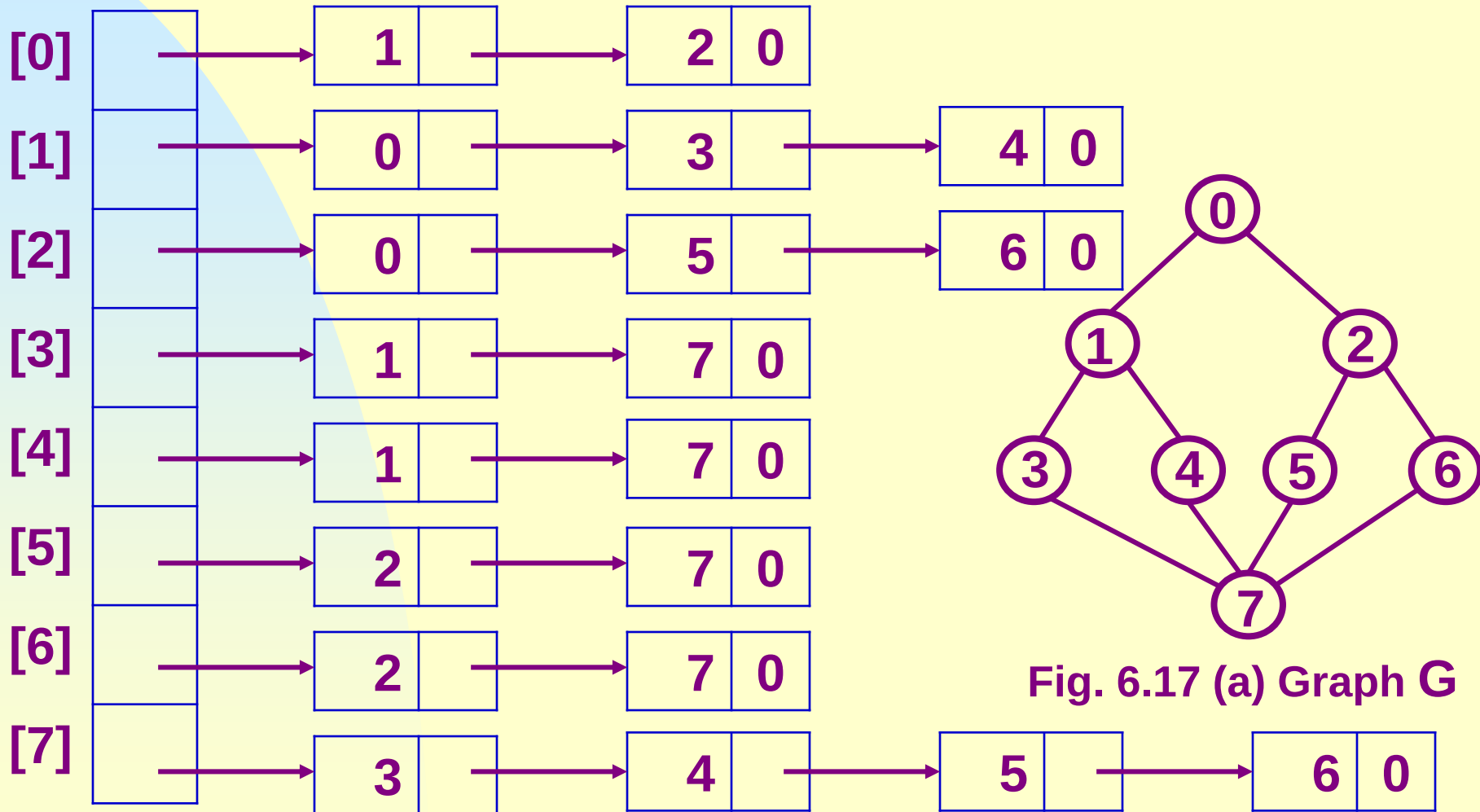


Fig. 6.17 (a) Graph G

Fig 6.17 (b) Adjacency lists of G

Analysis of DFS:

- ✓ Initiating “visited” array needs $O(n)$.
- ✓ If G is represented by its **adjacency lists**, it examines **each chain node at most once**, and **there are $2e$ list nodes**, so the total time is **$O(e+n)$** .
- ✓ If G is represented by its **adjacency matrix**, the time to **determine all vertices adjacent to v is $O(n)$** , there are n vertices to visit, so the total time is **$O(n^2)$** .

6.2.2 Breadth-First Search

Idea:

- ✓ First, visiting the start vertex **v**.
- ✓ Next, **all unvisited vertices adjacent to v** are visited.
- ✓ **Unvisited vertices adjacent to these newly visited vertices** are then visited, and so on.

```
virtual void Graph::BFS (int v)
{
    visited = new bool[n];
    //visited is declared as a bool* data member of Graph
    fill(visited, visited + n, false);
    visited[v] = true;
```

```
Queue<int> q;  
q.Push(v);  
while ( !q.IsEmpty()) {  
    v = q.Front();  
    q.Pop();  
    for (all vertices w adjacent to v)  
        if ( !visited[w] ) {  
            q.Push(w);  
            visited[w] = true;  
        }  
    } // end of while  
    delete [ ] visited;  
}
```

Example 6.2

BFS: 0, 1, 2, 3, 4, 5, 6, 7

Queue: 0=>1,2 =>2,3,4 =>3,4,5,6 =>4,5,6,7

adjLists

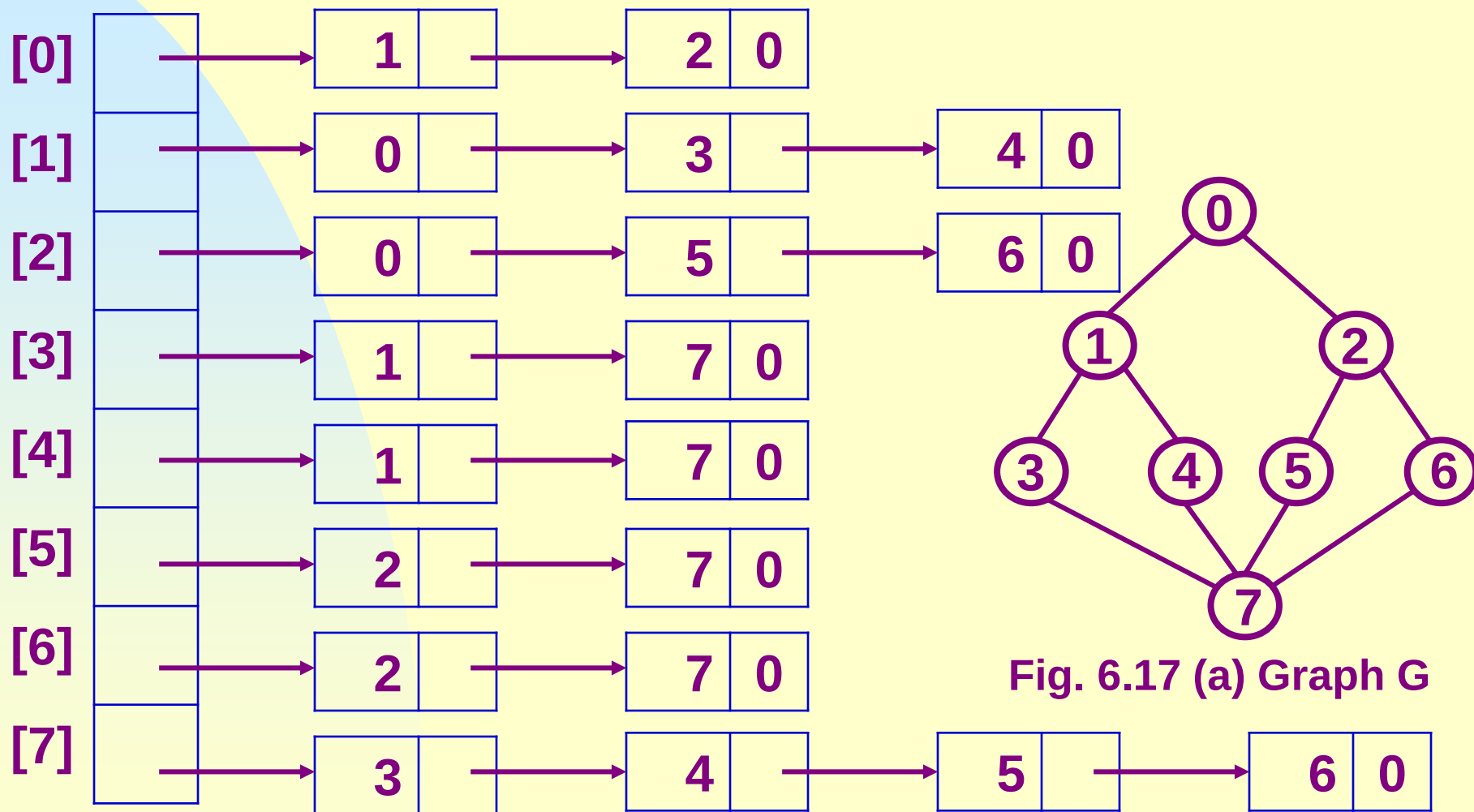


Fig. 6.17 (a) Graph G

Fig 6.17 (b) Adjacency lists of G

Analysis of BFS:

- ✓ Initiating “visited” array needs $O(n)$.
- ✓ Each visited vertex **enters the queue exactly once**, so the while loop is iterated at most **n** times.
- ✓ If **adjacency lists** are used, the loop has a total cost of $d_0 + \dots + d_{n-1} = O(e)$, so the total time is **$O(n+e)$** .
- ✓ If an **adjacency matrix** is used, the loop takes $O(n)$ time for each vertex, so the total time is **$O(n^2)$** .

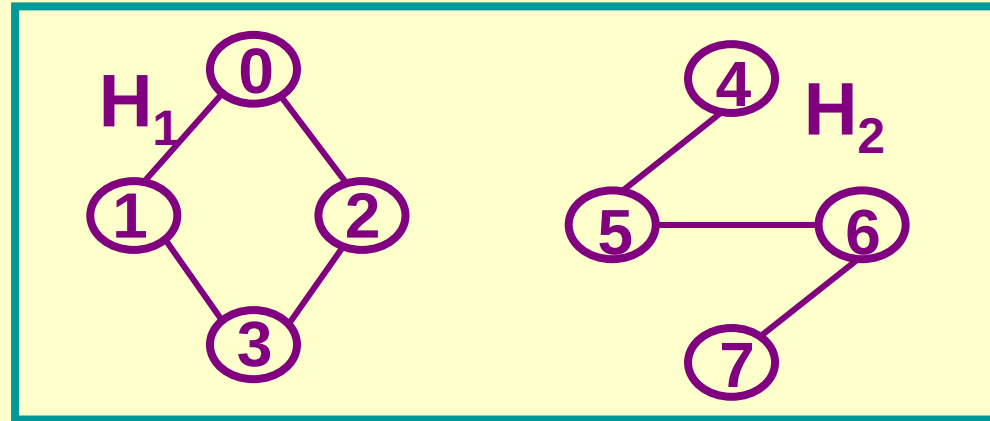
6.2.3 Connected Components

- By simply making a **DFS** or **BFS** call, one can determine whether an undirected graph G **is connected or not**:
 - ✓ If there is no unvisited vertex, G is connected.
 - ✓ If there is an unvisited vertex, G is unconnected.
- If G is an unconnected, we can make repeated **DFS** and **BFS** calls to obtain **all the connected components**.

```

virtual void Graph::Components()
{ // Determine the connected components of the graph.
  visited = new bool[n];
  fill(visited, visited+n, false);
  for (int i=0; i<n; i++)
    if (!visited[i]) {
      DFS(i); // find a component
      OutputNewComponent();
    }
  delete [ ] visited;
}

```



- Function **OutputNewComponent** output **all vertices** visited in the most recent invocation of DFS, together with **all edges** incident on them. ⁴⁶

Analysis of Components:

- ✓ If G is represented by its **adjacency lists**:
 - The time taken by **DFS** is $O(e)$.
 - The **OutputNewComponent** can be completed in $O(e)$ if **DFS** keeps a list of all newly visited vertices.
 - Since the for loop takes $O(n)$ time, the total time to generate all the connected components is $O(e+n)$.
- ✓ If G is represented by **adjacency matrix**:
 - The time taken by **DFS** is $O(n^2)$.
 - The **OutputNewComponent** can be completed in $O(n^2)$ if **DFS** keeps a list of all newly visited vertices.
 - The total time to generate all the connected components is $O(n^2)$.

6.2.4 Spanning Trees

Any tree consisting solely of edges in G and including **all vertices** in G is called a **spanning tree**.

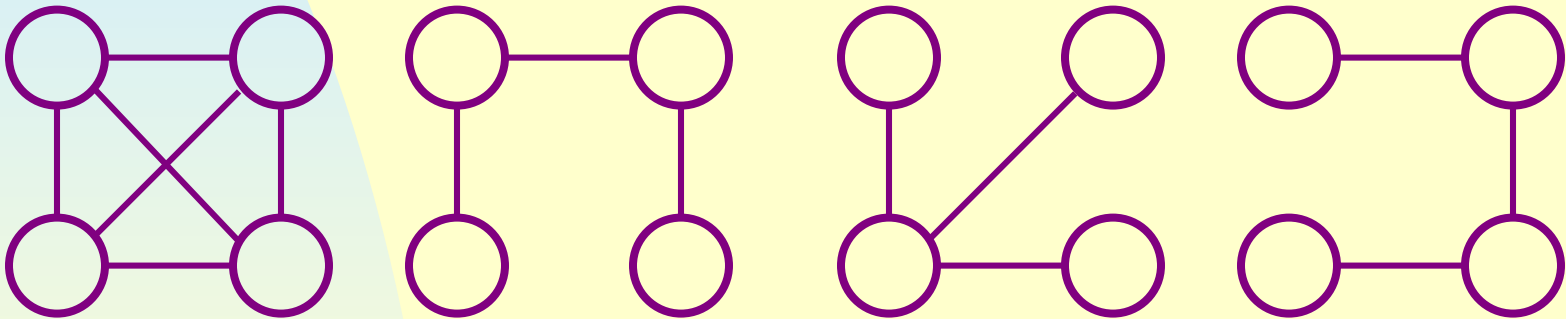


Fig. 6.18 A complete graph and three of its spanning trees

Depth-first spanning tree and Breadth-first spanning tree

- When G is connected, a **DFS** or **BFS** starting at any vertex leads to a tree T . An edge of T can be obtained by visiting an unvisited vertex during the search.
 - **Depth-first spanning tree** --- the spanning tree resulting from a depth-first search
 - **Breadth-first spanning tree** --- the spanning tree resulting from a breadth-first search
- T may be obtained by inserting the statement “ $T = T \cup \{(v, w)\}$ ” in the **if clauses** of **DFS** or **BFS**.

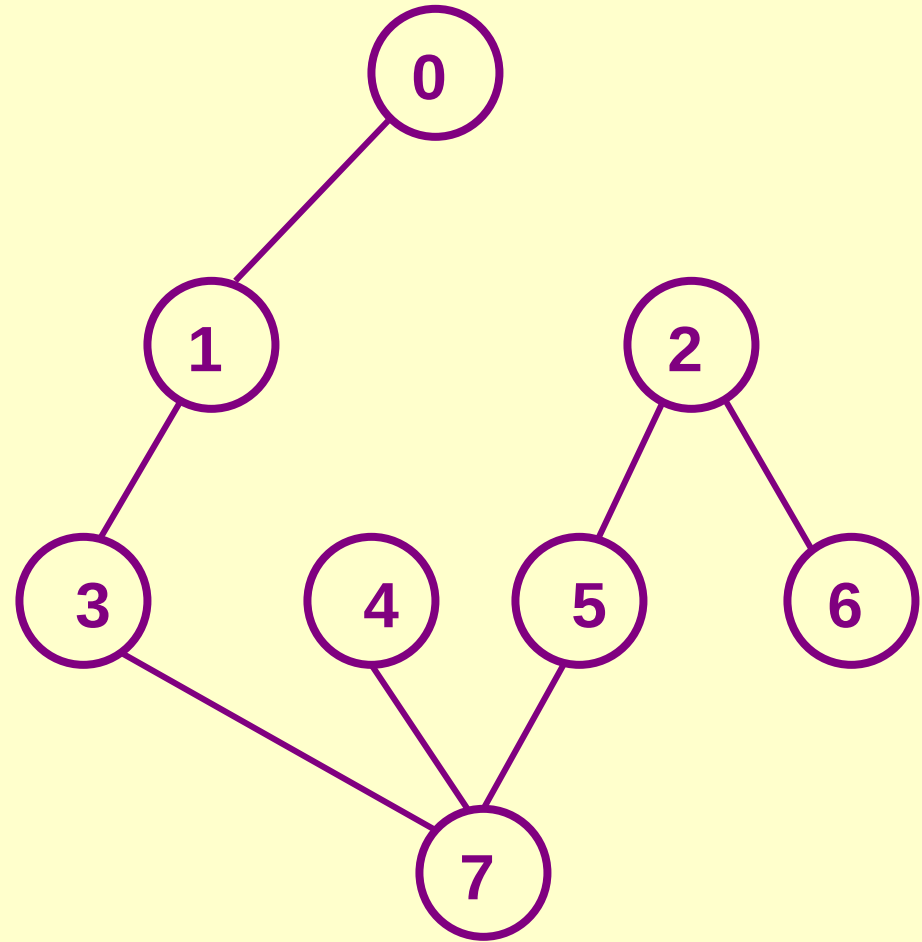
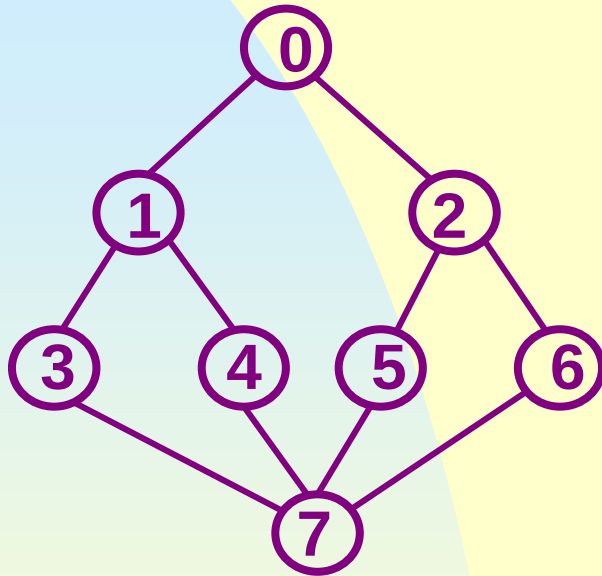


Fig. 6.19 (a) **DFS(0) spanning tree** for graph of Fig.6.17

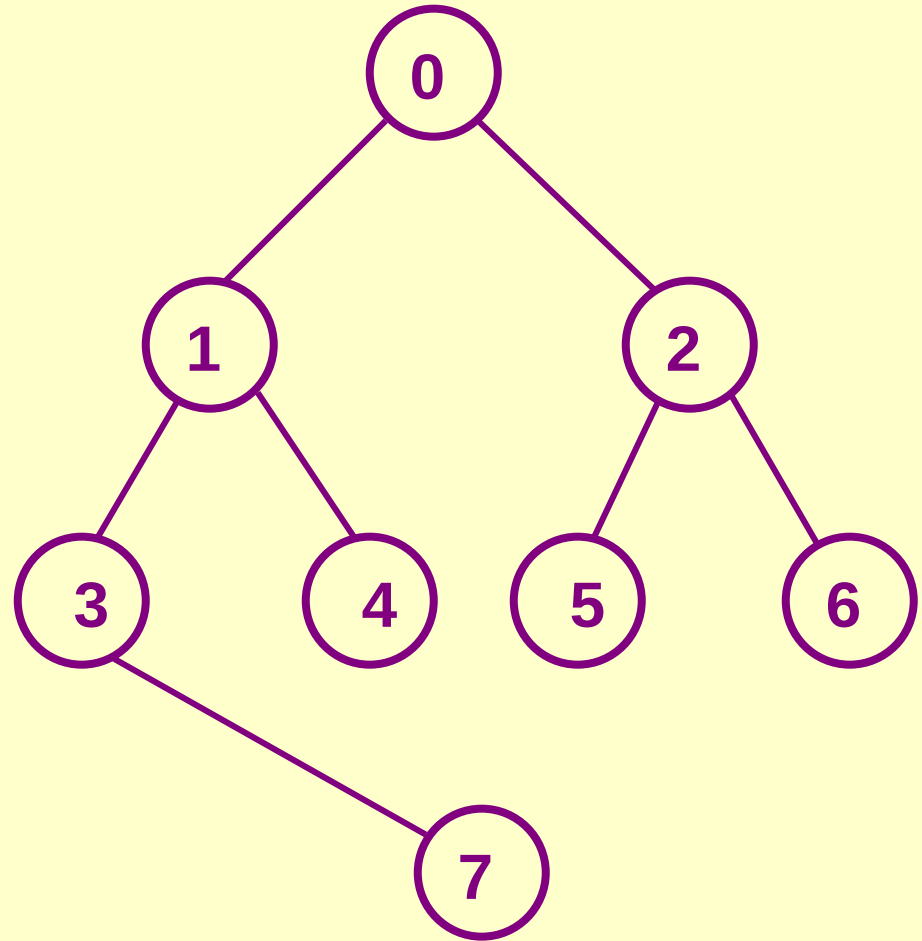
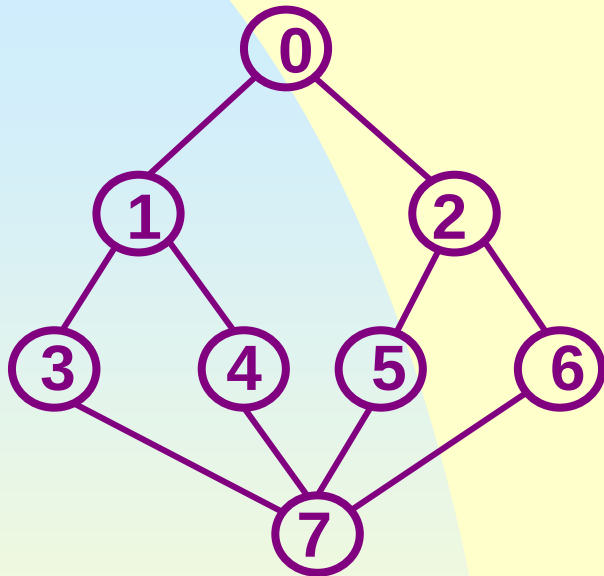


Fig. 6.19 (b) **BFS(0) spanning tree** for graph of Fig.6.17

Properties and applications of spanning tree

- **Property 1:** If a nontree edge (v, w) is introduced into any spanning tree T , then a **cycle is formed**.
- **Application:** Using this property of spanning trees to obtain an **independent set of circuit equations for electrical network**.
- **Example 6.3 [Creation of circuit equation] :** To obtain the circuit equations:
 - ✓ We first obtain a spanning tree for the electrical network.
 - ✓ Then we introduce the nontree edges into the spanning tree one at a time. The introduction of each such edge produces a **cycle**.
 - ✓ Next we use Kirchhoff's second law ($\sum U=0$) on this cycle to obtain a circuit equation.

■ These cycles obtained in this way are independent. Thus the circuit equations are also independent.

- ✓ Among all cycles formed by adding a nontree edge to the spanning tree, we cannot obtain any one by the linear combination of the remaining cycles, since each cycle contains a nontree edge that is not contained in any other cycle.
- ✓ In fact, all cycles formed by adding a nontree edge to the spanning tree form a cycle basis. This means that all other cycles in the graph(for example, a cycle obtained by introducing two nontree edges) can be constructed by the linear combination of the cycles in the basis.

- **Property 2:** A spanning tree is a **minimal subgraph** G' of G such that $V(G')=V(G)$, and G' is connected. It means that a spanning tree of a graph with n vertices must have $n-1$ edges.
- ✓ Any **connected graph** with n vertices must have **at least $n-1$ edges**, and all connected graphs with $n-1$ edges are **trees**. Therefore we conclude that a spanning tree of a graph with n vertices has $n-1$ edges.
- ✓ Constructing minimal subgraphs has frequent applications.

- **Application example:** Suppose that n vertices of a graph G represent n cities, and the edges represent communication links between cities. If these cities can communicate one another, then the minimum number of links is $n-1$, and the communication links forms a tree. Constructing different spanning trees of G produces different feasible choices.
- **Further consideration:** Since the cost of constructing communication links between cities is rarely the same, how to select a spanning tree with lowest total cost?

6.3 Minimum-Cost Spanning Trees

- A design strategy called **greedy method** can be used to obtain a minimum-cost spanning tree.
- The greedy method is a strategy used to solve optimization problems.
- It makes **locally optimal choices** at each step with the hope of finding a global optimum solution.
 - ✓ At each step, it makes the best decision (**using some criterion and satisfying the constraints**).
 - ✓ We cannot change this decision later.

- The greedy method is suitable for the problem that has the following properties: **Greedy Choice Property** and **Optimal Substructure**.
 - ✓ The **greedy choice property** means that an optimal solution can be achieved by making the best choice at each step without reconsidering previous steps.
 - ✓ The **optimal substructure property** indicates that the optimal solution to the problem corresponds to the optimal solutions of its subproblems.
- **Limitations:** This method do not reconsider previous choices, as it makes decisions based on current information without looking ahead. **It does not always produce the optimal solution.**

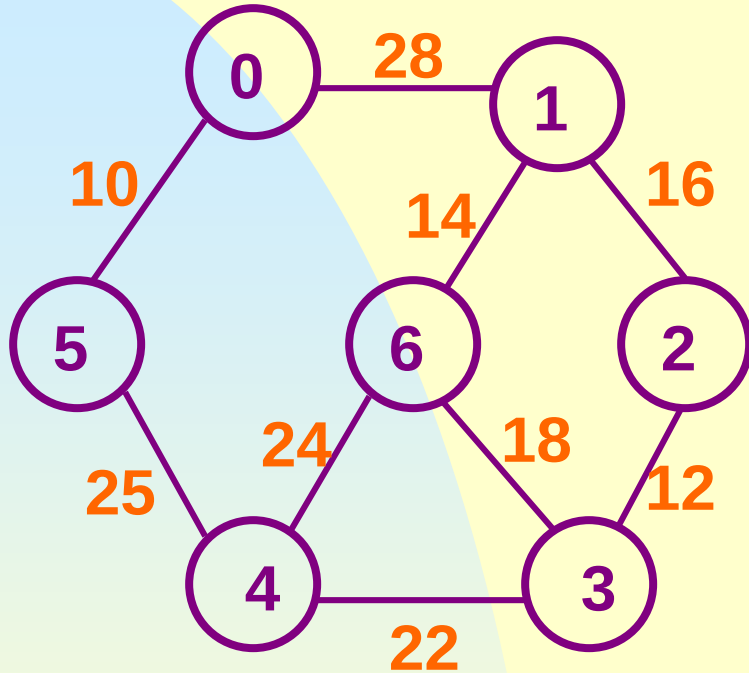
■ To construct a minimum-cost spanning tree, we use a **least-cost criterion**, with the following **constraints** at each step:

- (1) We must use only edges within the graph G .
- (2) We must use exactly **$n-1$ edges**.
- (3) We may not use edges that produce a cycle.

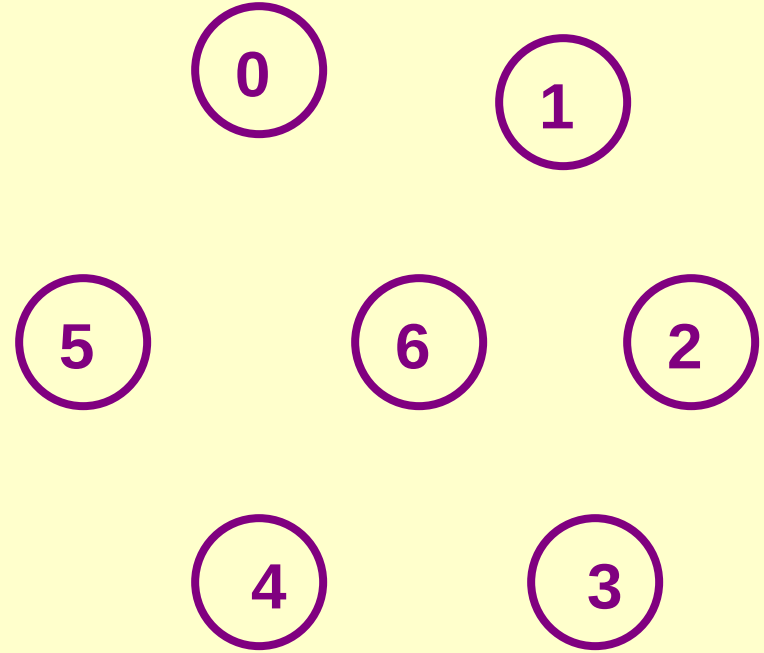
6.3.1 Kruskal's Algorithm

- Kruskal's algorithm builds the minimum-cost spanning tree by adding edges to the edge set T one at a time.
- The remaining edges are considered in non-decreasing order of their cost.
- A remaining edge with minimum cost is added to T if it does not form a cycle with the edges that are already in T . Otherwise it is discarded.
- Since G is connected and has $n > 0$ vertices, exactly $n-1$ edges will be selected for inclusion in T .

Example 6.4:

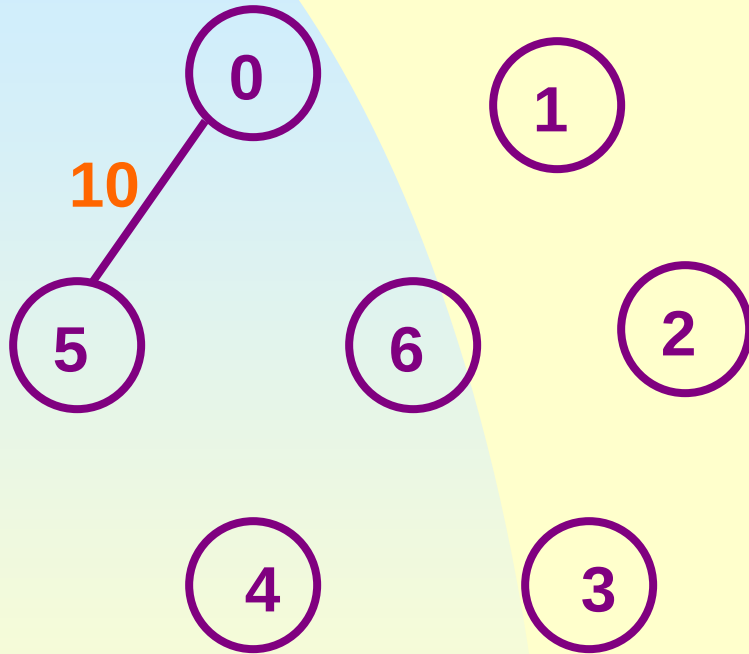


(a) G



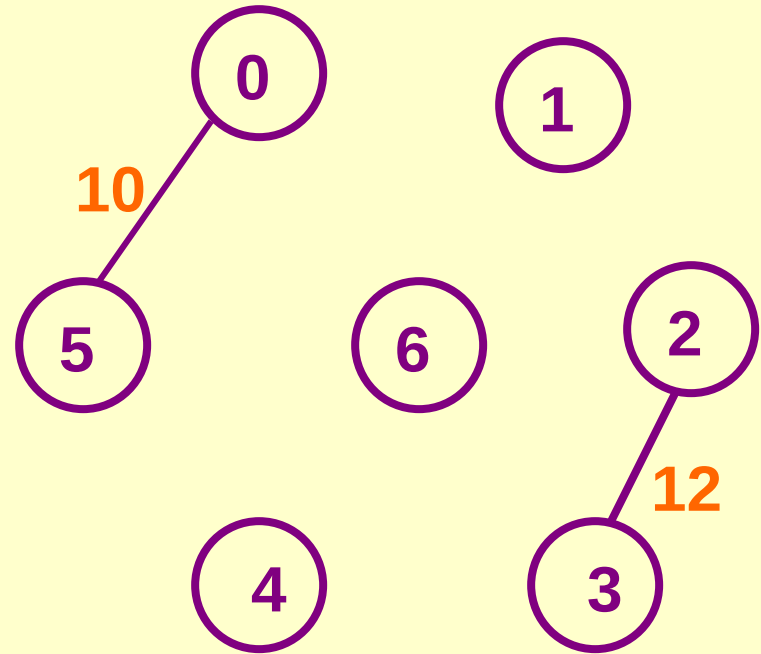
(b)

The cost of remaining edges:
10, 12, 14, 16, 18, 22, 24, 25, 28



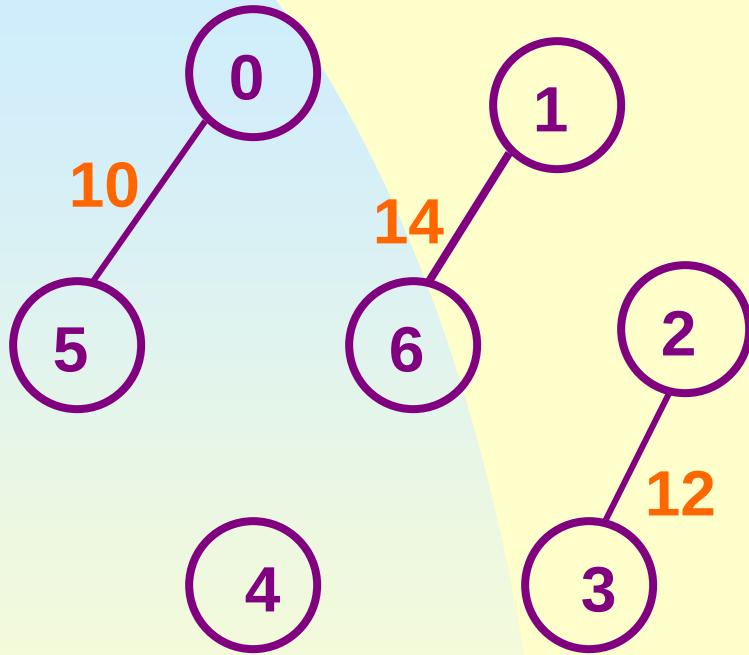
(c)

The cost of remaining edges:
12, 14, 16, 18, 22, 24, 25, 28



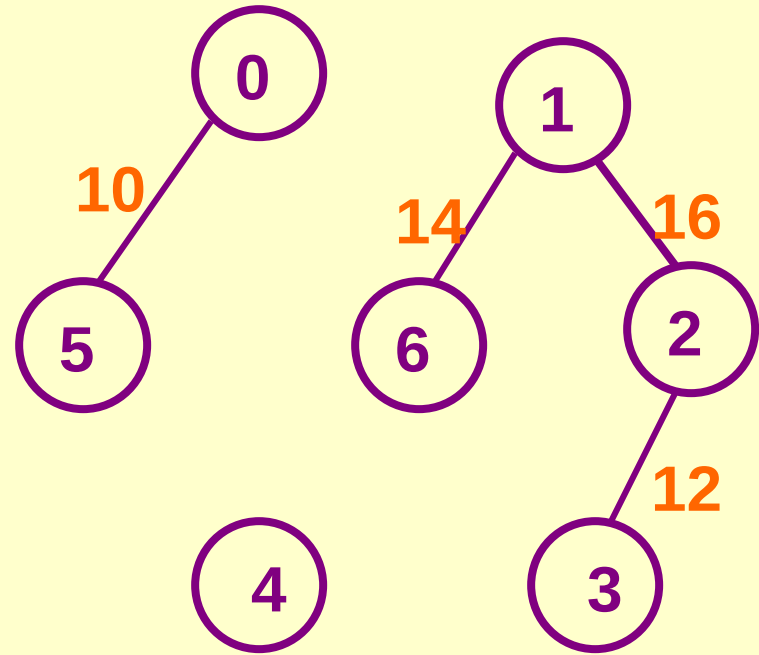
(d)

The cost of remaining edges:
14, 16, 18, 22, 24, 25, 28



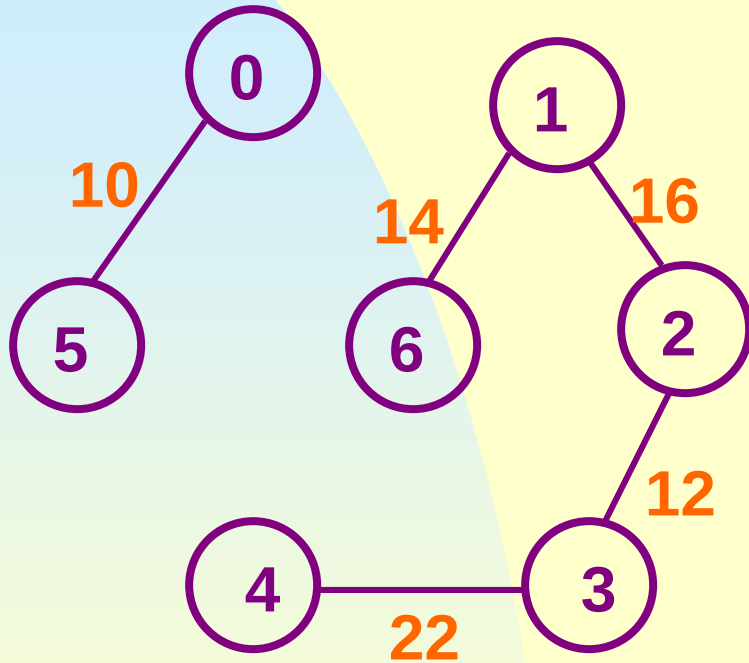
(e)

The cost of remaining edges:
16, 18, 22, 24, 25, 28



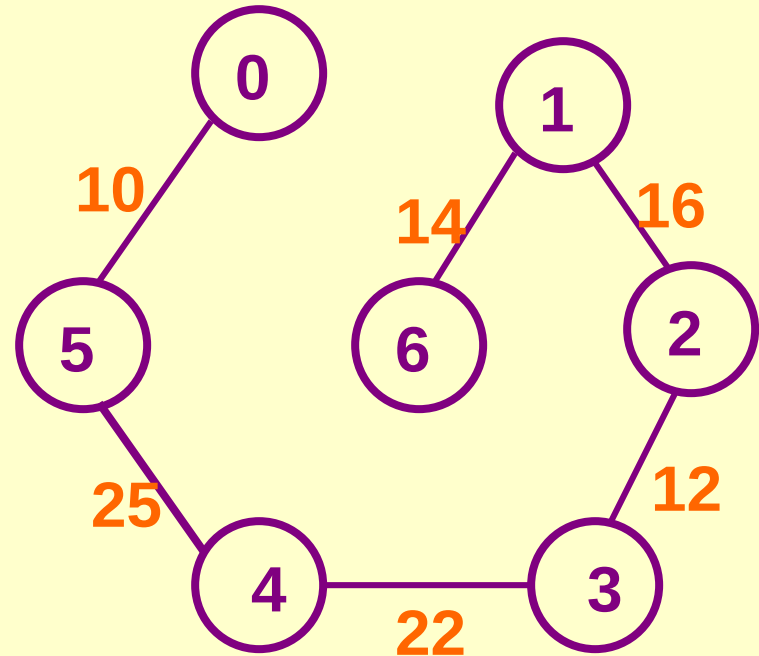
(f)

The cost of remaining edges:
18—(3,6), **22**, 24—(4,6), 25, 28



(g) (3,6) discarded

The cost of remaining edges:
24—(4,6), **25**, 28



(h) (4, 6) discarded

Given $G = (V, E)$, we have the Kruskal's algorithm:

```
1   $T = \emptyset$ ;  
2  while (( $T$  contains less than  $n-1$  edges) && ( $E$  not Empty)) {  
3      choose an edge  $(v, w)$  from  $E$  of lowest cost;  
4      delete  $(v, w)$  from  $E$ ;  
5      if (( $v, w$ ) does not create a cycle in  $T$ ) add  $(v, w)$  to  $T$ ;  
6      else discard  $(v, w)$ ;  
7  }  
8  if ( $T$  contains fewer than  $n-1$  edges)  
    cout<<"no spanning tree"<<endl;
```

■ **Key points should be considered:**

How to choose an edge (v, w) from E of lowest cost?

How to determine whether (v, w) creates a cycle in T ?

How to choose an edge (v, w) from E of lowest cost?

- To choose an edge (v, w) from E of lowest cost efficiently, edges in E can be organized as a **min heap**, as it permits the next edge to be chosen or deleted in **$O(\log e)$** time.
- Note that:
 - ✓ The construction of the heap itself takes **$O(e)$** (Refer to Section 7.6).
 - ✓ If the graph is represented by **adjacent matrix or adjacent list**, its representation should be **converted to edges array** to build a min heap, where each edge is expressed as **(u, v, weight)** .

How to determine whether (v, w) leads to a cycle in T ?

- ✓ To determine whether (v, w) creates a cycle in T efficiently, we let all vertices in the same connected component of T be placed into a set. Then two vertices v and w are connected in T iff they are in the same set.
- ✓ For an edge (v, w) , we can use “Find” (Chapter 5) to determine whether v and w are already connected. If v and w are already connected, then the edge (v, w) will create a cycle, so it will be discarded. If v and w are not connected, (v, w) will be added to T , and the two sets containing u and v respectively are unioned by “Union” (Chapter 5).

Analysis of Kruskal's algorithm:

- ✓ The time complexity to choose an edge (v, w) from E of lowest cost (lines 3 and 4) in the worst case is $O(e \log e + e)$ (Note: It is not $O(n \log e + e)$ since every edge may be considered).
- ✓ Determining whether (v, w) creates a cycle in T (lines 5 and 6) is also efficient.
- ✓ So the total time in the worst case is $O(e \log e)$.
- ✓ Note $n-1 \leq e \leq n^2$, so $O(\log e) = O(\log n)$.

Correctness:

Theorem 6.1: Let G be any undirected, connected graph, **Kruskal's algorithm** generates a minimum-cost spanning tree.

The proof is left as a self-study exercise.

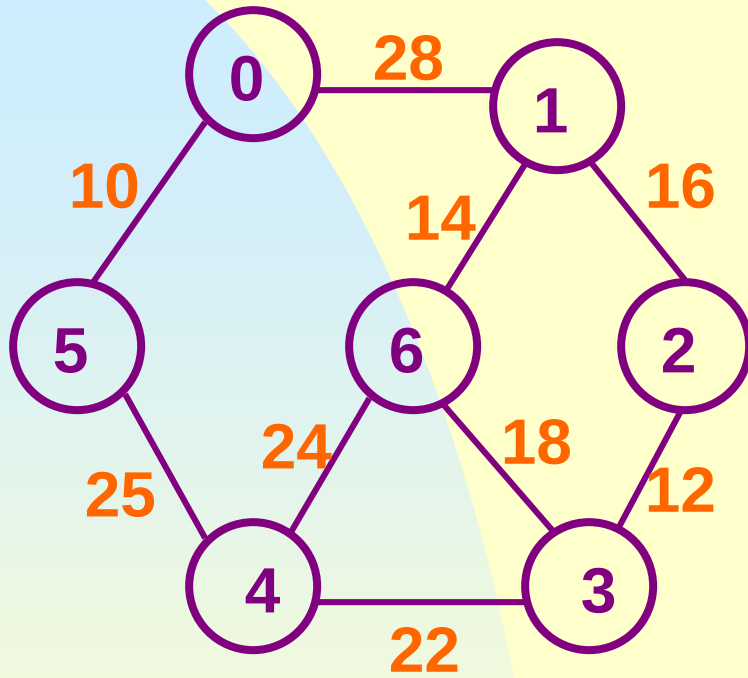
6.3.1 Prim's Algorithm

- Like Kruskal's, the minimum-cost spanning tree is built **by adding edges**.
- However, at all times, the set of selected edges **forms a tree**, but the set of selected edges in Kruskal's algorithm **forms a forest**.

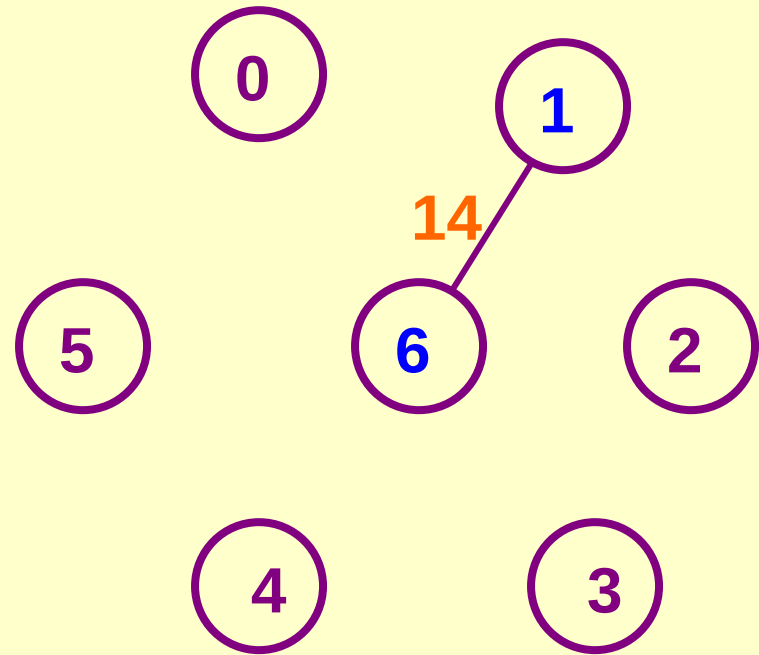
Idea:

- Begin with a tree T that contains an **arbitrary single vertex**.
- **Add a least-cost edge (u,v) to T such that $T \cup \{(u,v)\}$ is kept as a tree**. Note that edge (u,v) is such that **exactly one of u and v is in T** .
- Repeat this edge-addition step until T contains **$n-1$ edges**.

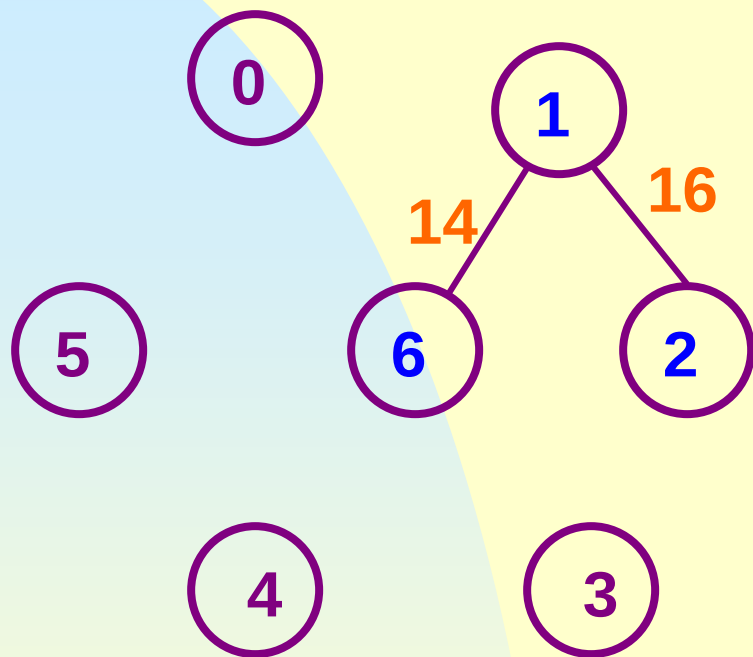
Example: stages in Prim's algorithm (starting at vertex 6)



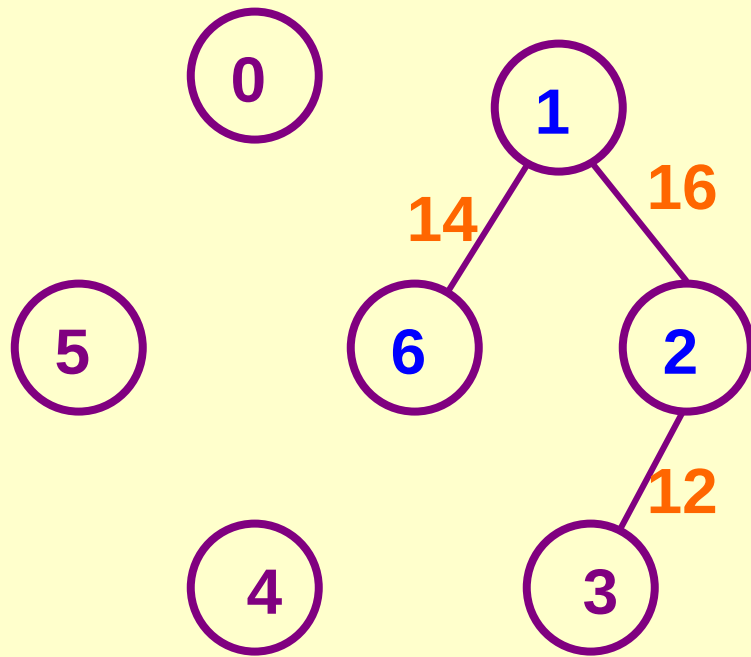
(a) G



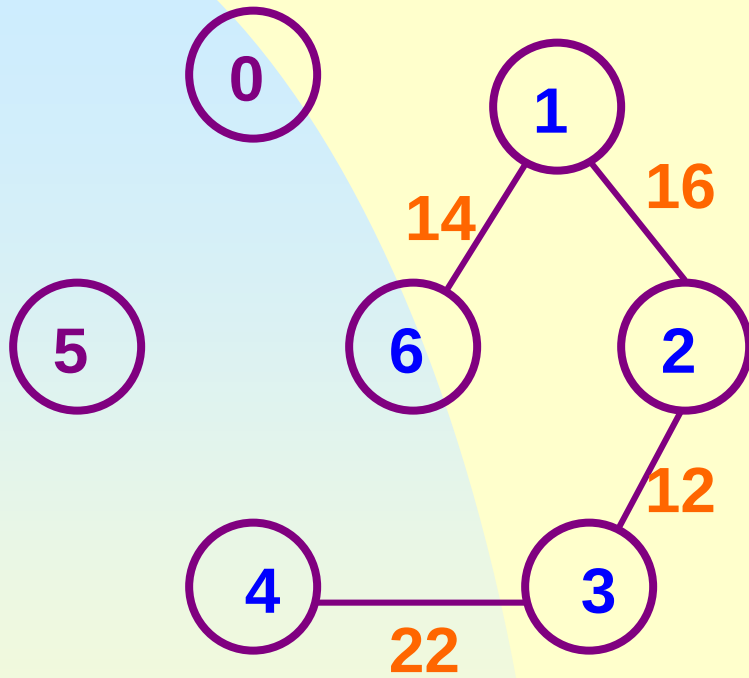
(b)



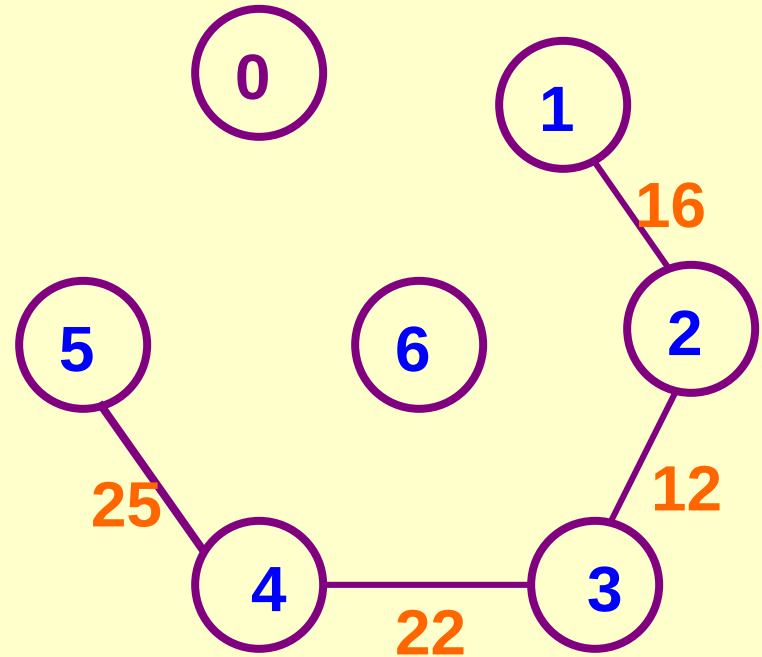
(c)



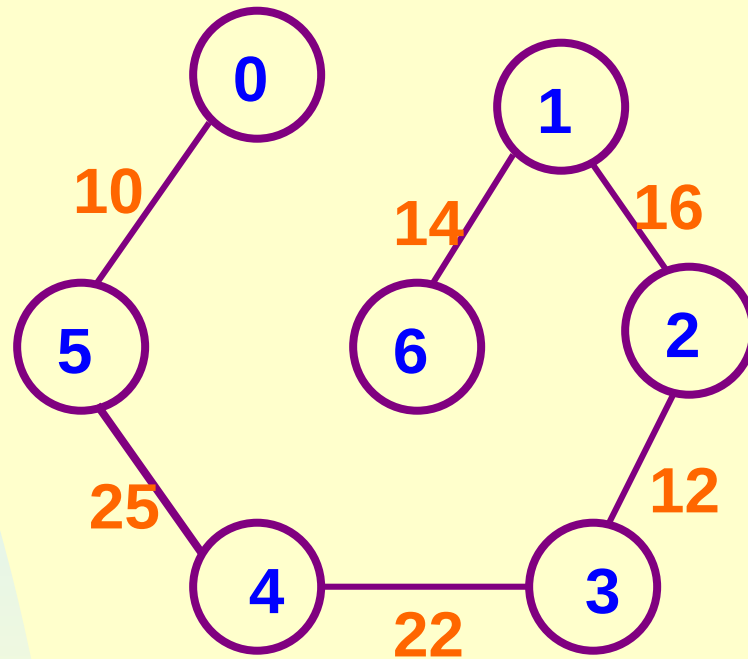
(d)



(e)



(f)



(g)

Given $G = (V, E)$, we have the Prim's algorithm:

// Assume that G has at least one vertex.

```
1 TV = {0}; //start with vertex 0 and no edges
2 for(T =  $\emptyset$ ; T contains fewer than n-1 edges); add (u,v) to T{
3     Let (u,v) be a least-cost edge such that  $u \in TV$  but  $v \notin TV$ ;
4     if (there is no such edge) break;
5     add v to TV;
6 }
7 if (T contains fewer than n-1 edges)
    cout<<"no spanning tree"<<endl;
```

■ Key points should be considered:

How to choose an edge (u, v) from E that have a least-cost edge such that $u \in TV$ but $v \notin TV$

How to choose an edge (u, v) from E that have a least-cost edge such that $u \in TV$ but $v \notin TV$

- ✓ To find the overall minimal cost, for each vertex $v \notin TV$ (v is viewed as fixed), we keep the vertex $\text{near}(v)$ and the $\text{cost}(\text{near}(v), v)$, such that $\text{near}(v)$ is a vertex $\in TV$ and $\text{cost}(\text{near}(v), v)$ is minimal among all possible choices. (we assume that $\text{cost}(w, v) = \infty$ if $(w, v) \notin E$).
- ✓ Initially, there is only one vertex in TV , so the cost between a vertex $\notin TV$ and this vertex is also the minimal.
- ✓ When a new vertex u which is just now added to TV , we should update $\text{near}(v)$ and $\text{cost}(\text{near}(v), v)$ and for every v that is not yet in TV :

// if the graph is represented by adjacent matrix

for (every v that is not yet in TV)

if ($\text{cost}(u, v) < \text{cost}(\text{near}(v), v)$)

{

$\text{near}(v) = u;$

$\text{cost}(\text{near}(v), v) = \text{cost}(u, v);$

}

// if the graph is represented by adjacent lists

for (every v that is adjacent to u but not yet in TV)

if ($\text{cost}(u, v) < \text{cost}(\text{near}(v), v)$)

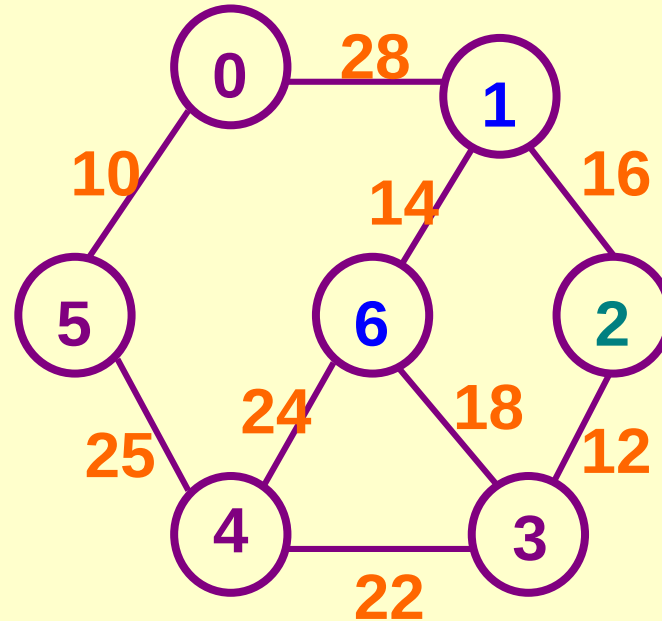
{

$\text{near}(v) = u;$

$\text{cost}(\text{near}(v), v) = \text{cost}(u, v);$

}

An example to update $\text{cost}(\text{near}(v), v)$ and $\text{near}(v)$



The case that 2 is
added to $TV = \{6, 1, 2\}$

- $u=2, v=0$: $\text{near}(v)=1$, $\text{cost}(u,v)=\infty$, $\text{cost}(\text{near}(v),v)=28$, $\text{near}(v)=1$
- $u=2, v=3$: $\text{near}(v)=6$, $\text{cost}(u,v)=12$, $\text{cost}(\text{near}(v),v)=18$, $\text{near}(v)=2$
- $u=2, v=4$: $\text{near}(v)=6$, $\text{cost}(u,v)=\infty$, $\text{cost}(\text{near}(v),v)=24$, $\text{near}(v)=6$
- $u=2, v=5$: $\text{near}(v)=1$, $\text{cost}(u,v)=\infty$, $\text{cost}(\text{near}(v),v)=\infty$, $\text{near}(v)=1$

Analysis of Prim's Algorithm:

Prim's algorithm has the time complexity $O(n^2)$ if the graph is represented by adjacent matrix:

- ✓ When we determine a new edge to add to T , the time to find the overall minimal cost is also $O(n)$.
- ✓ When a new vertex v is added to TV , the time to update cost and $\text{near}(v)$ is $O(n)$.
- ✓ There are $n-1$ edges to add to T , so the total time is $O(n^2)$.

Analysis of Prim's Algorithm(cont'd) :

Prim's algorithm has the time complexity $O(n^2)$ if the graph is represented by adjacent lists:

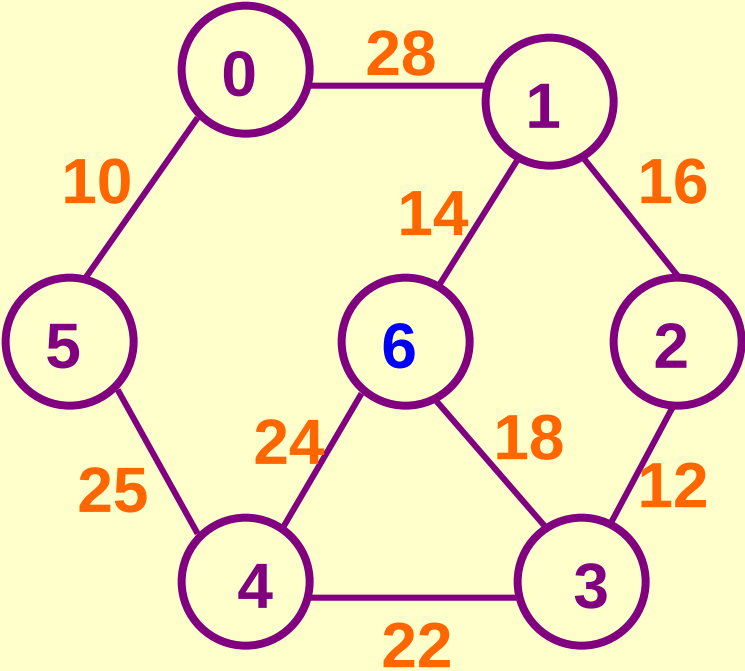
- ✓ When we determine a new edge to add to T , the time to find the overall minimal cost is $O(n)$.
- ✓ When a new vertex v is added to T , the time to update cost and $\text{near}(v)$ is $O(d_v)$.
- ✓ There are $n-1$ edges to add to T , so the total time is $O(n^2)$.

■ **Thinking:** Can we choose the overall minimal cost using min-heap if the graph is represented by adjacent lists?

Correctness:

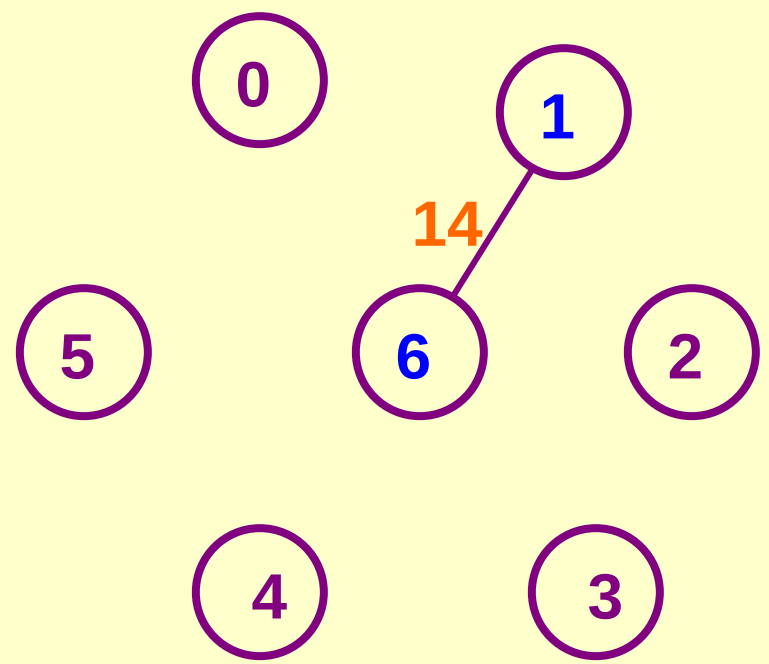
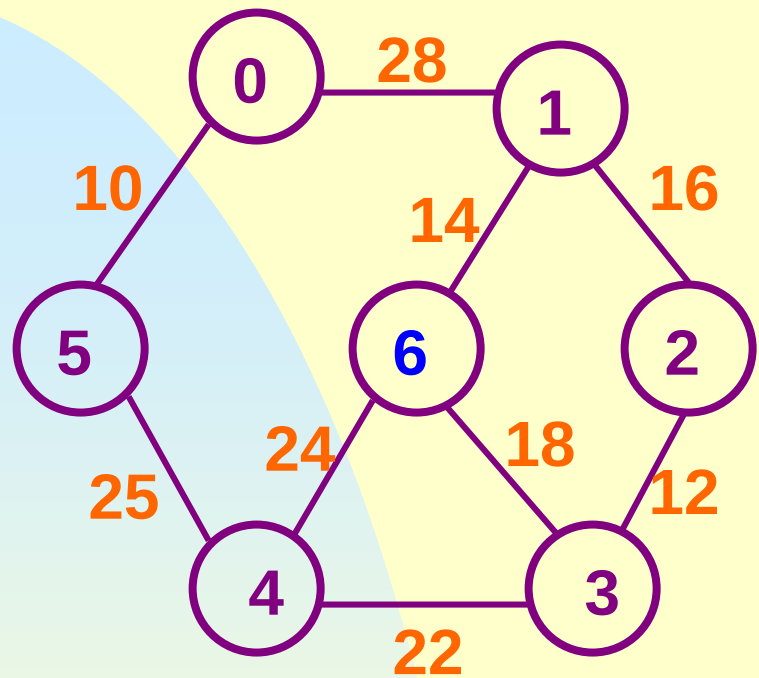
Establishing the correctness of Prim's algorithm is left an exercise.

vertex 6 is chosen(Initial)



vertex	0	<u>1</u>	2	3	4	5	6
TV	0	0	0	0	0	0	1
near	6	6	6	6	6	6	6
cost	∞	<u>14</u>	∞	18	24	∞	/

vertex 1 is added to TV

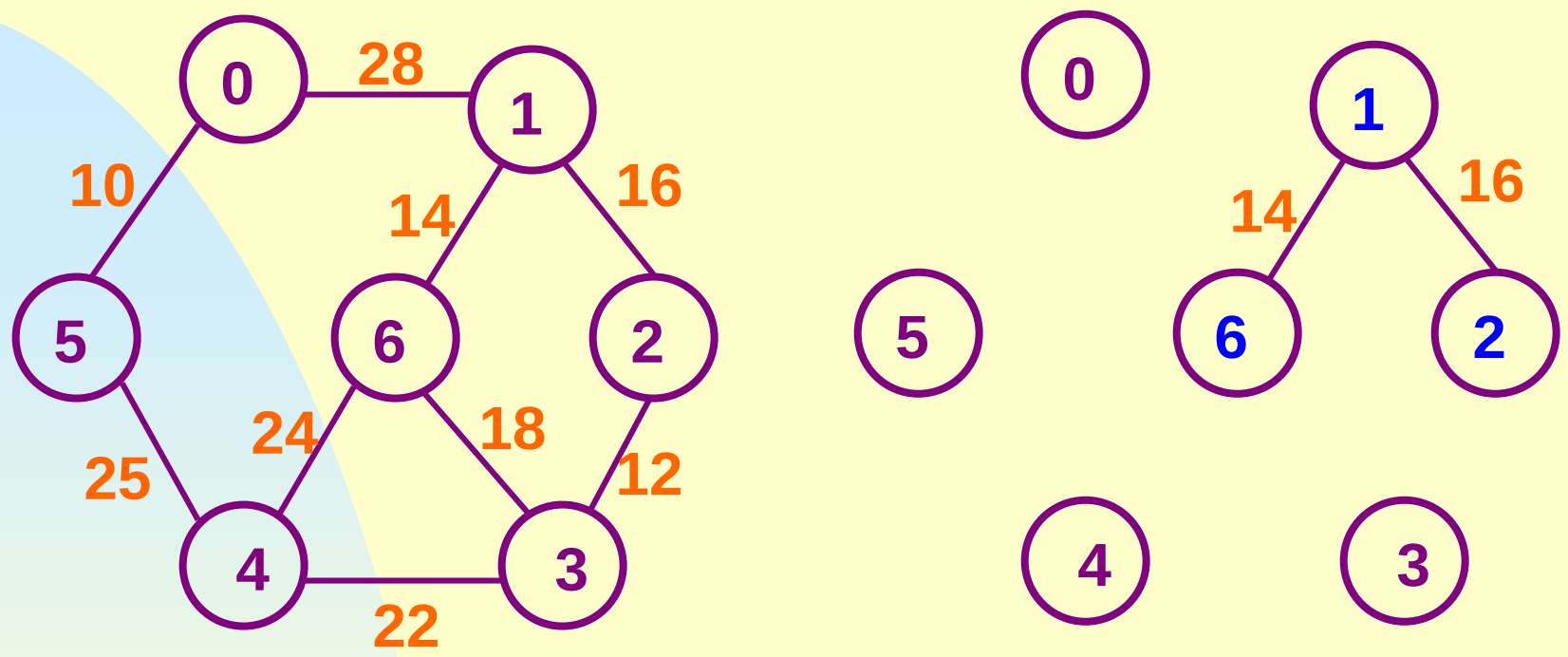


u=1

vertex	0	1	<u>2</u>	3	4	5	6
TV	0	1	0	0	0	0	1
near	1	6	1	6	6	6	6
cost	28	14	<u>16</u>	18	24	∞	/

if (cost(u,v) < cost(near(v),v)), update

vertex 2 is added to TV

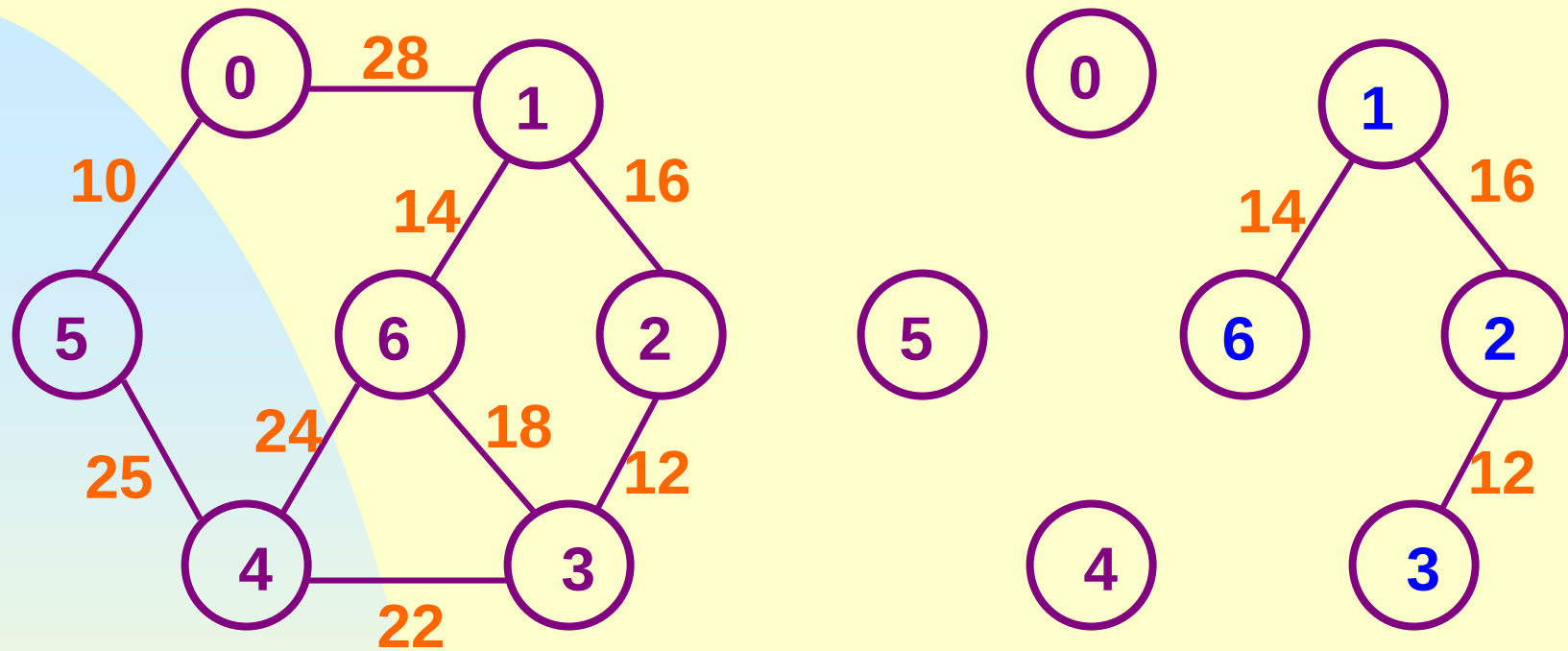


u=2

vertex	0	1	2	<u>3</u>	4	5	6
TV	0	1	1	0	0	0	1
near	1	6	1	2	6	6	6
cost	28	14	16	<u>12</u>	24	∞	/

if (cost(u,v) < cost(near(v),v)), update

vertex 3 is added to TV

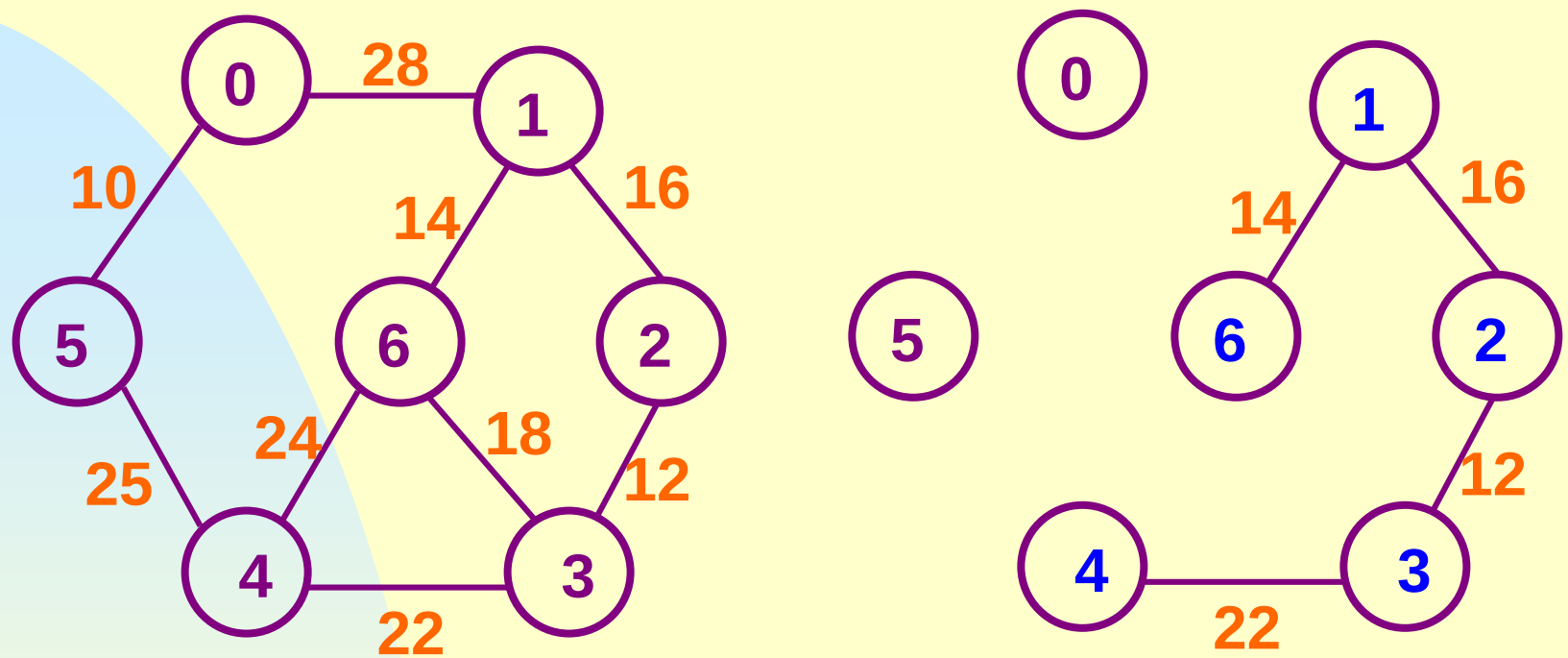


u=3

vertex	0	1	2	3	<u>4</u>	5	6
TV	0	1	1	1	0	0	1
near	1	6	1	2	3	6	6
cost	28	14	16	12	<u>22</u>	∞	/

if (cost(u,v) < cost(near(v),v)), update

vertex 4 is added to TV

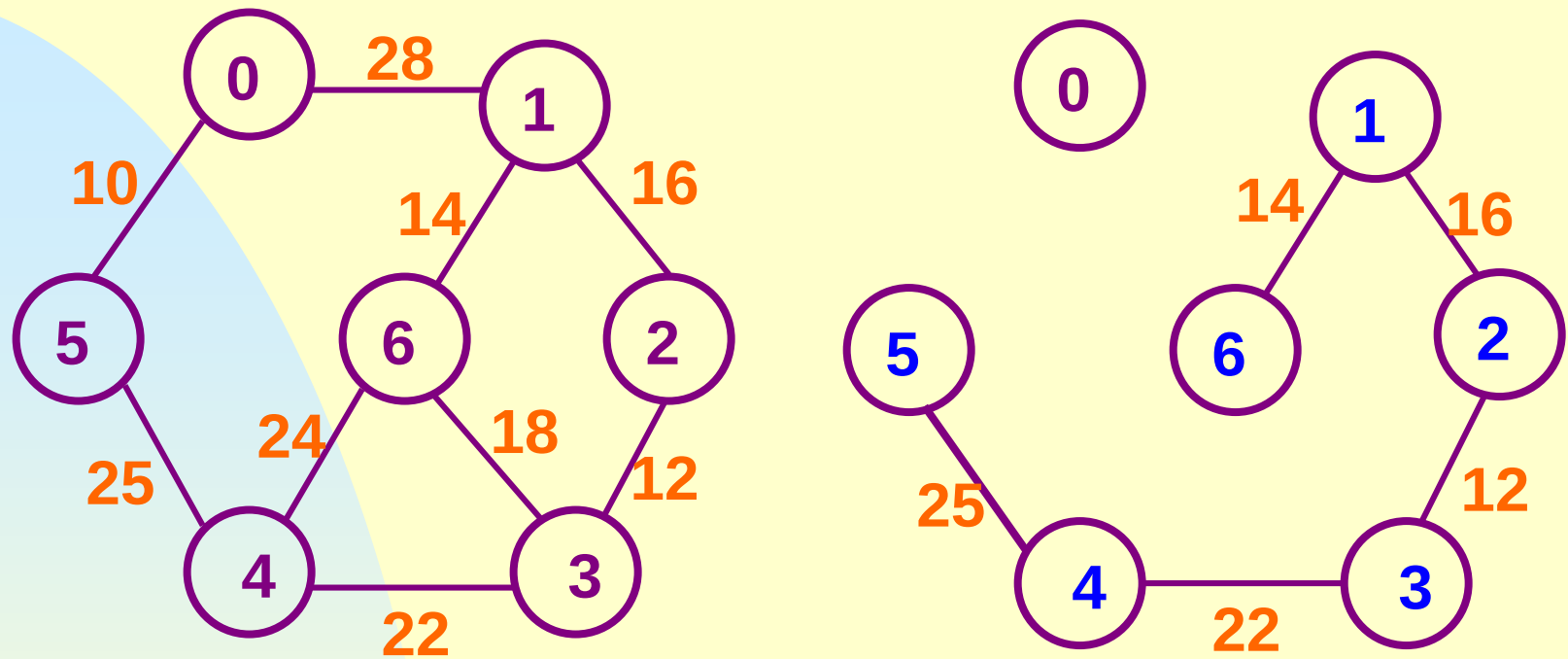


u=4

vertex	0	1	2	3	4	5	6
TV	0	1	1	1	1	0	1
near	1	6	1	2	3	4	6
cost	28	14	16	12	22	<u>25</u>	/

if (cost(u,v) < cost(near(v),v)), update

vertex 5 is added to TV

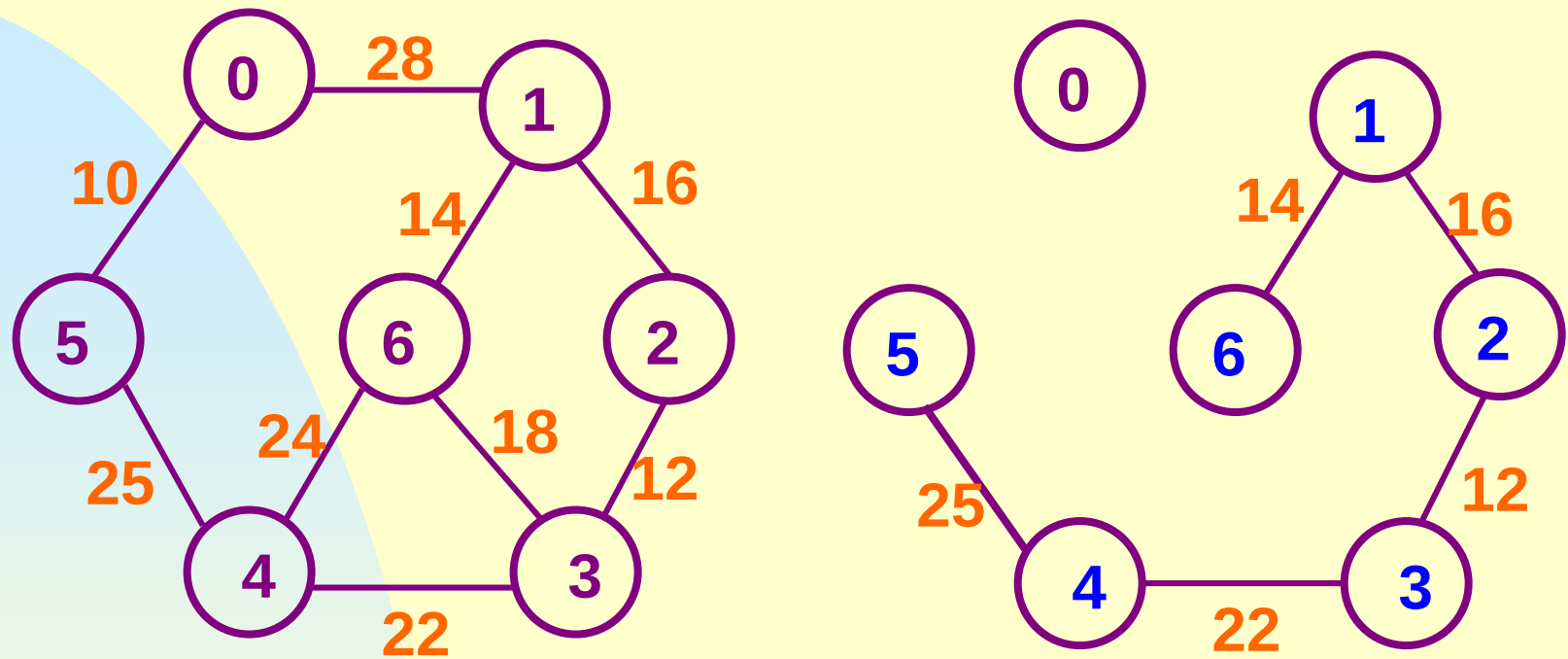


u=5

vertex	<u>0</u>	1	2	3	4	5	6
TV	0	1	1	1	1	1	1
near	5	6	1	2	3	4	6
cost	<u>10</u>	14	16	12	22	25	/

if (cost(u,v) < cost(near(v),v)), update

vertex 0 is added to TV (finished)



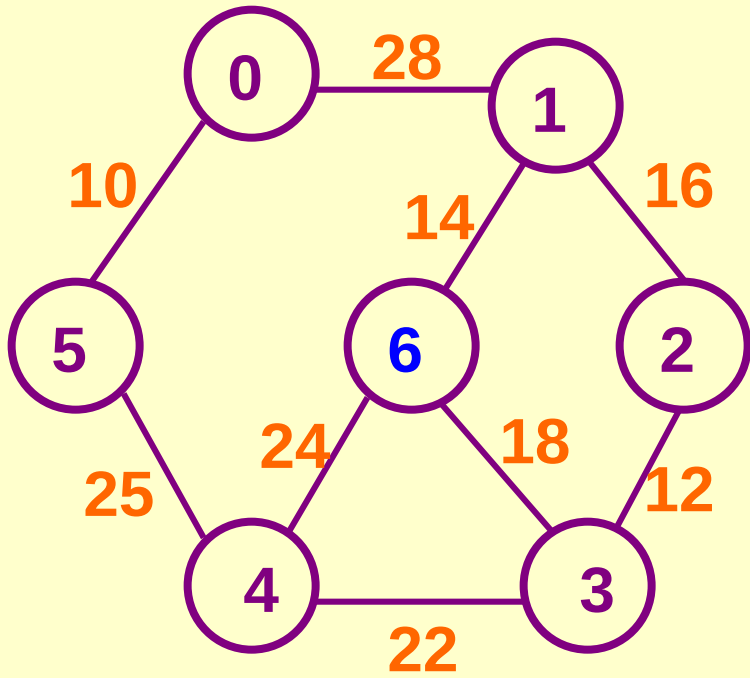
u=0

vertex	0	1	2	3	4	5	6
TV	1	1	1	1	1	1	1
near	5	6	1	2	3	4	6
cost	10	14	16	12	22	25	/

Improvement:

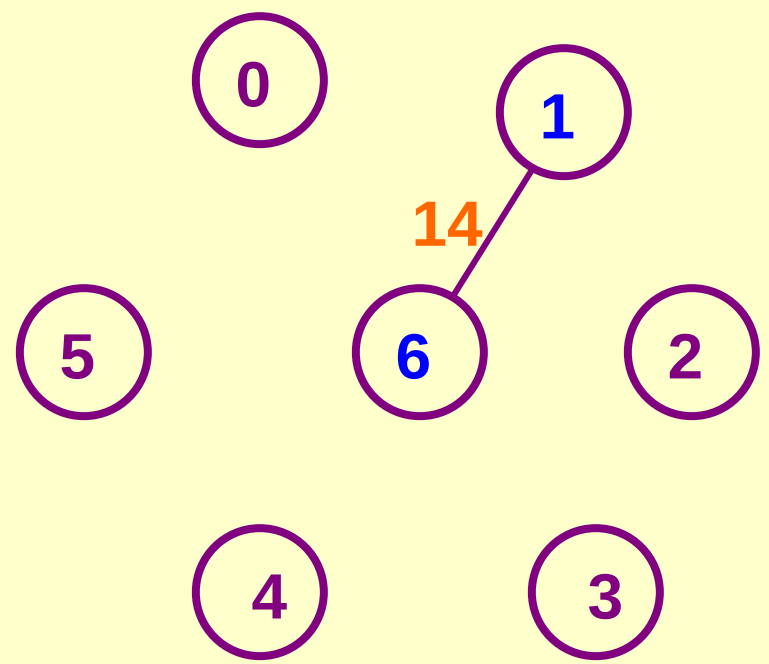
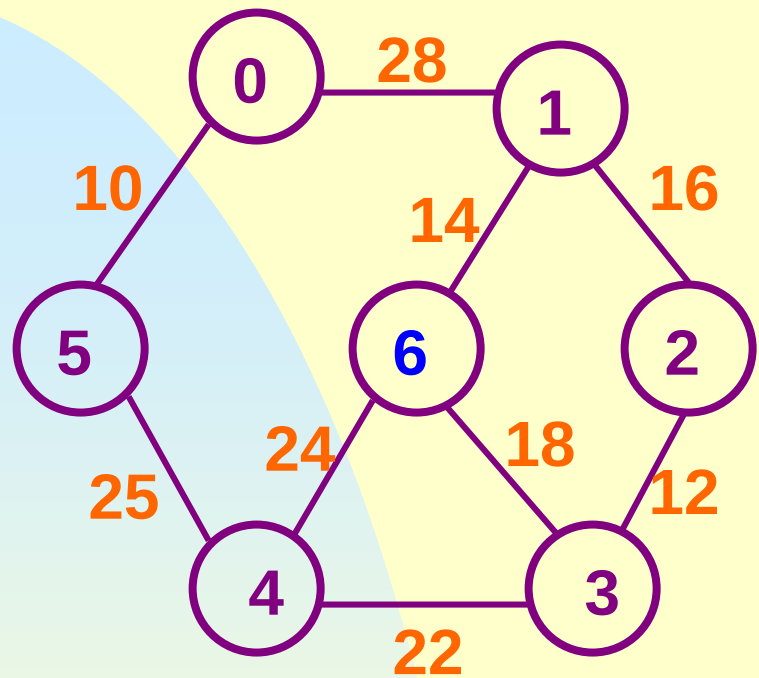
- In fact, **TV array can be omitted** because we can use **cost array** to express whether a vertex v is in TV or not.
 - ✓ When a vertex u is added into the “TV” set, the **cost of u can be set to 0**.
 - ✓ So, if **$\text{cost}[u] == 0$** , then the vertex u is already in the “TV” set.
 - ✓ We can find the minimal cost value among those that are **greater than 0** to determine the next edge.

vertex 6 is chosen(Initial)



vertex	0	<u>1</u>	2	3	4	5	6
TV	0	0	0	0	0	0	1
near	6	6	6	6	6	6	6
cost	∞	<u>14</u>	∞	18	24	∞	0

vertex 1 is added to TV

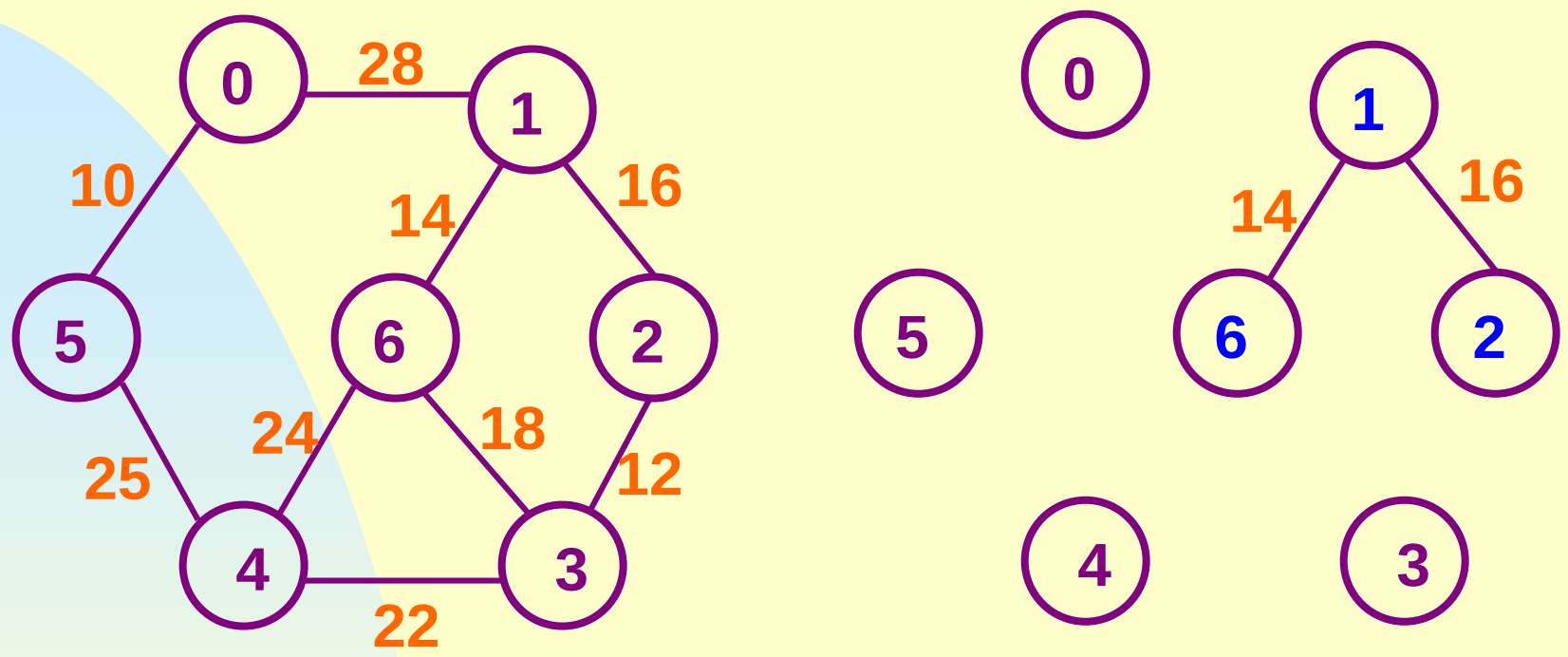


u=1

vertex	0	1	<u>2</u>	3	4	5	6
TV	0	1	0	0	0	0	1
near	1	6	1	6	6	6	6
cost	28	0	<u>16</u>	18	24	∞	0

if (cost(u,v) < cost(near(v),v)), update

vertex 2 is added to TV

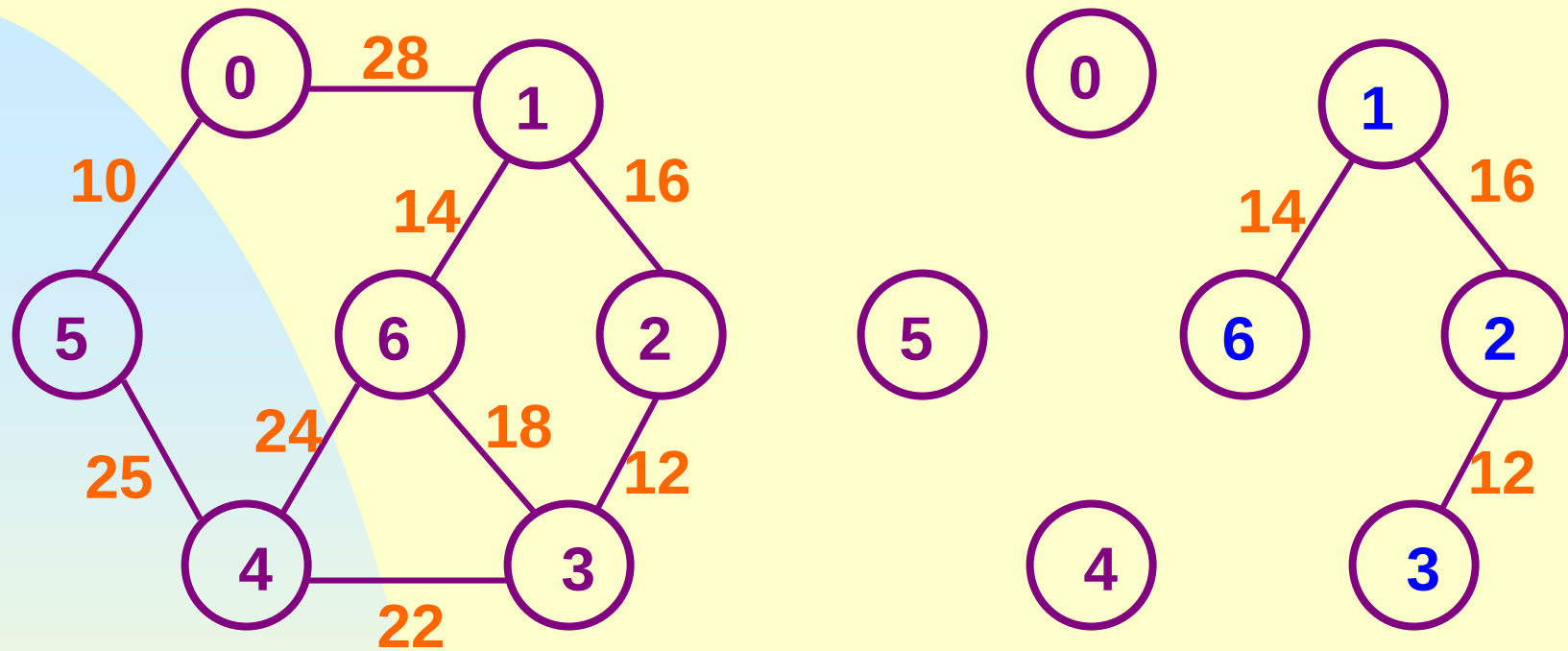


u=2

vertex	0	1	2	<u>3</u>	4	5	6
TV	0	1	1	0	0	0	1
near	1	6	1	2	6	6	6
cost	28	0	0	<u>12</u>	24	∞	0

if (cost(u,v) < cost(near(v),v)), update

vertex 3 is added to TV

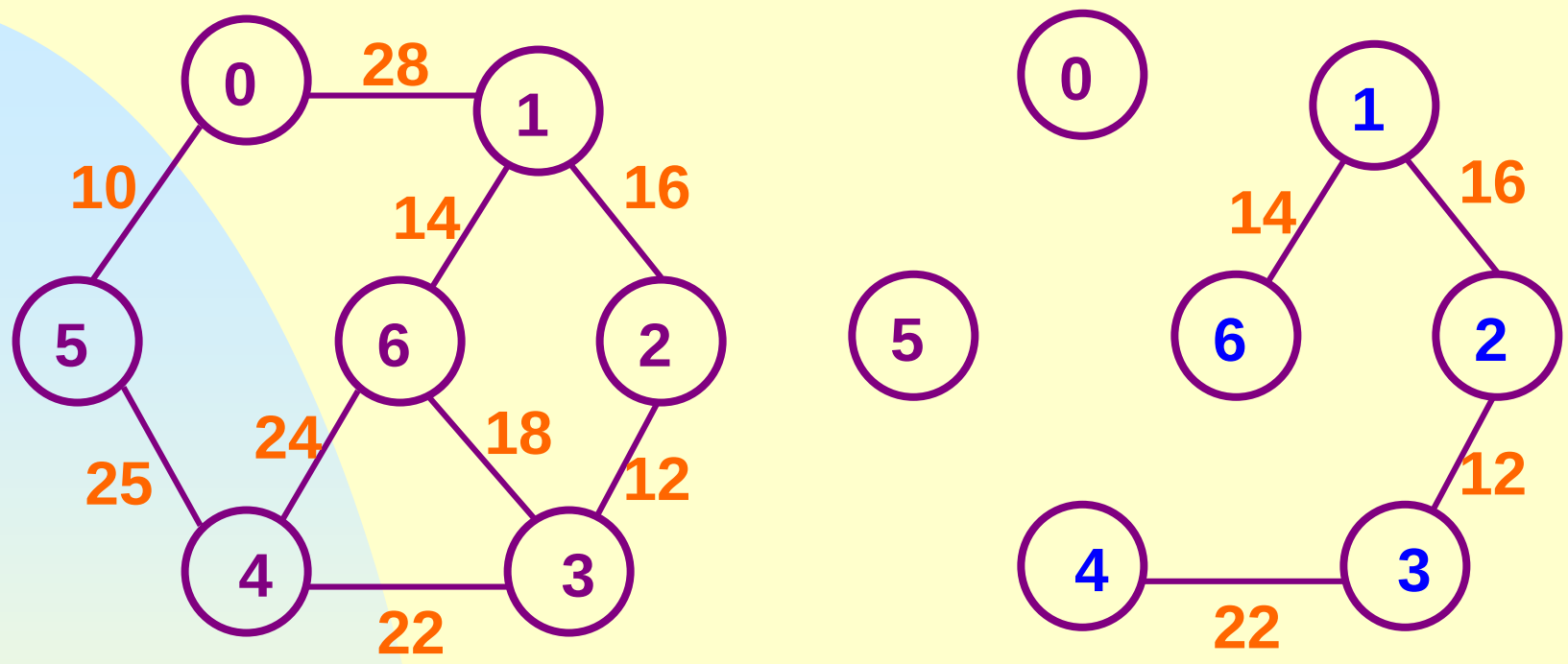


u=3

vertex	0	1	2	3	<u>4</u>	5	6
TV	0	1	1	1	0	0	1
near	1	6	1	2	3	6	6
cost	28	0	0	0	<u>22</u>	∞	0

if (cost(u,v) < cost(near(v),v)), update

vertex 4 is added to TV

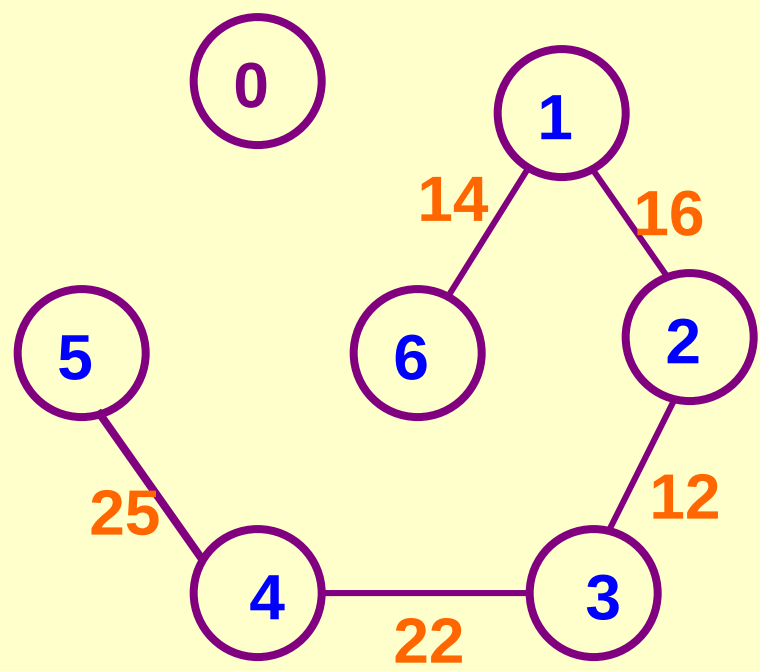
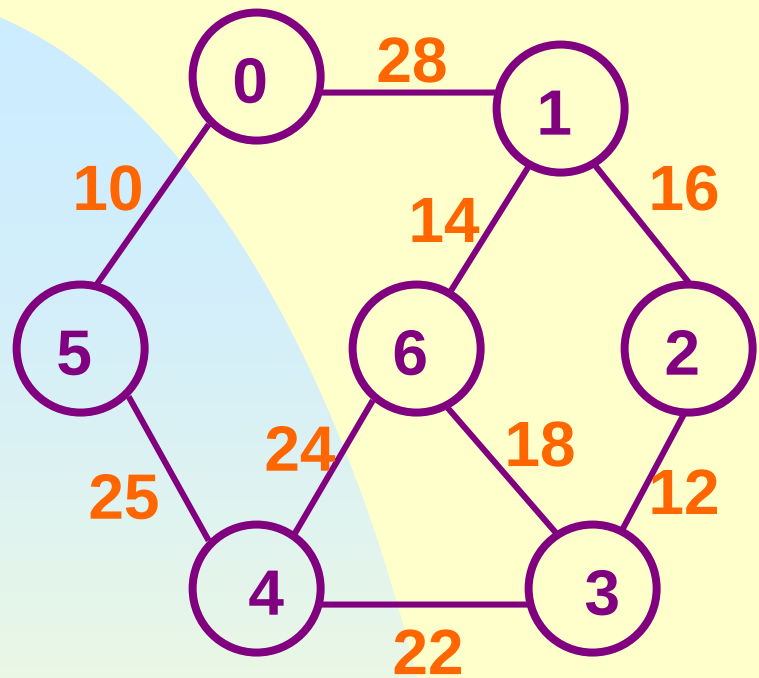


u=4

vertex	0	1	2	3	4	5	6
TV	0	1	1	1	1	0	1
near	1	6	1	2	3	4	6
cost	28	0	0	0	0	<u>25</u>	0

if (cost(u,v) < cost(near(v),v)), update

vertex 5 is added to TV

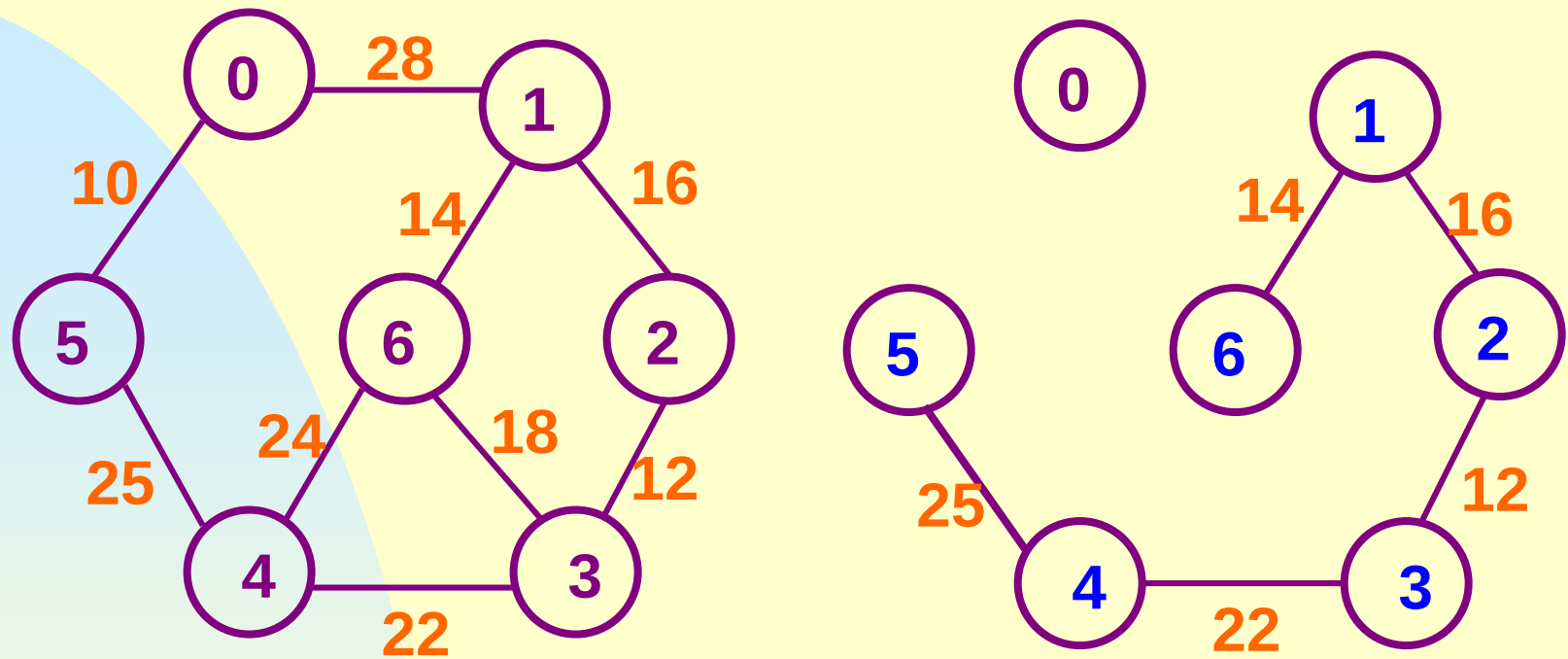


u=5

vertex	<u>0</u>	1	2	3	4	5	6
TV	0	1	1	1	1	1	1
near	5	6	1	2	3	4	6
cost	<u>10</u>	0	0	0	0	0	0

if (cost(u,v) < cost(near(v),v)), update

vertex 0 is added to TV (finished)



u=0

vertex	0	1	2	3	4	5	6
TV	1	1	1	1	1	1	1
near	5	6	1	2	3	4	6
cost	0	0	0	0	0	0	0

Prim's Algorithm using an min-heap

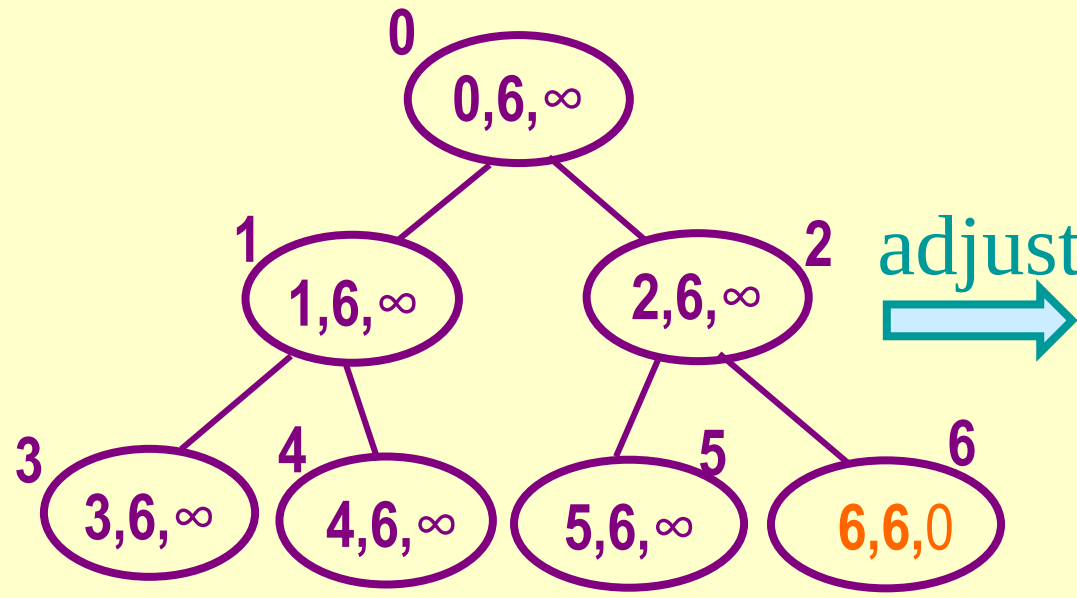
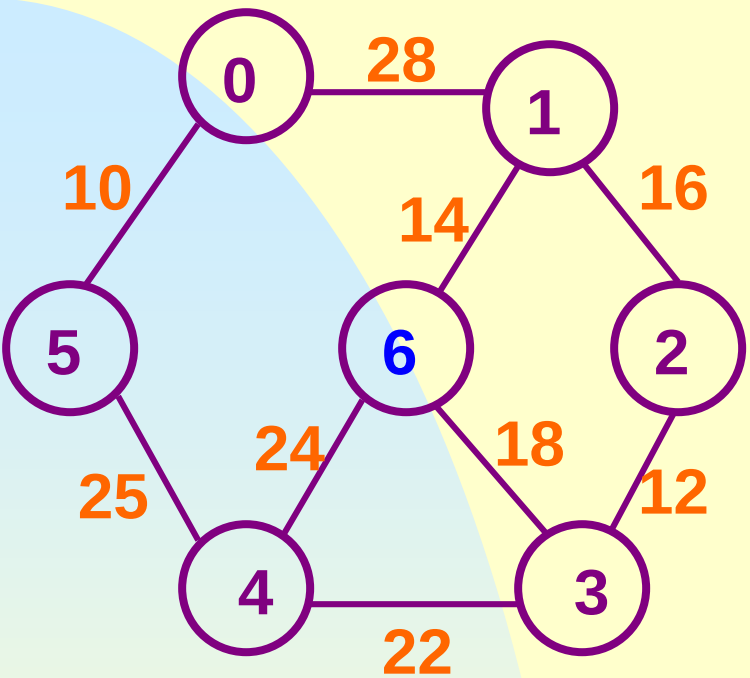
- For each vertex v not in TV , we use **an min-heap to store the edge with the minimal cost** among all the edges (v, w) with w being a vertex in TV .
- **The size of the min-heap is n** , where n is the number of vertices in the graph.
- When an edge outputs from the heap, we let the vertex that is not in TV be an element of TV , and we check the adjacent list for this vertex to update and adjust the heap.
- The time to output an edge from the heap, or update and adjust the heap is $\log n$.
- The number of edges in the graph is e , so the total time is $O(e \log n)$.

How to update and adjust the min-heap

- We assume the name of the min-heap is **Edge_Cost heap (EC_heap)**, and every edge in the min-heap is with the form **(v, w, cost)**, where **v** is a vertex not yet in **TV**, **w** is a vertex in **TV**, and **cost** is the weight of the edge **(v, w)**.
- For every vertex **v** not in **TV**, we **record the position of the edge (v, w, cost) in the heap in a array Pos**. It means that **EC_heap[Pos[i]]** store the edge **(v, w, cost)**, so it is easy to update an item when a new vertex is just added into **TV**.
- When an item in the heap is updated, we can easily use **an a bubbling up process** to adjust the heap since the cost of the updated item must decrease.

vertex 6 is chosen firstly(Initial)

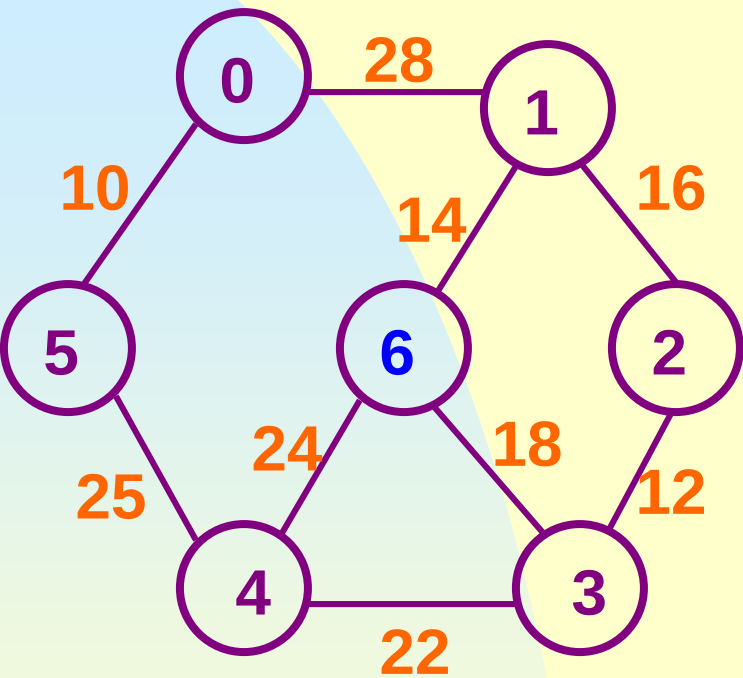
Edge_Cost heap (EC_heap)



index	0	1	2	3	4	5	6
Pos	0	1	2	3	4	5	6

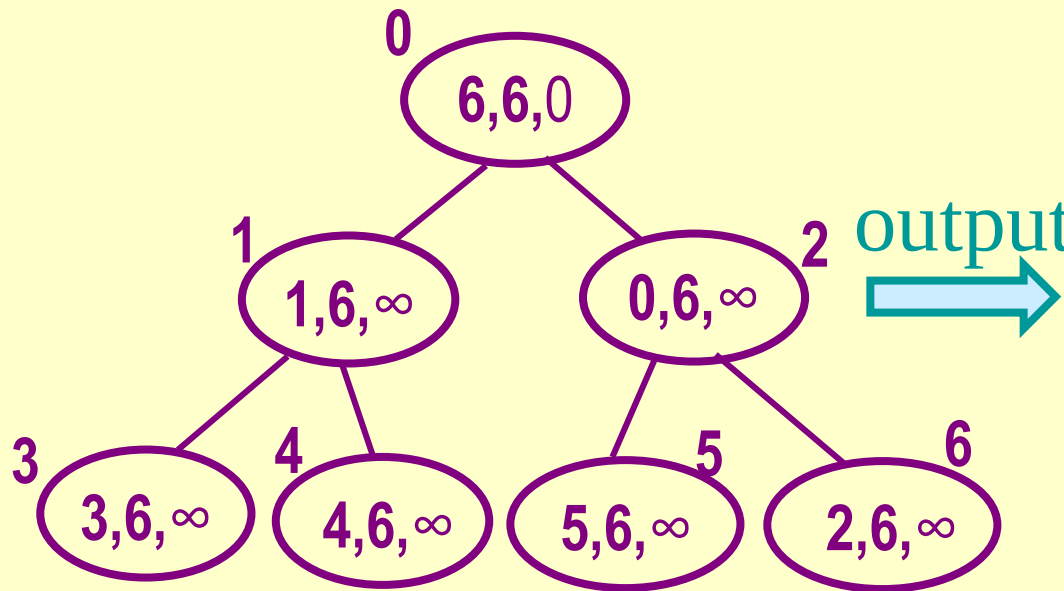
- When the cost of (v,w,cost) is ∞ , it means initially vertices v and w has no edge, or the cost of edge (v,w) is initially assigned to ∞ .

adjust heap and Pos

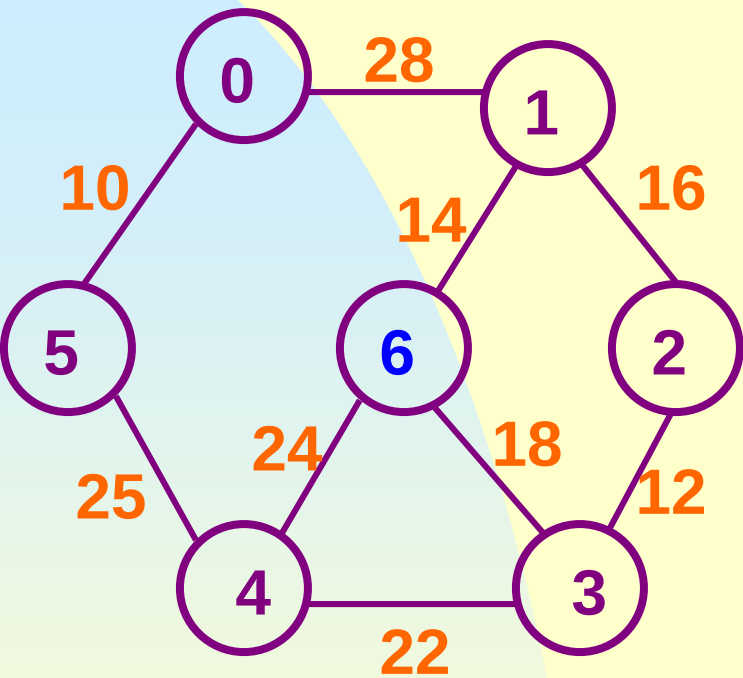


index	0	1	2	3	4	5	6
Pos	2	1	6	3	4	5	0

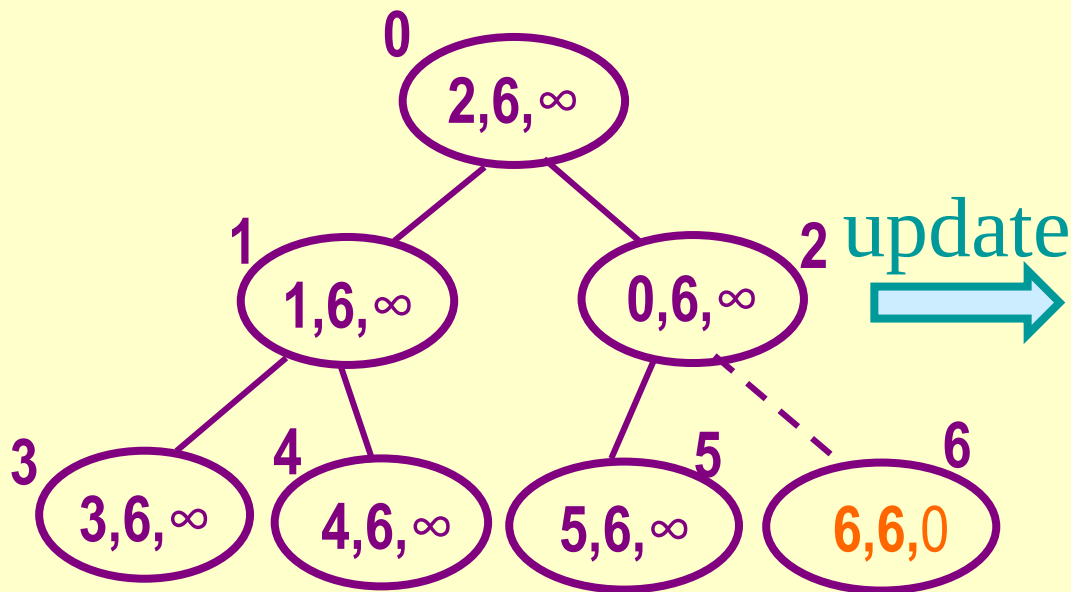
EC_heap



output

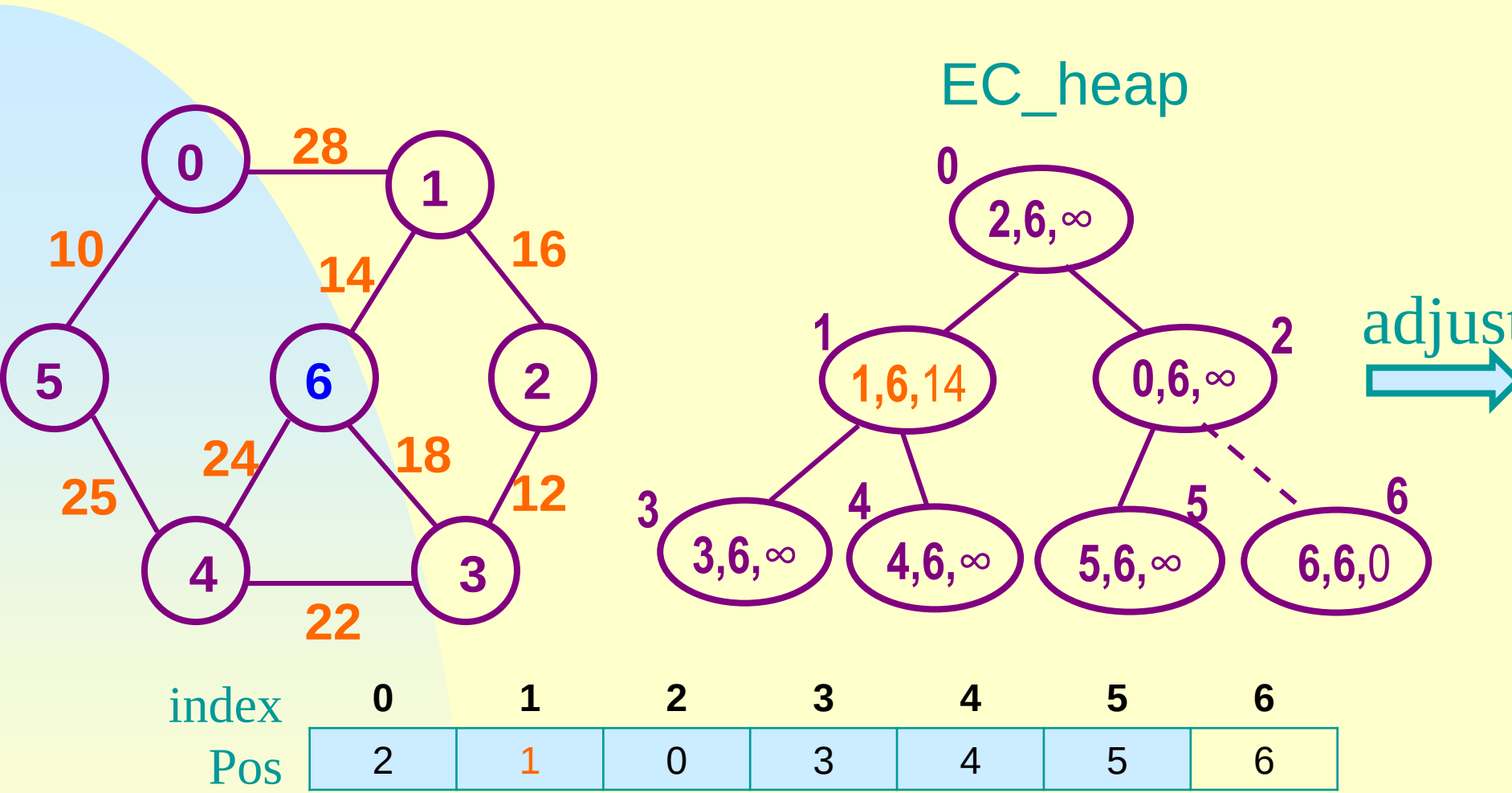


EC_heap



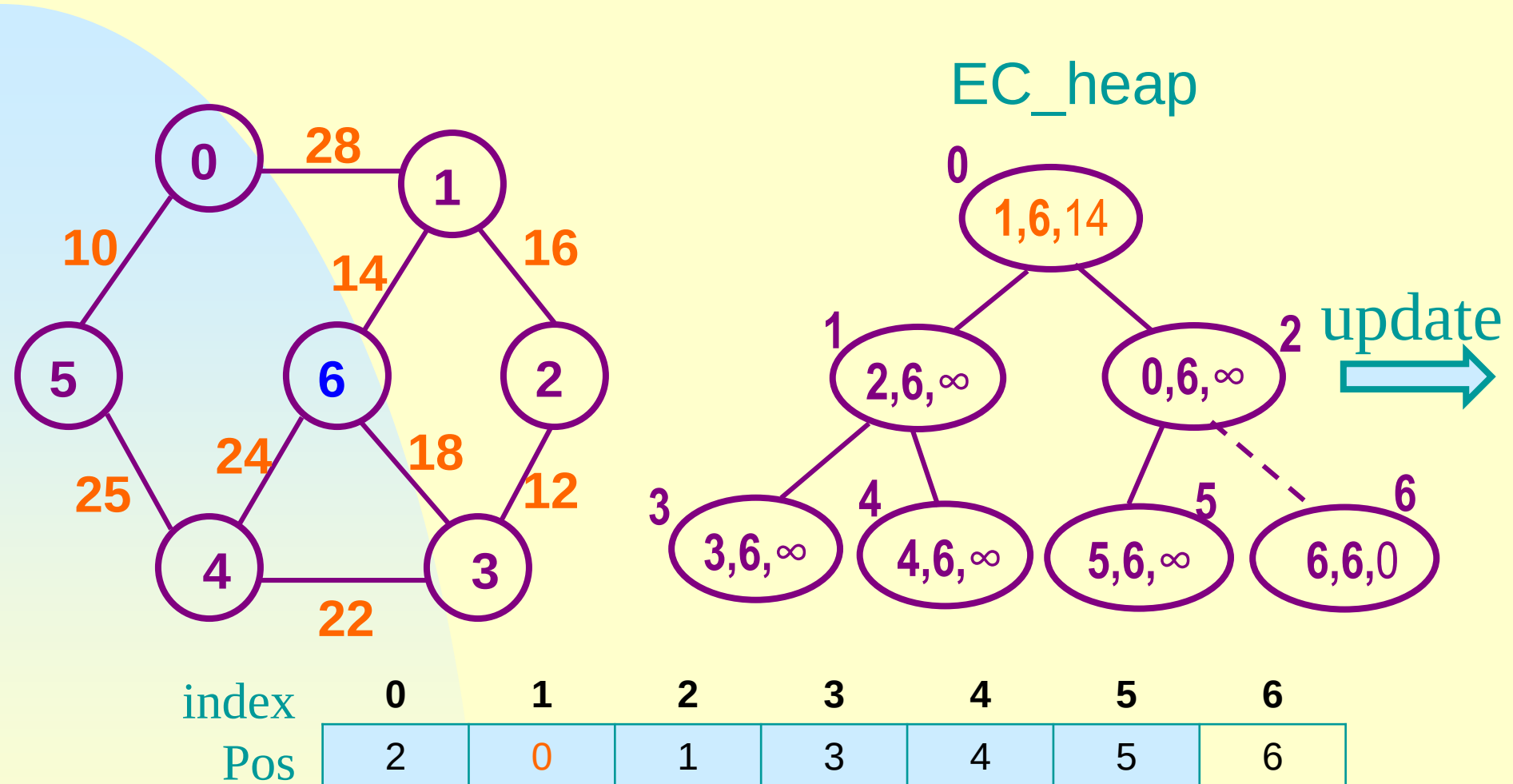
index	0	1	2	3	4	5	6
Pos	2	1	0	3	4	5	6

update EC_heap[Pos[1]]



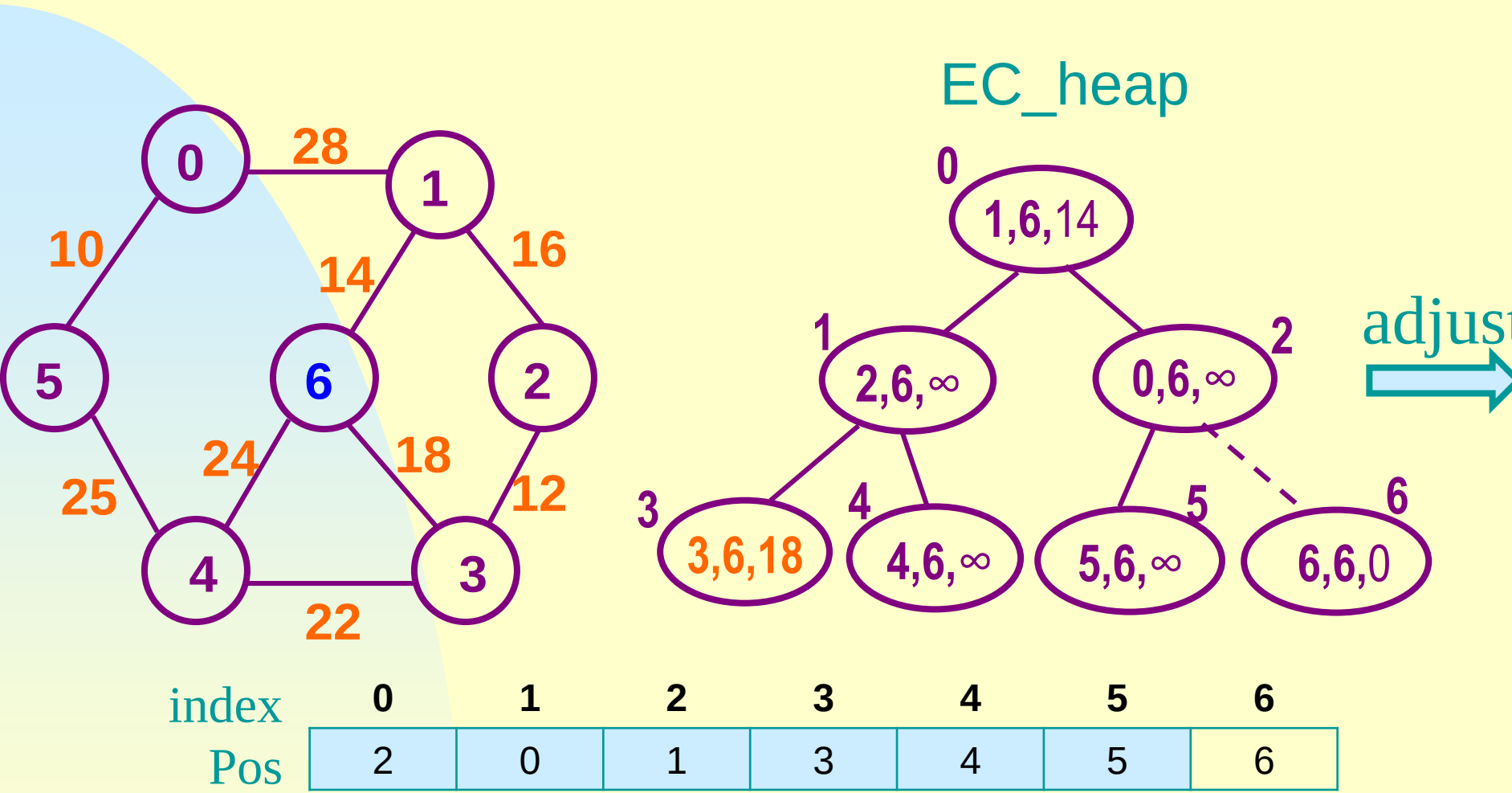
- We update EC_heap[Pos[1]] to (1,6,14).

adjust heap and Pos



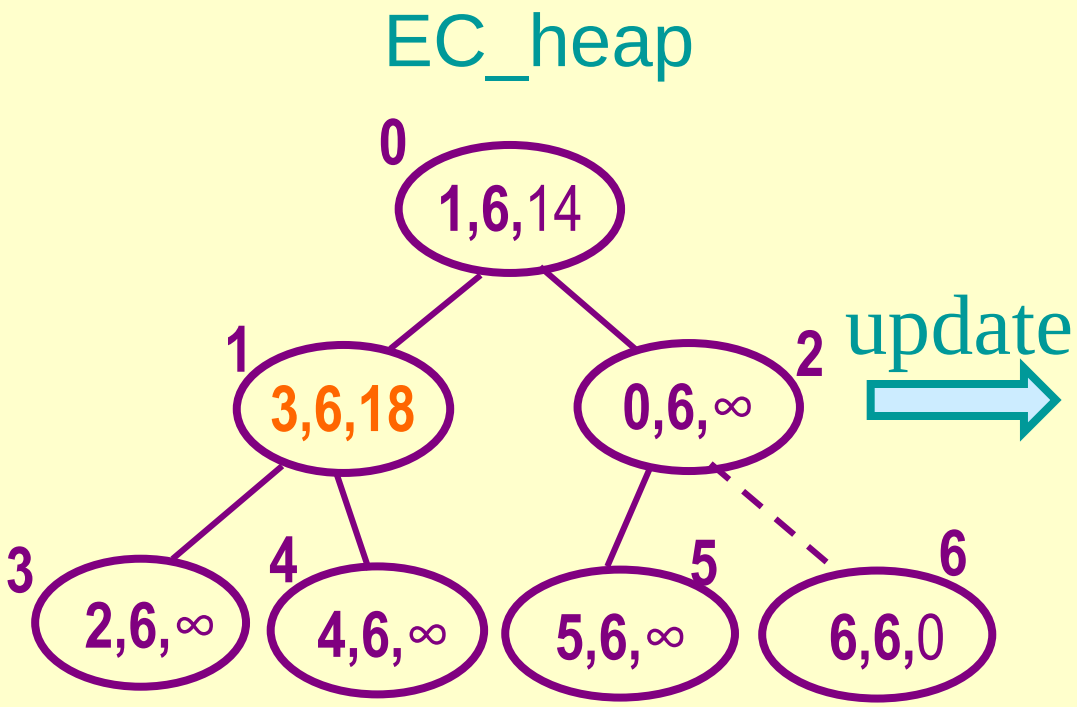
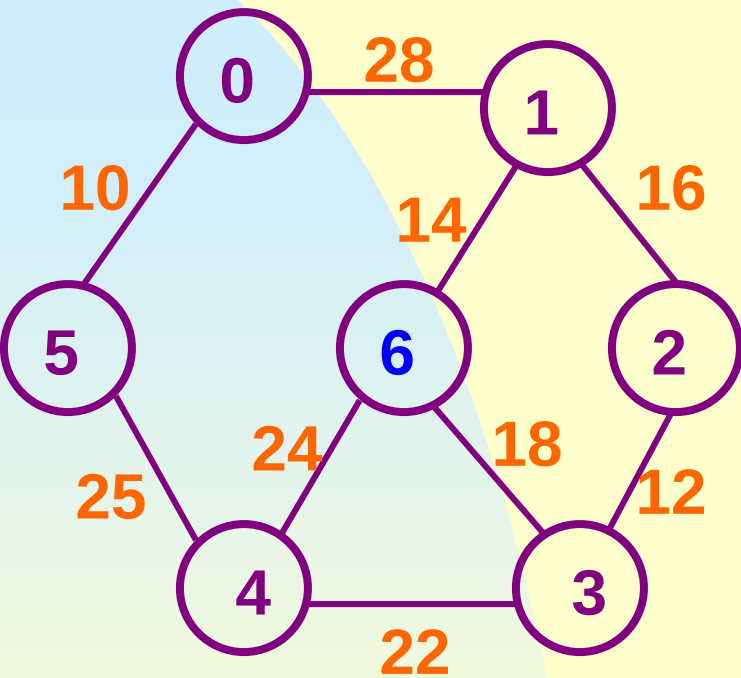
- We adjust the EC_heap by a **bubbling up process** since the cost of the updated item must decrease.

update EC_heap[Pos[3]]



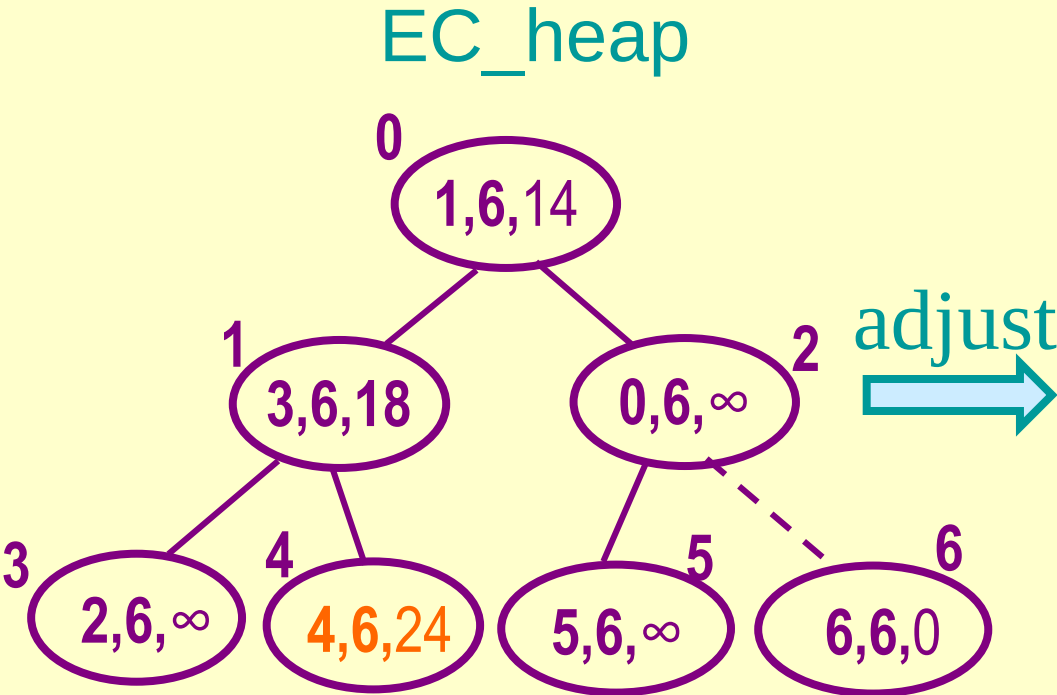
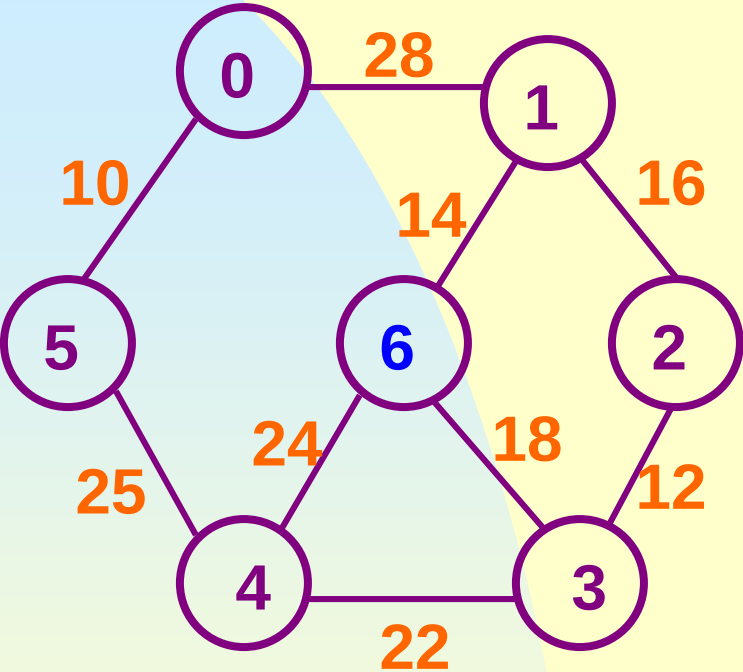
- We update EC_heap[Pos[1]] to (3,6,18).

adjust heap and Pos



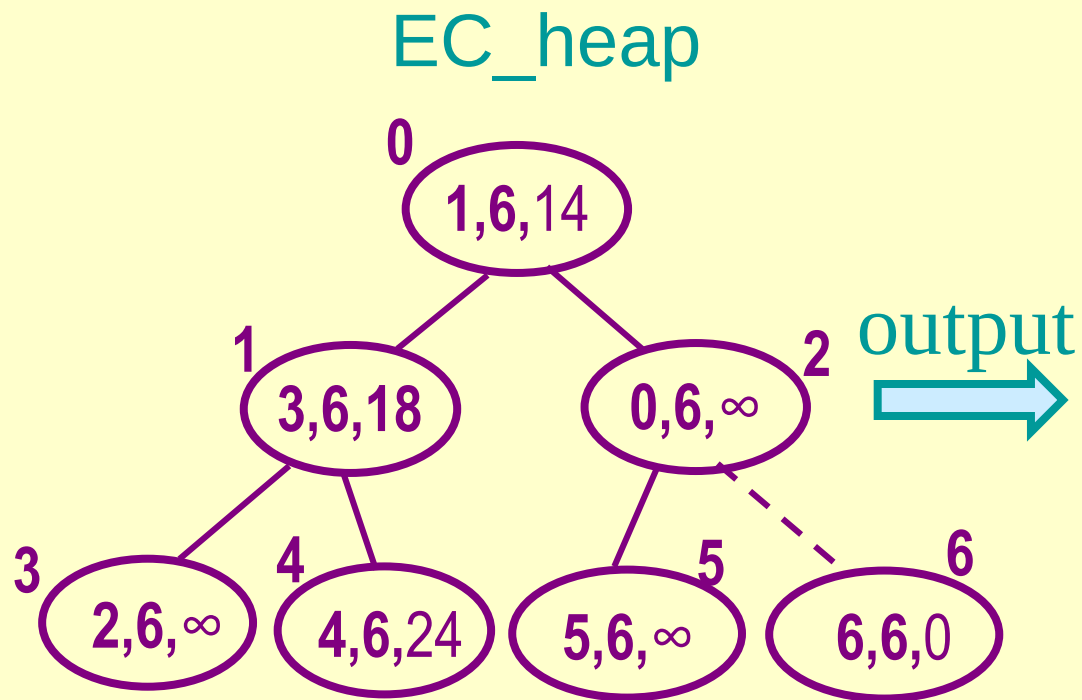
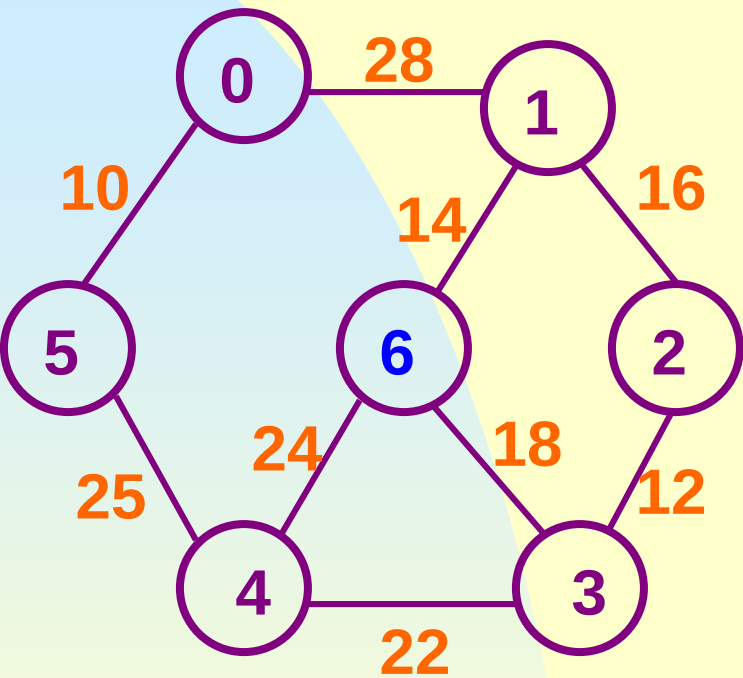
index	0	1	2	3	4	5	6
Pos	2	0	3	1	4	5	6

update EC_heap[Pos[4]]



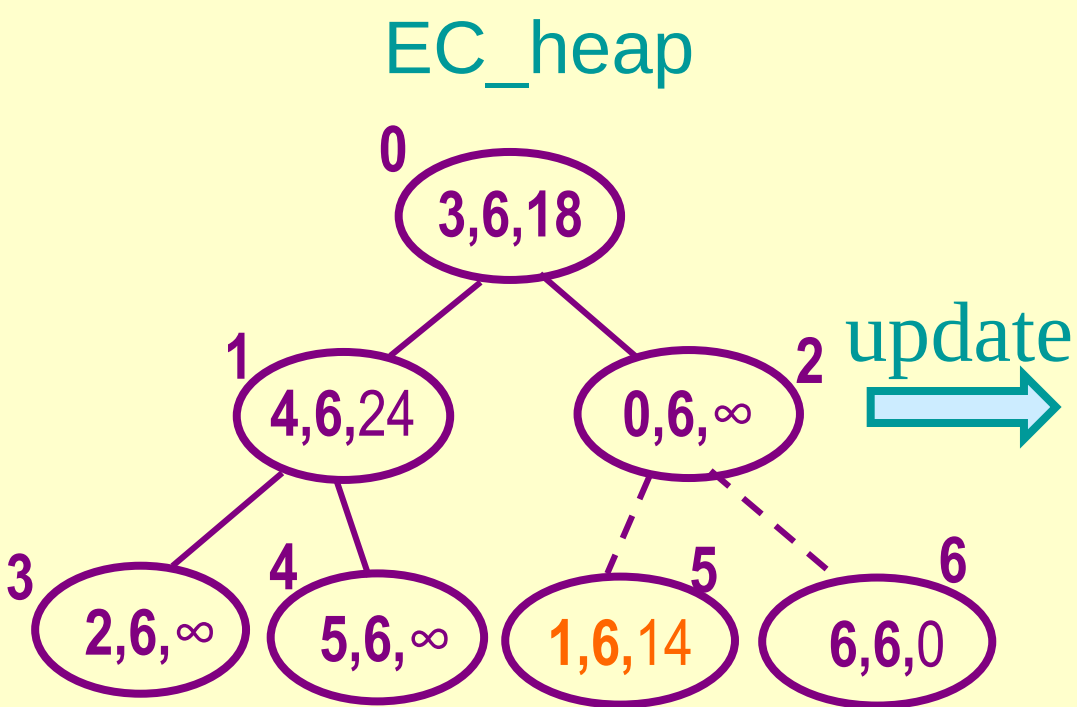
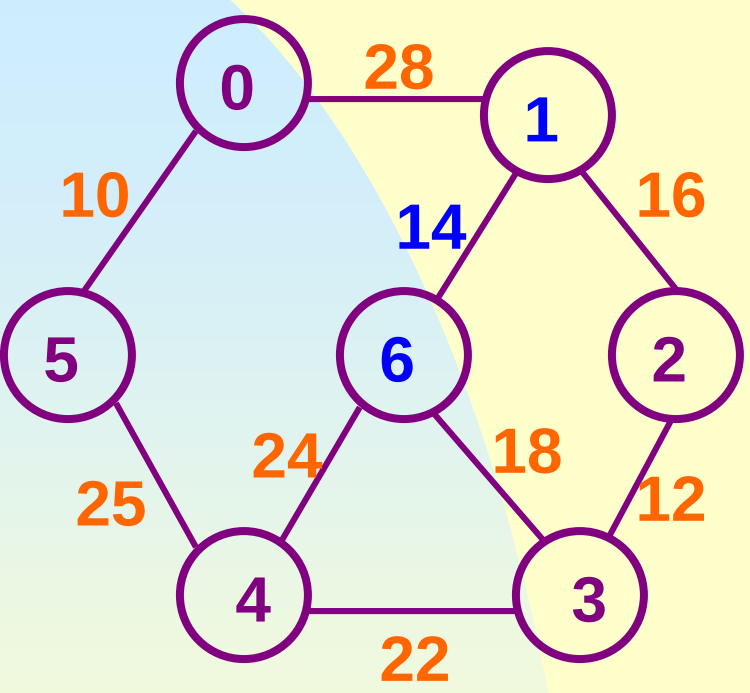
index	0	1	2	3	4	5	6
Pos	2	0	3	1	4	5	6

adjust heap and Pos



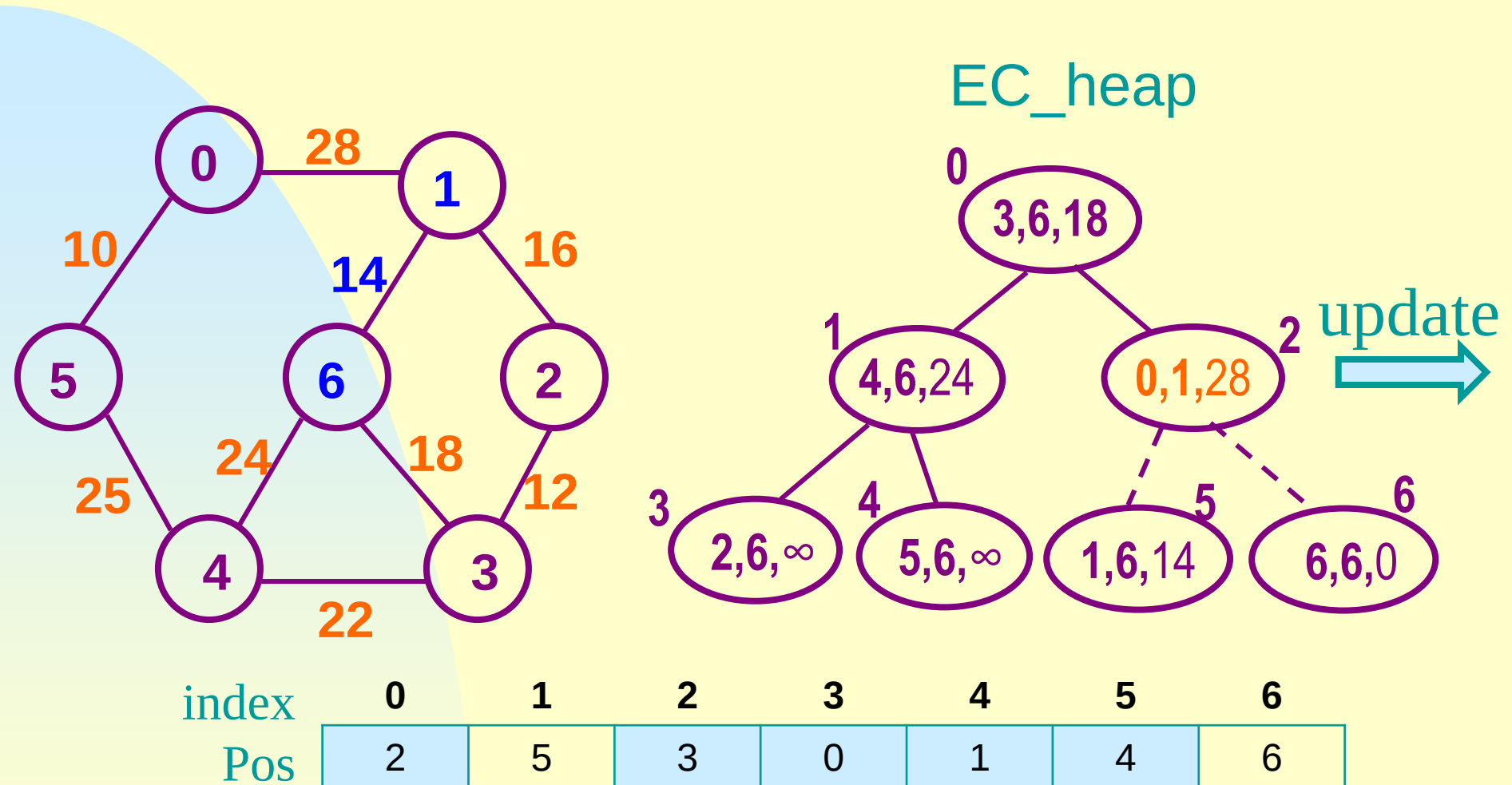
index	0	1	2	3	4	5	6
Pos	2	0	3	1	4	5	6

output and vertex 1 is chosen

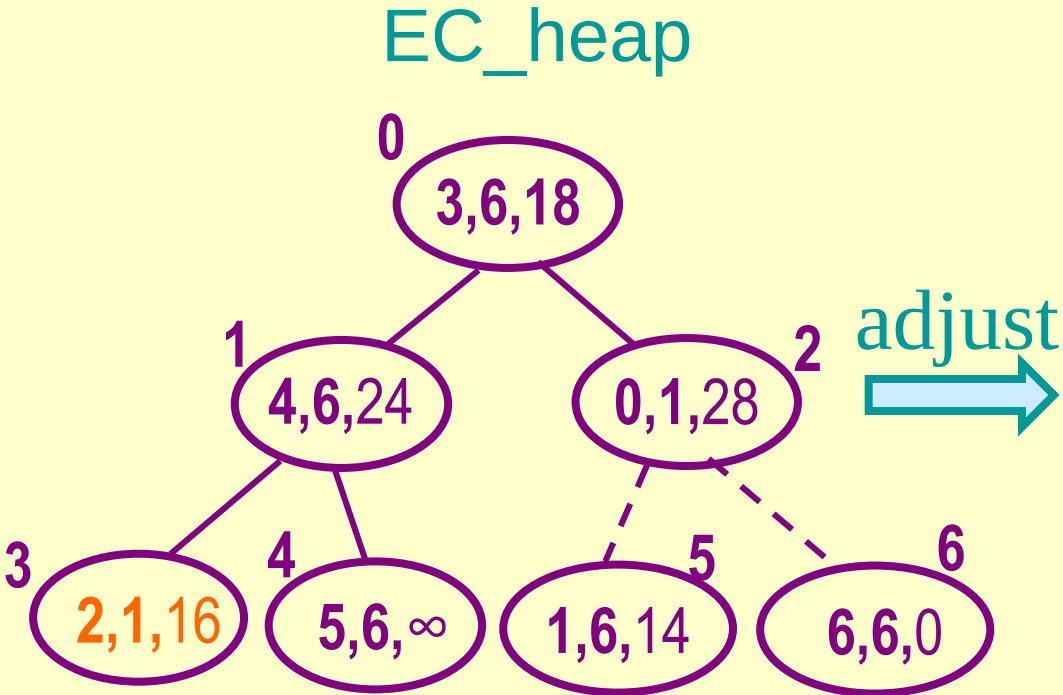
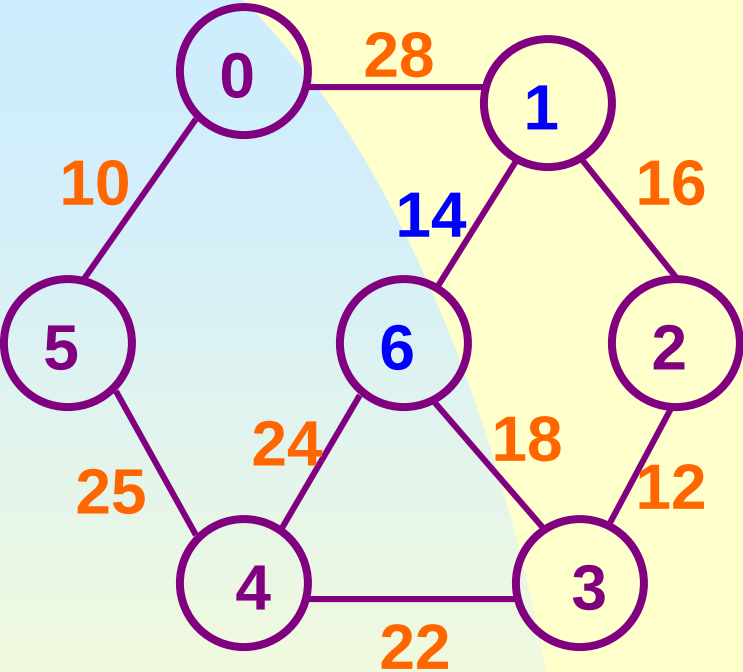


index	0	1	2	3	4	5	6
Pos	2	5	3	0	1	4	6

update EC_heap[Pos[0]], no need to adjust heap

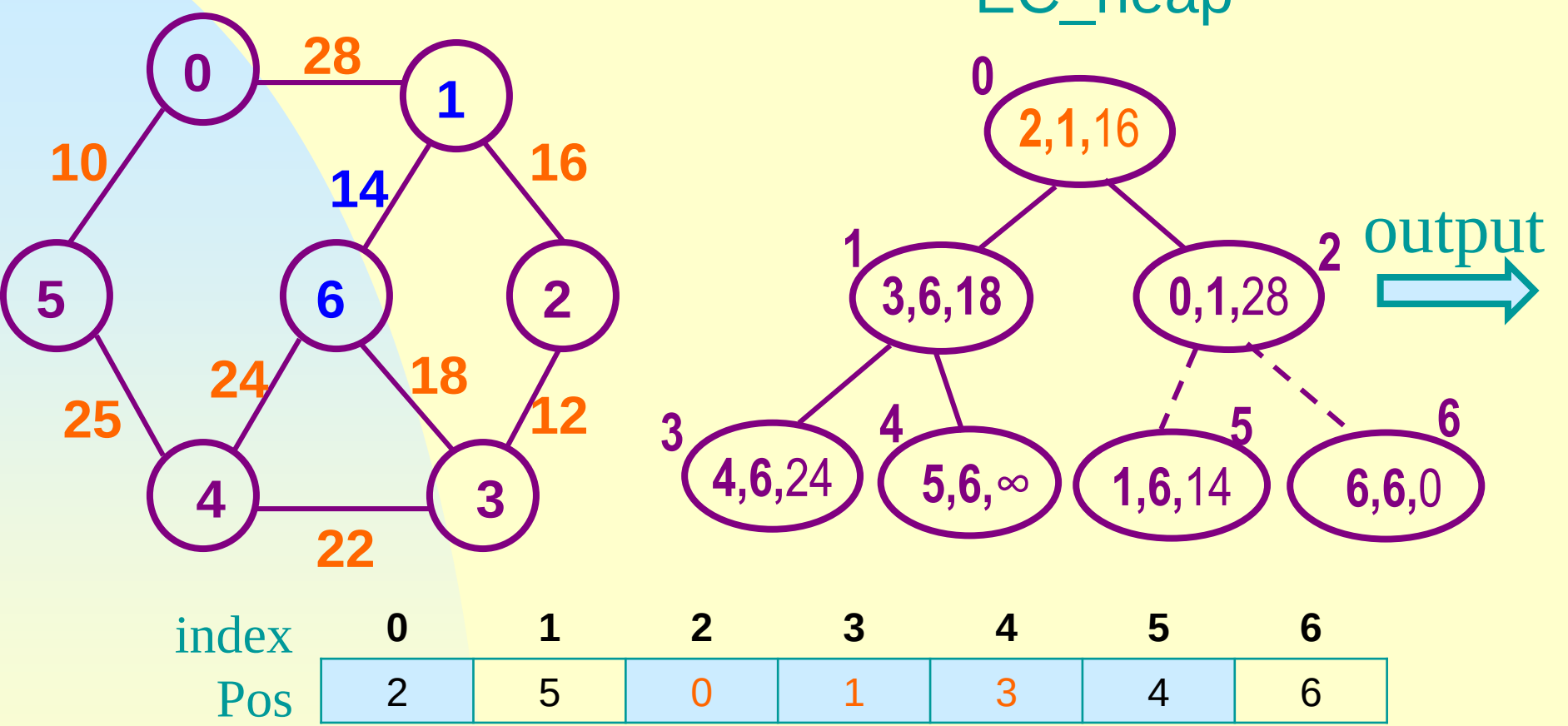


update EC_heap[Pos[2]]

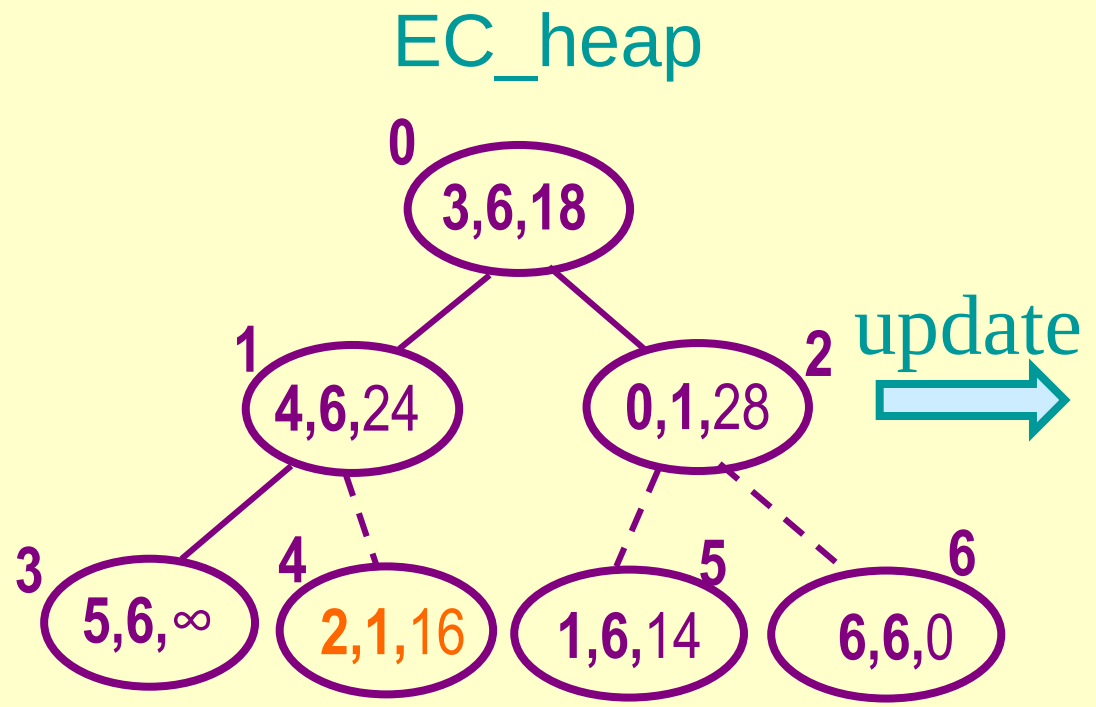
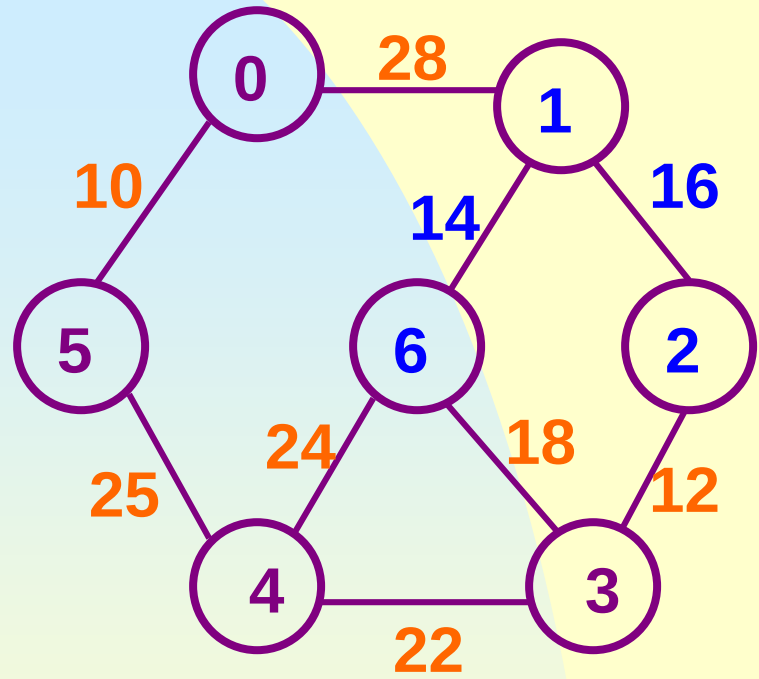


index	0	1	2	3	4	5	6
Pos	2	5	3	0	1	4	6

adjust heap and Pos

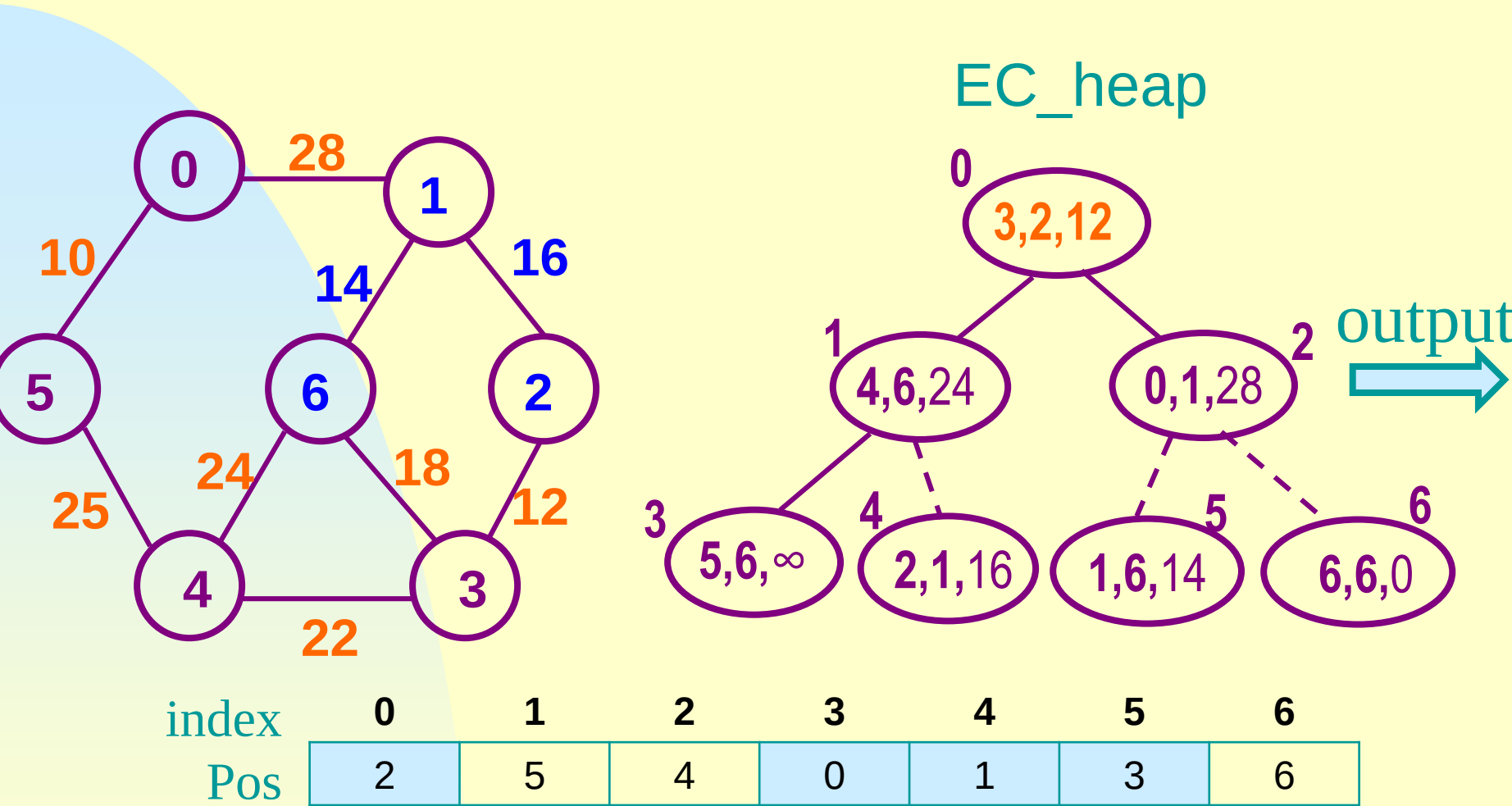


output and vertex 2 is chosen

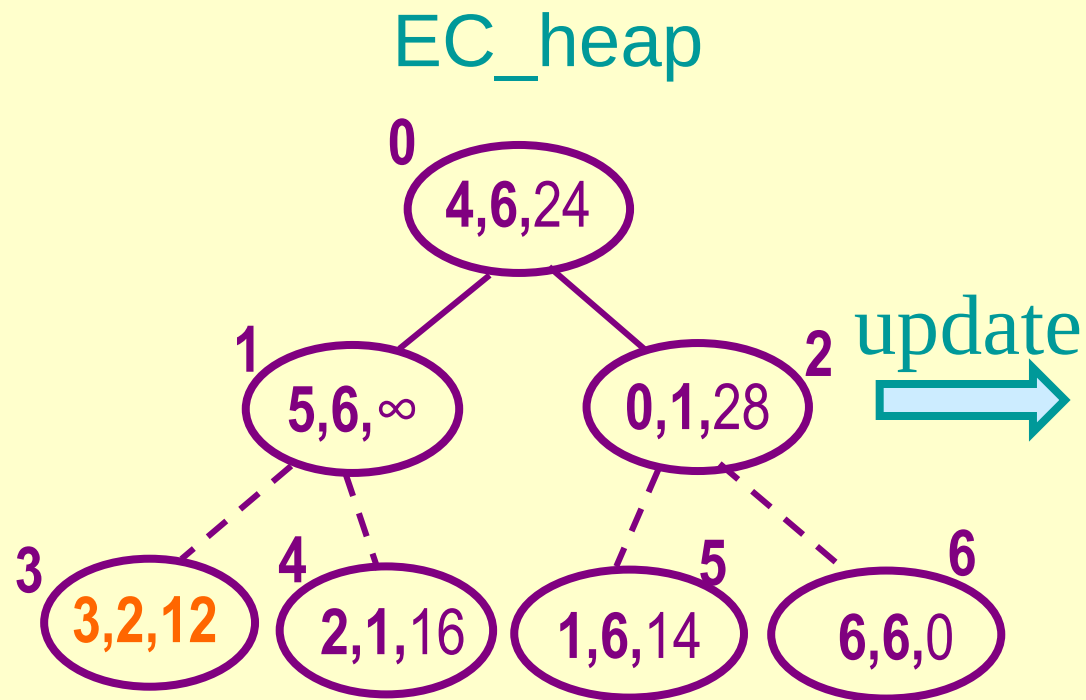
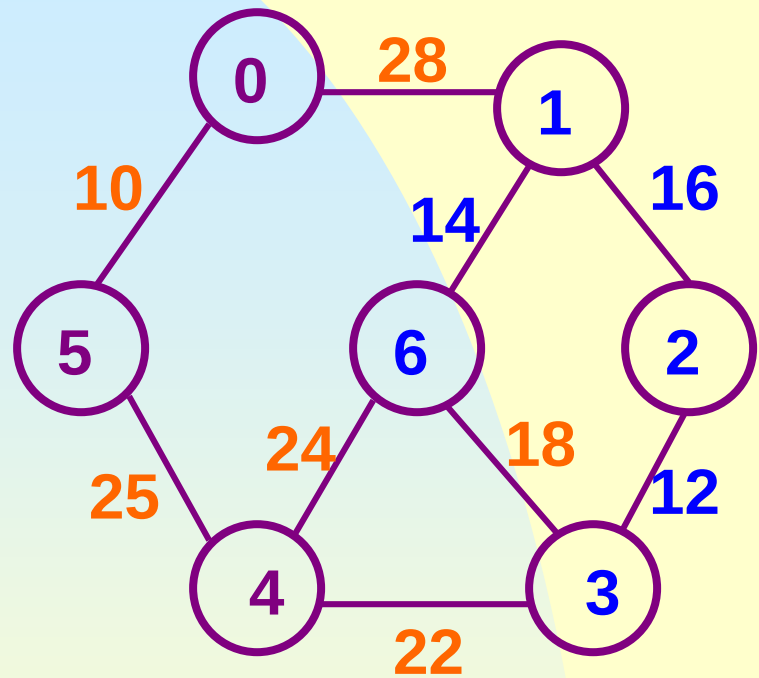


index	0	1	2	3	4	5	6
Pos	2	5	4	0	1	3	6

update EC_heap[Pos[3]], no need to adjust heap

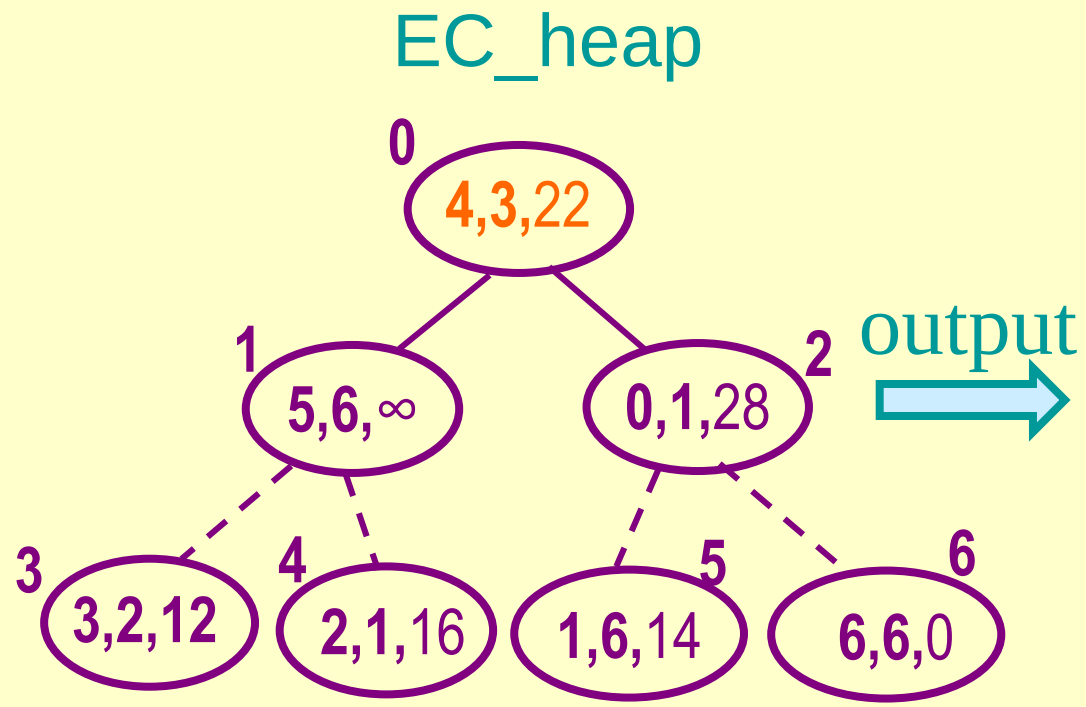
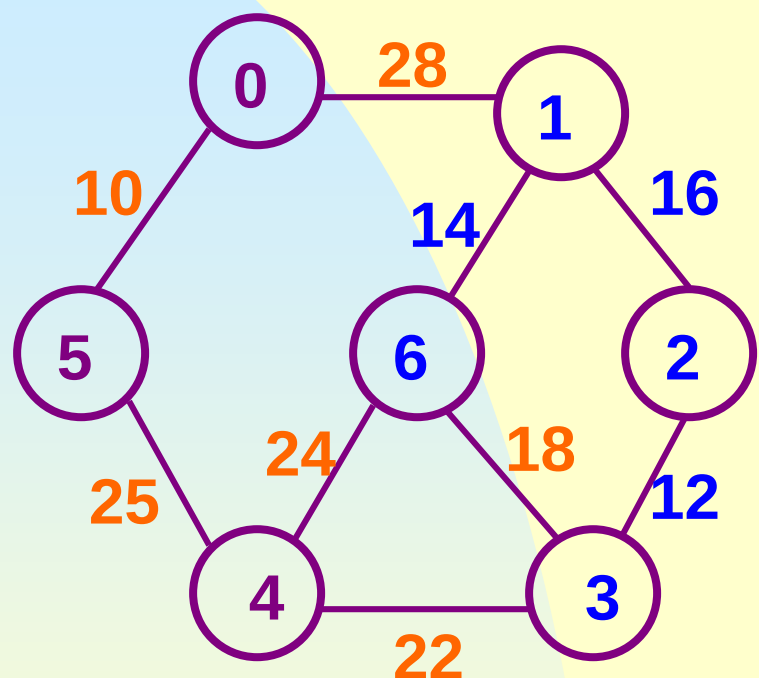


output and vertex 3 is chosen



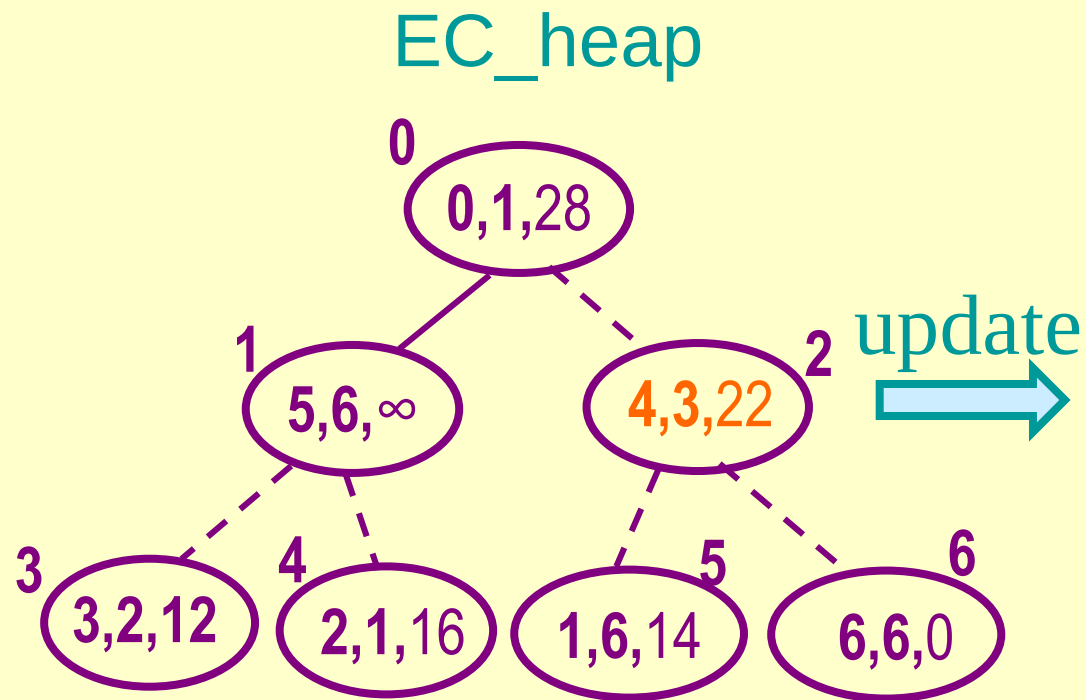
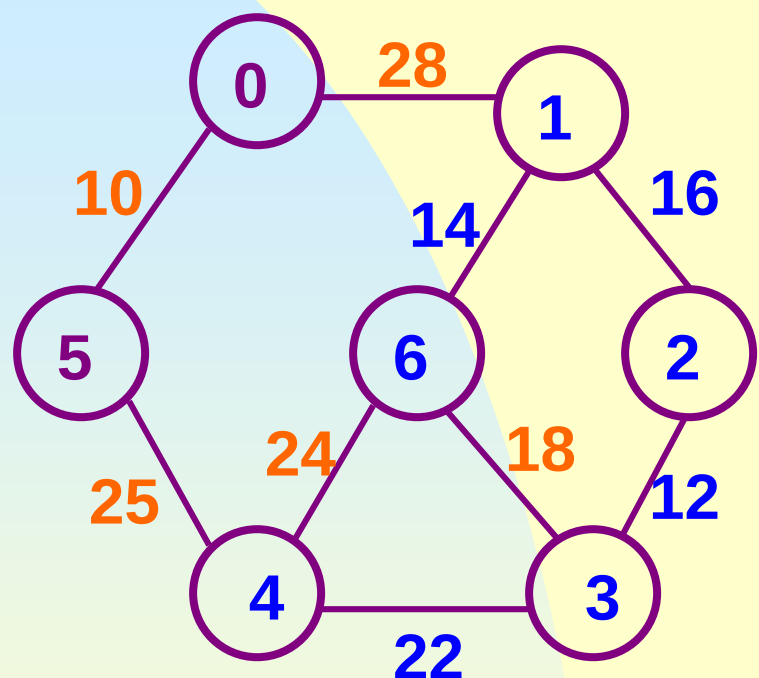
index	0	1	2	3	4	5	6
Pos	2	5	4	3	0	1	6

update EC_heap[Pos[4]], no need to adjust heap



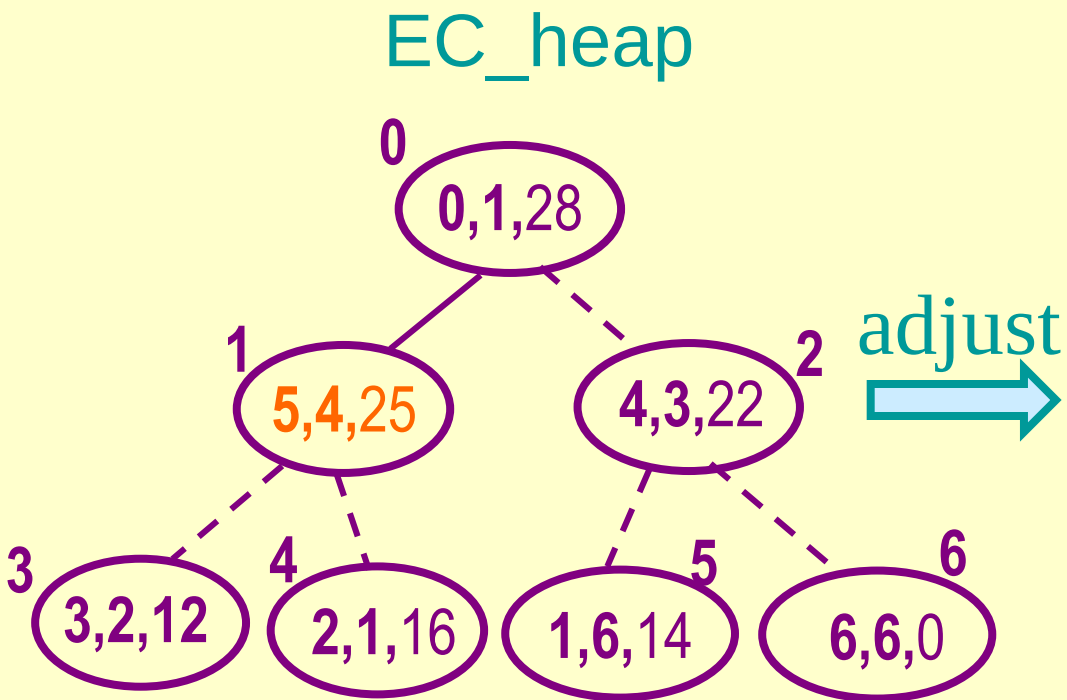
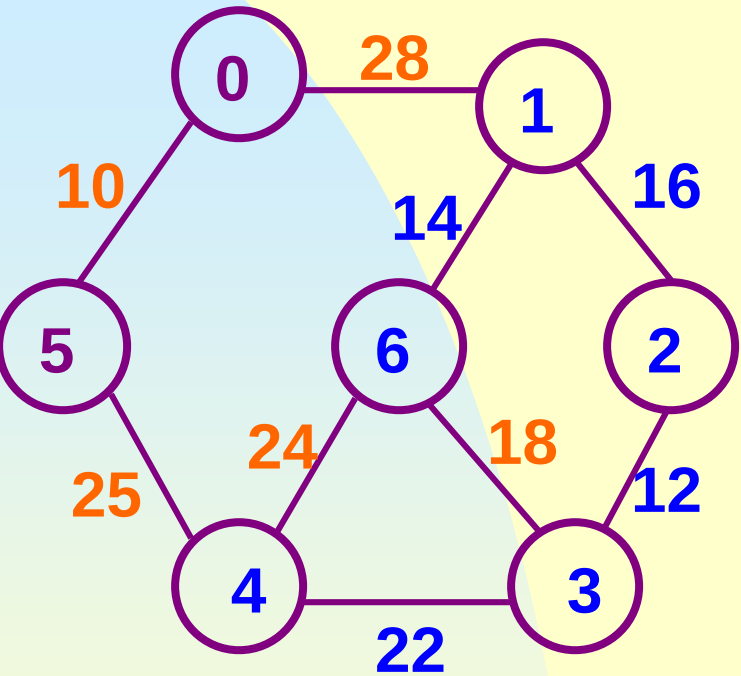
index	0	1	2	3	4	5	6
Pos	2	5	4	3	0	1	6

output and vertex 4 is chosen



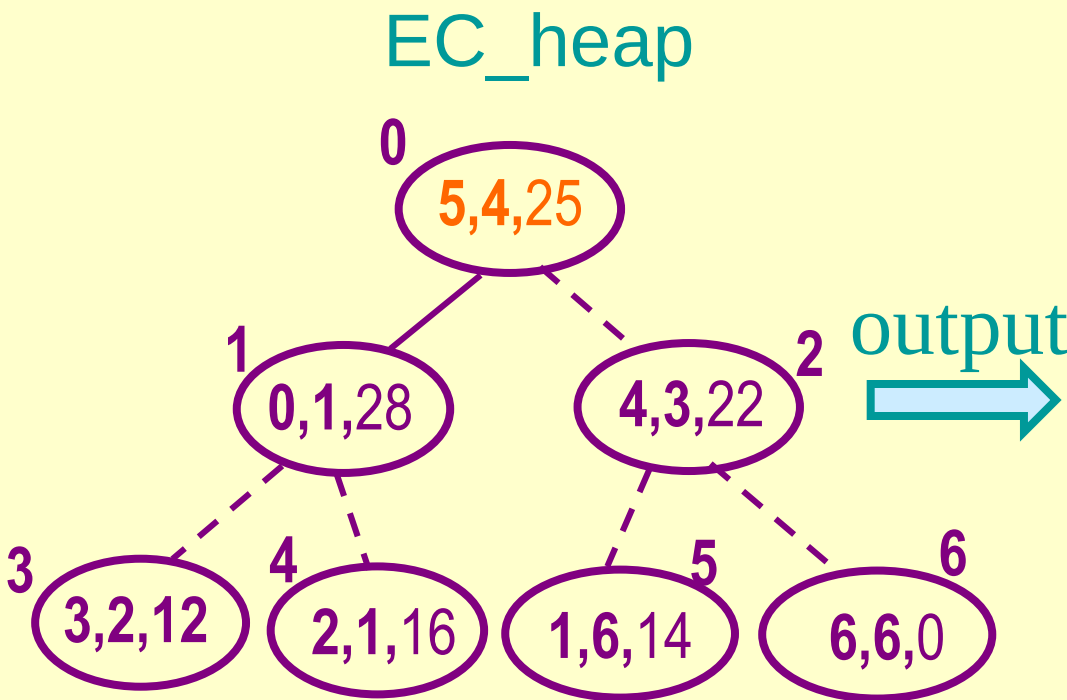
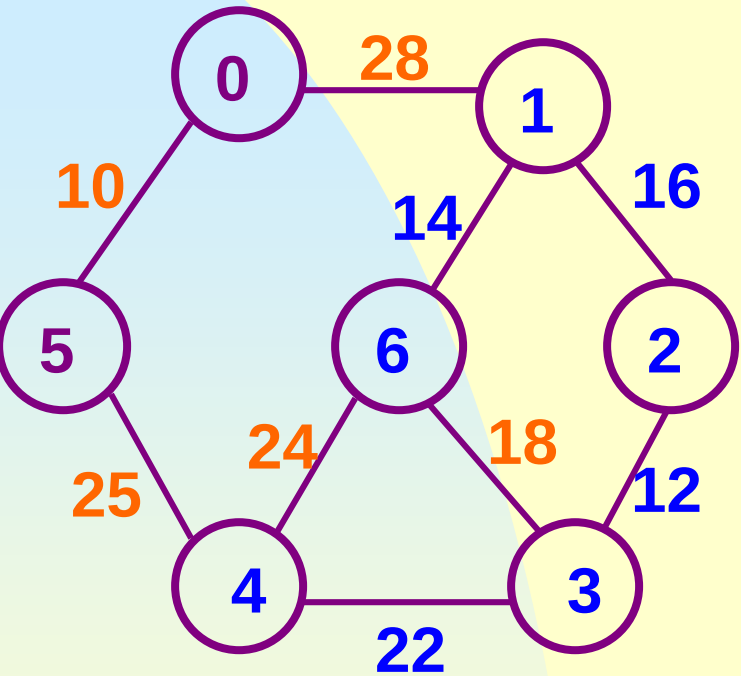
index	0	1	2	3	4	5	6
Pos	0	5	4	3	2	1	6

update EC_heap[Pos[5]]



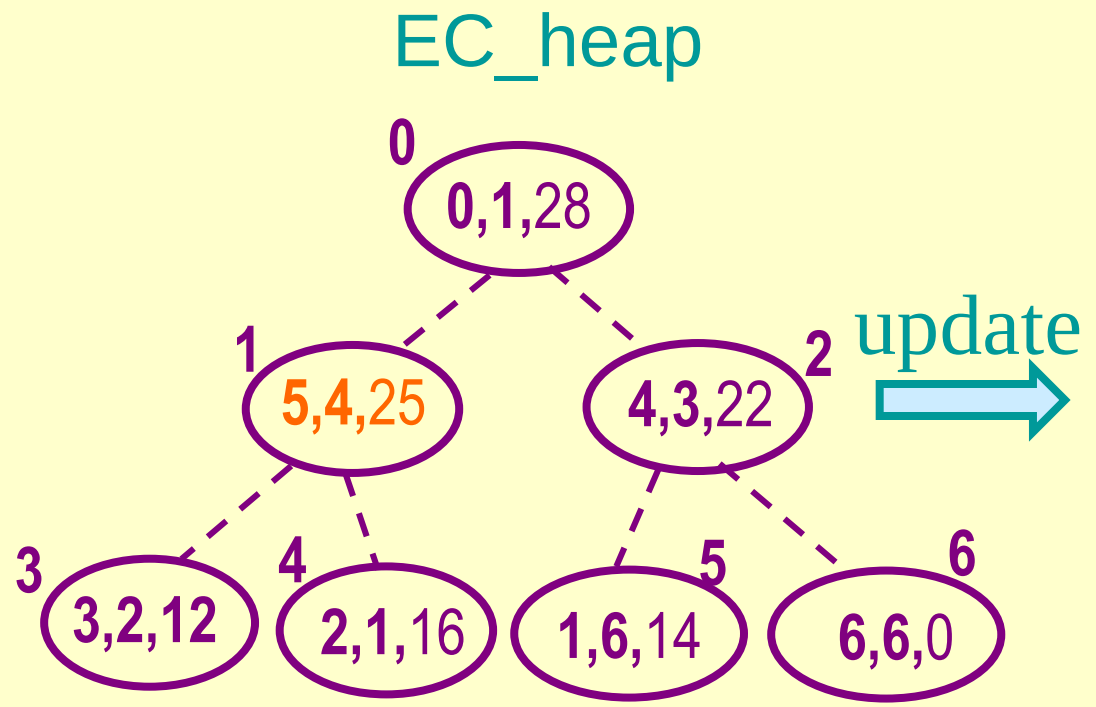
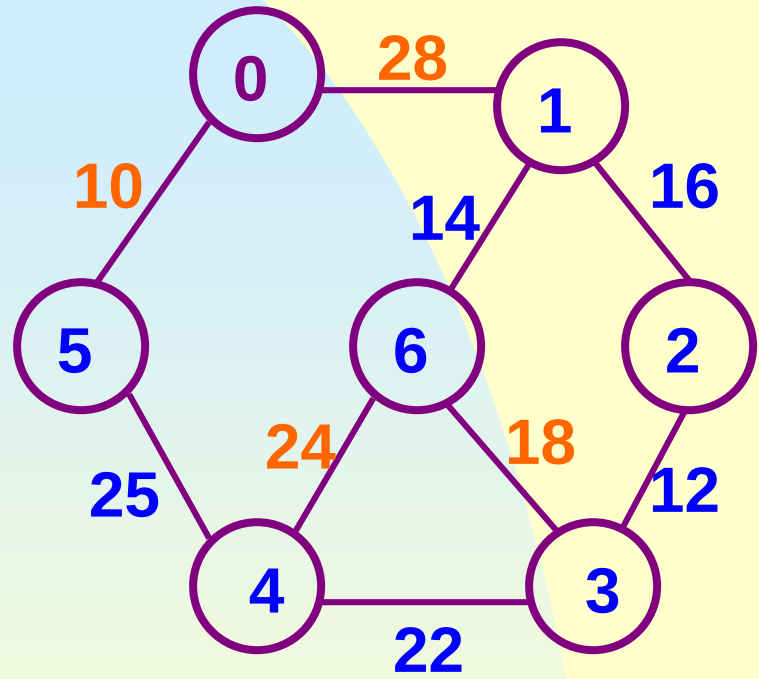
index	0	1	2	3	4	5	6
Pos	0	5	4	3	2	1	6

update EC_heap[Pos[5]]



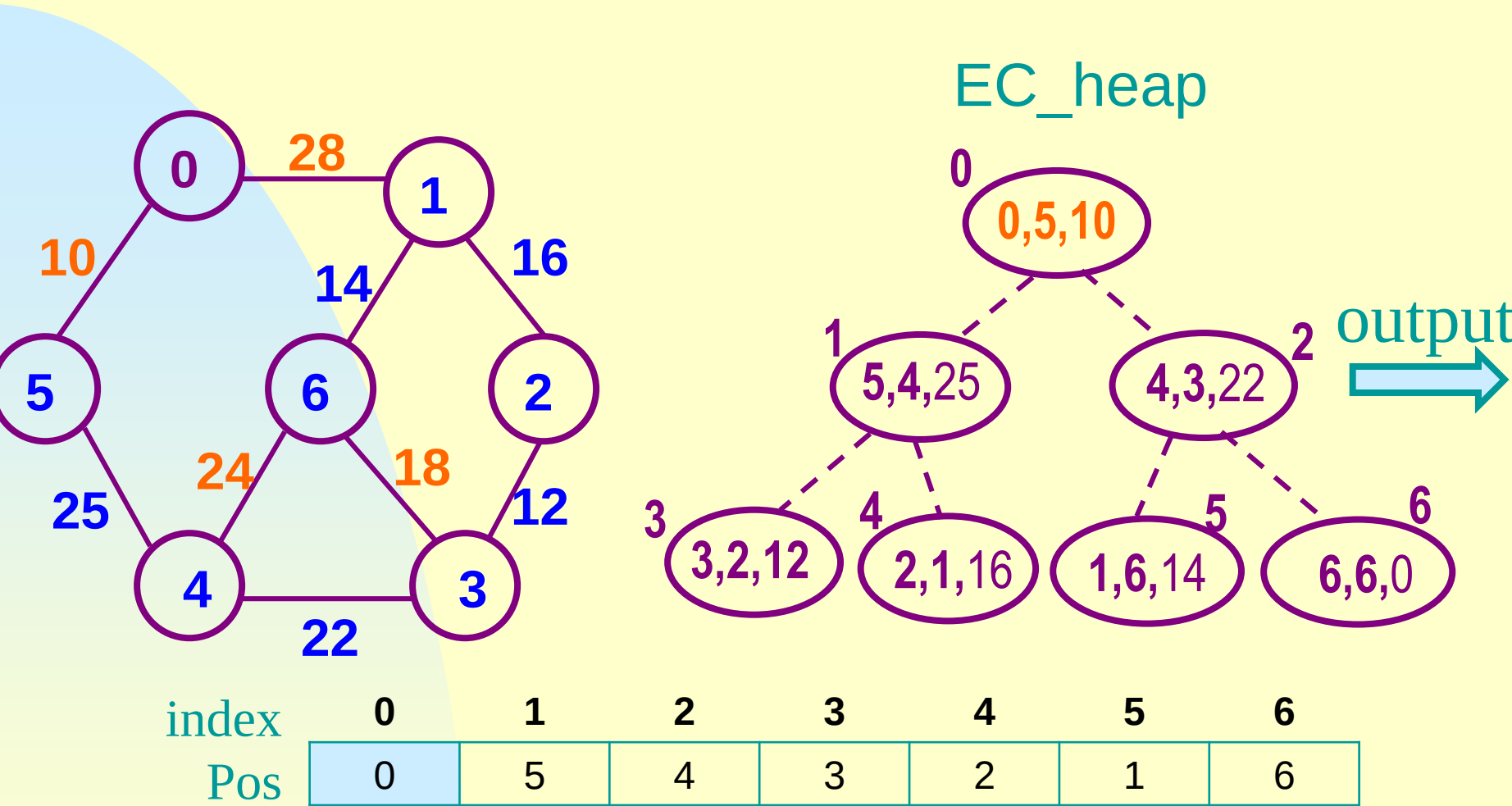
index	0	1	2	3	4	5	6
Pos	1	5	4	3	2	0	6

output and vertex 5 is chosen

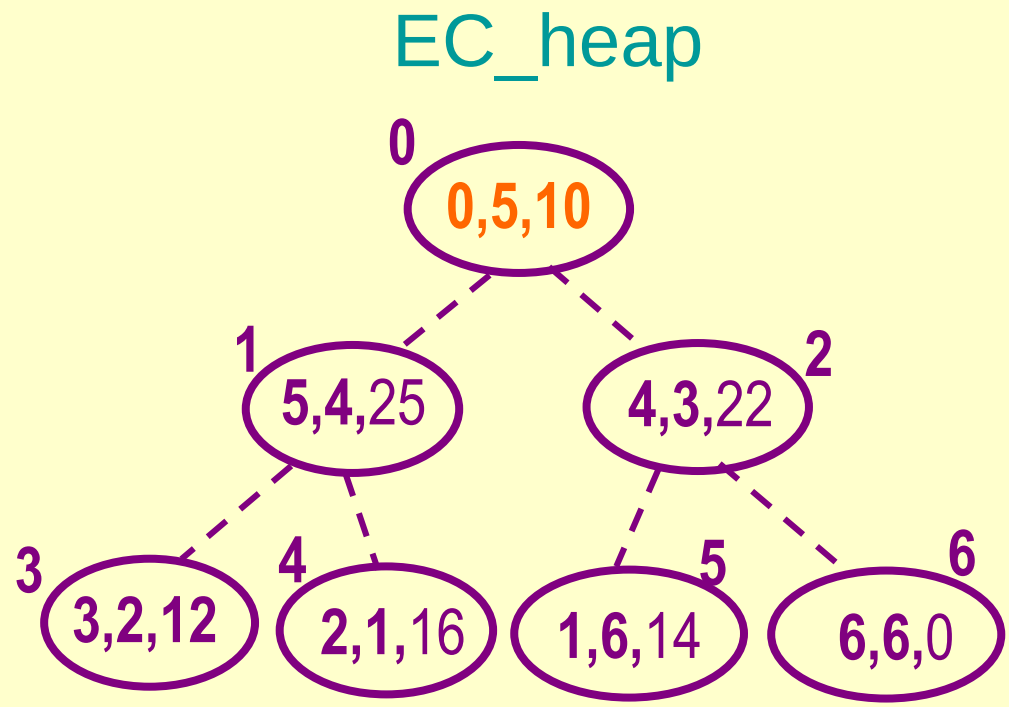
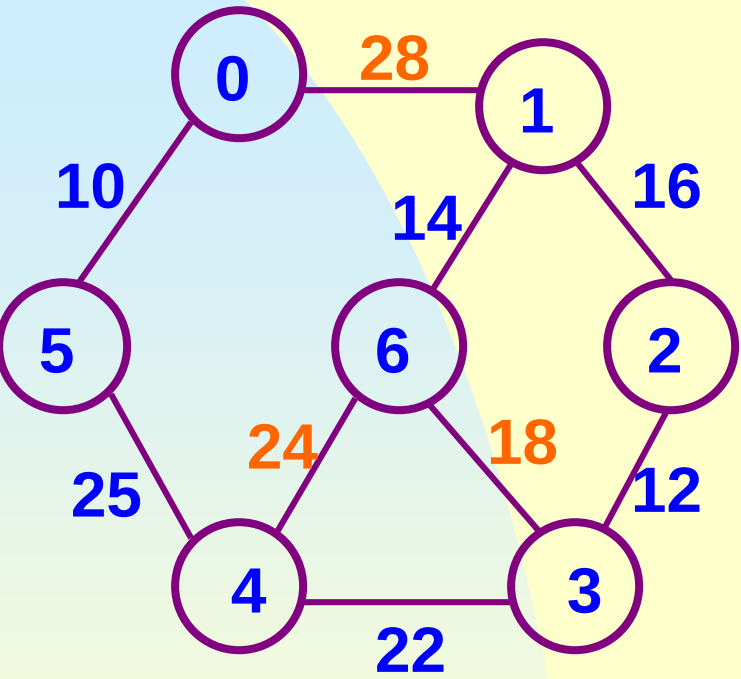


index	0	1	2	3	4	5	6
Pos	0	5	4	3	2	1	6

update EC_heap[Pos[0]], no need to adjust heap



output and vertex 0 is chosen



index	0	1	2	3	4	5	6
Pos	0	5	4	3	2	1	6

- n-1 edges has been output.

6.4 Shortest Paths

6.4.1 Single Source/All Destinations: Nonnegative Edge Cost

- **Problem:** For a digraph $G(V, E)$ with nonnegative edge cost, how to determine a shortest path from the source vertex v to each of remaining vertices of G .
- We can use a greedy algorithm to generate these shortest paths in nondecreasing order of length. This algorithm was given by Dijkstra.

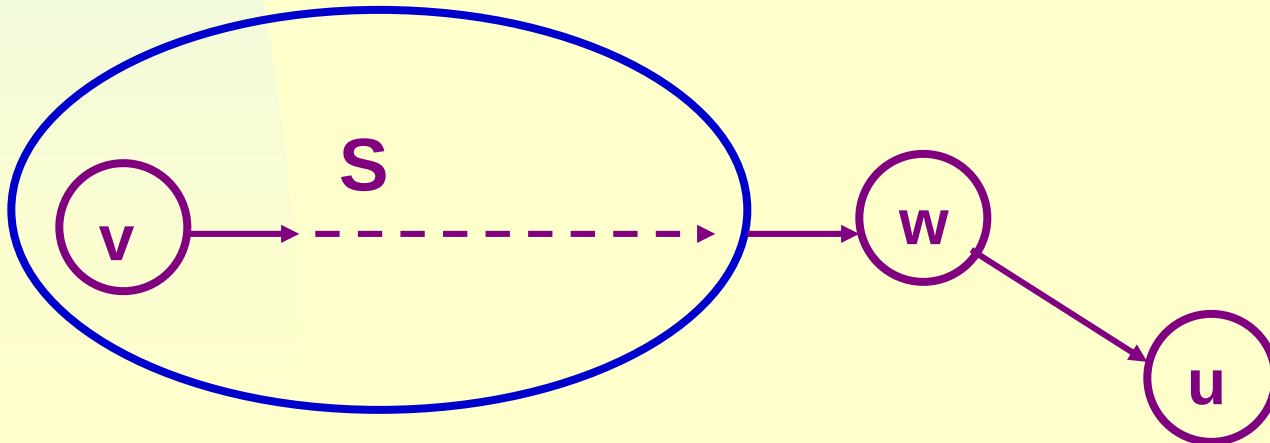
Some conventions:

- ✓ Let S denote the set of vertices (including the source v) to which the shortest paths have been found.
- ✓ For $w \notin S$, let $\text{dist}[w]$ be the length of the shortest path starting from v , going through only the vertices that are in S , and ending at w .

Some observations:

(1) If the next shortest path is to vertex u , then this path goes through only vertices in S .

To prove this we must show that all of intermediate vertices on the shortest path to u must be in S . Assume there is a vertex w on this path that is not in S , then the v -to- u path also contains a path from v to w whose length is less than that of the v -to- u path, as shown below:



By assumption, the shortest paths are being generating in **nondecreasing order** of path length, so the shorter path from v to w has been generated already. Hence there is no intermediate vertex that is not in S .

(2) The destination of the next path generated must be the vertex **u** that has the minimum distance, **dist[u]**, among all vertices not in **S**:

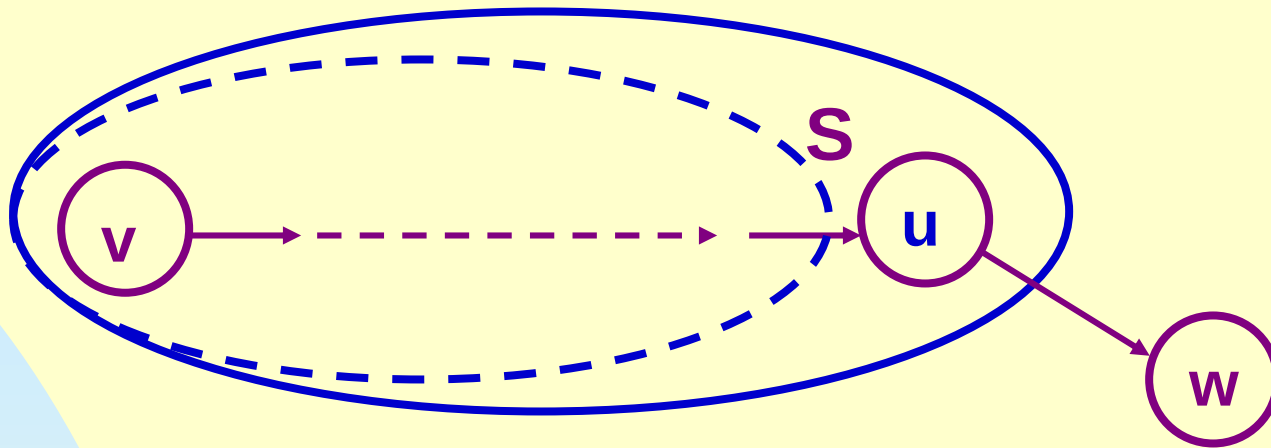
$$\text{dist}[u] = \min_{v \notin S} \{ \text{dist}[v] \}$$

This follows from **the definition of dist and observation (1)**. If there are several vertices not in **S** with the same **dist**, then any of these may be selected.

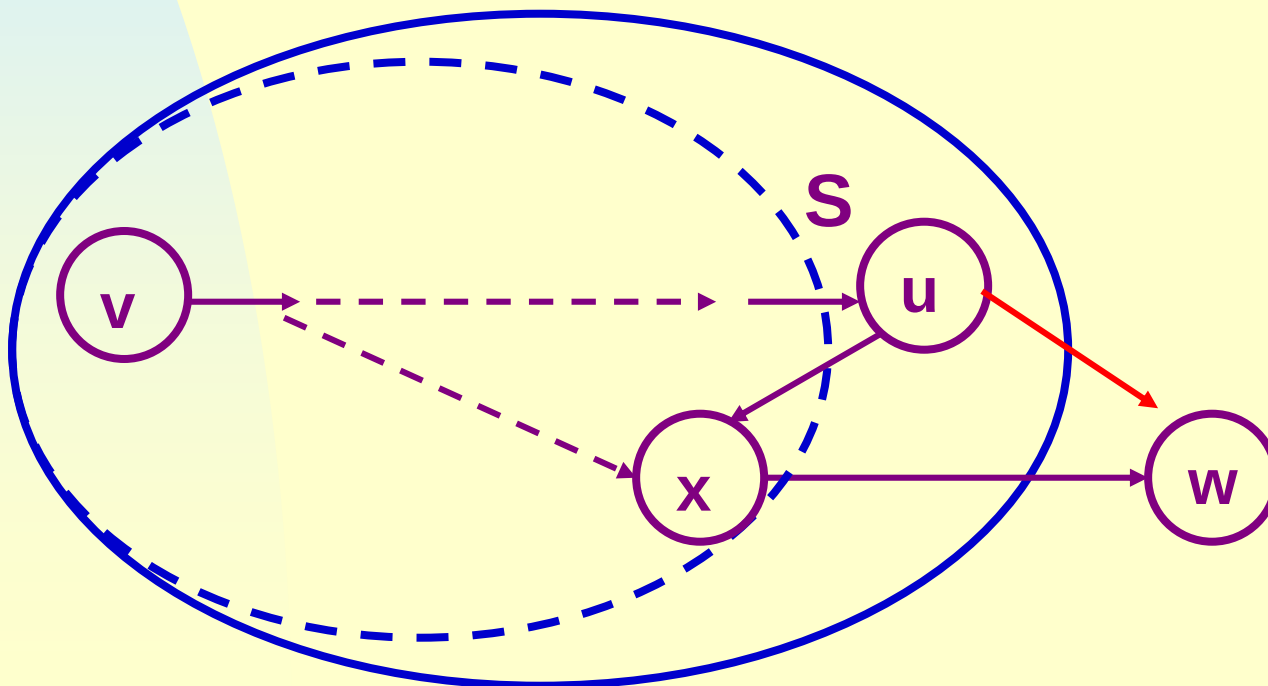
(3) When the vertex **u** selected in (2) becomes a member of **S**, the **shortest v-to-u path** can be obtained. At this point, for a vertex **w not in S** the value of $\text{dist}[w]$ may **decrease**.

If it **does change**, then the change must be **due to a shorter path starting at v going to u and then to w**. According to the definition of $\text{dist}[w]$, we know that the intermediate vertices on the **v-to-u** path and the **u to w** path **must all in S**.

Note: the **u-to-w** path can be chosen so that it does not contain any intermediate vertices as shown in the next slide.



Reason: If the u -to- w path contains a vertex x in the old S before u is added, as shown below:



Then, $\text{length}(v-u-x) > \text{dist}[x]$, since x is added to S before u . Thus, $\text{length}(v-u-x-w) > \text{length}(v-x-w) \geq \text{dist}[w]$. This cannot affect $\text{dist}[w]$.

Therefore, if $\text{dist}[w]$ changes (decreases), the change is due to the path from v to u to w , where the path from v to u is the shortest such path and path from u to w does not contain any intermediate vertices. It is the edge $\langle u, w \rangle$. So, the length of this path is $\text{dist}[u] + \text{length}(u, w)$.

That means if $\text{dist}[u] + \text{length}(u, w) < \text{dist}[w]$, we should update $\text{dist}[w]$.

More assumptions for realizing the algorithm

- ✓ n vertices of G are numbered $0, 1, \dots, n-1$.
- ✓ The set S is maintained as a **boolean array** with **$s[i]=\text{false}$** if vertex $i \notin S$, and **$s[i]=\text{true}$** if $i \in S$.
- ✓ G is represented by its **length-adjacency matrix**, with $\text{length}[i][j]$ being the length of the edge $\langle i, j \rangle$.
- ✓ If $\langle i, j \rangle \notin E(G)$ and $i \neq j$, $\text{length}[i][j]$ may be set to some large number **LARGE**. For $i=j$, $\text{length}[i][j]$ may be set to **any non-negative number** without affecting the outcome of the function.
- ✓ We use the number function `ShortestPath` of Class `MatrixWDigraph` to compute the length of the shortest paths from v to each of the other vertices in G .

- ✓ The class definition of MatrixWDigraph is as follows:

```
class MatrixWDigraph {  
private:  
    double length[NMAX][NMAX]; // NMAX is a constant  
    double *dist; // dist[j] is set to the length of the shortest  
                    //path from the source to j.  
  
    bool *s;  
public:  
    void ShortestPath(const int, const int);  
    int Choose(const int);  
};
```

- ✓ The constructor will apply space for array **dist** and **s**. **Choose** returns a value u such that $\text{dist}[u] = \text{minimum}\{\text{dist}[w]\}$, where $s[w]=\text{false}$.

Program 6.8

```
1 void MatrixWDigraph::ShortestPath(const int n, const int v)
2 { //dist[j] is set to the length of the shortest path from v to j
3   //in a digraph G with n vertices
4   for (int i=0; i<n; i++) {s[i]=false; dist[i]=length[v][i];}
5   s[v]=true;
6   dist[v]=0;
7   for (i=0; i<n-2; i++) { //determine n-1 paths from v
8     int u=Choose(n); //Choose returns a value u such that
9       //dist[u]=minimum dist[w], where s[w]=false
10    s[u]= true;
11    for ( int w=0; w<n; w++)
12      if (!s[w] && dist[u]+length[u][w]<dist[w])
13        dist[w]=dist[u]+length[u][w];
14  } //end of for (i=0;...)
15}
```

Note: ShortestPath only get the length of the shortest paths from v to each of the other vertices in G, **not the path.**

■ How to generate the paths?

Inserting “**path[w]=u;**” after line 13 including in the if clause, where **path[w]** stores **the preceding vertex of w** for the shortest path from the source vertex to w.

■ How to initialize a path?

If there is a edge from the source vertex v to vertex j , then $\text{path}[j]=v$; else $\text{path}[j]=-1$.

■ How to output a path from v to u reversely?

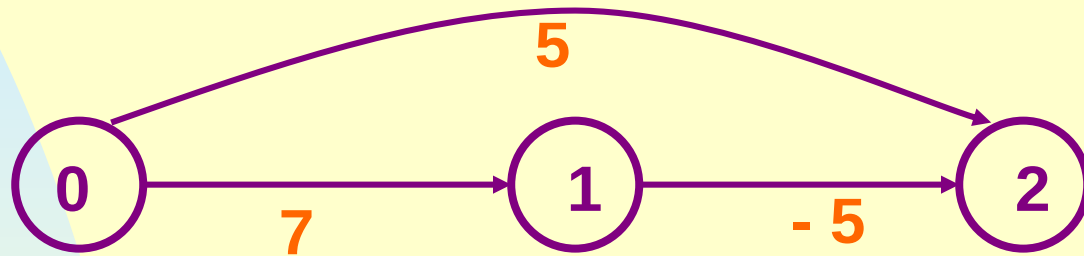
```
if (path[u]==-1)
    cout<<"There is no path from v to u"
else
{
    cout<<u;
    while(v!=path[u])
    {
        cout<<path[u];
        u=path[u];
    }
    cout<<v;
}
```

Analysis of ShortestPath:

- ✓ The time for loop of line 4 is $O(n)$. The for loop of line 7 is executed **$n-2$ times**. Each execution of this loop requires $O(n)$ at line 8 to select the next vertex and again at line 11 to 13 to update dist. So the **total** time for this loop is **$O(n^2)$** .
- ✓ For the **length-adjacency matrices** representation, then, algorithm ShortestPath is **optimal** to within a constant factor.
 - Any shortest-path algorithm **must examine each edge in the graph at least once**, since any of the edges could be in a shortest path.

- ✓ Even if a change to **adjacency lists** is made, the overall time for loop of lines 11 to 13 can be brought down to $O(e)$ (since the dist can change only for vertices that are adjacent from u), but the **total time for line 8** remains $O(n^2)$.
- ✓ Note: By using **Fibonacci heap and adjacency lists**, the greedy algorithm for the single-source/all-destinations problem can be implemented to have complexity $O(n \log n + e)$. Thus for sparse graphs, this implementation is superior to that of Program 6.8.

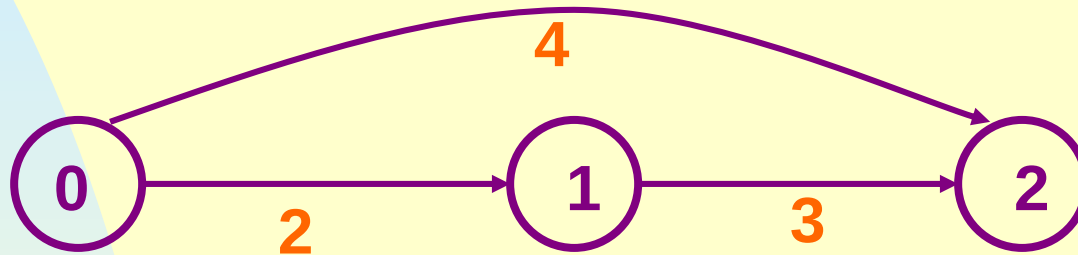
- Note the algorithm does not work when some edges may have **negative length**.
- For example, we assume vertex 0 is the source in the digraph as shown bellow.



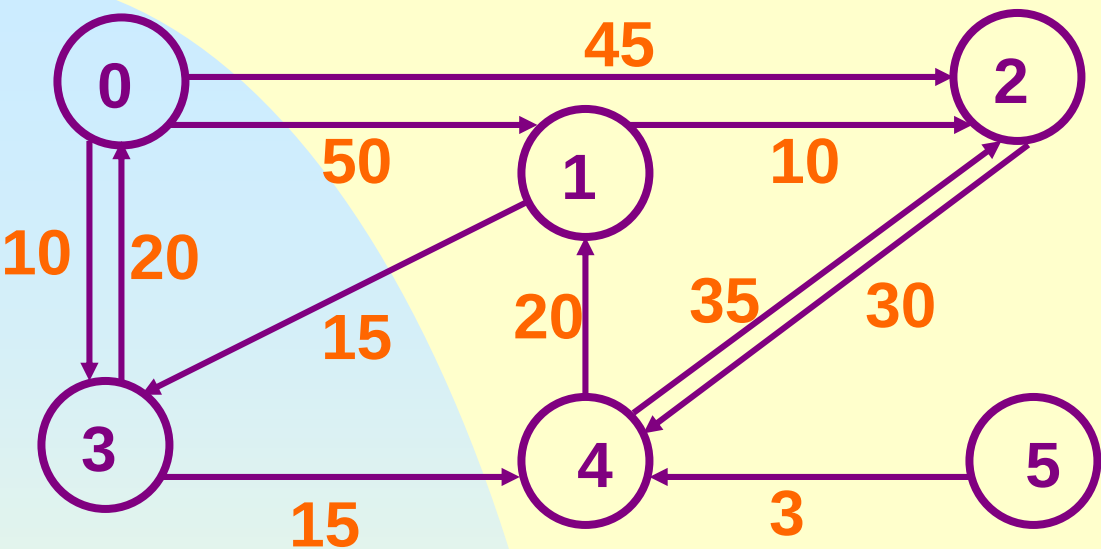
- According to the algorithm, the first shortest path generated is the edge (0,2), but in fact, the shortest path from 0 to 2 is 0, 1, 2.

■ Thinking:

- ✓ Can Dijkstra's algorithm generate a spanning tree?
- ✓ If so, is a spanning tree generated by Dijkstra's algorithm a minimum-cost spanning tree?



Example:



0 is the source vertex

	path	Length
1)	0, 3	10
2)	0, 3, 4	25
3)	0, 3, 4, 1	45
4)	0, 2	45
5)	0, 5	∞

vertex	0	1	2	3	4	5
s	1	0	0	0	0	0
dist	0	50	45	10	∞	∞
path	-1	0	0	0	-1	-1
s	1	0	0	1	0	0
dist	0	50	45	10	25	∞
path	-1	0	0	0	3	-1
s	1	0	0	1	1	0
dist	0	45	45	10	25	∞
path	-1	4	0	0	3	-1
s	1	1	1	1	1	0
dist	0	45	45	10	25	∞
path	-1	4	0	0	3	-1
s	1	1	1	1	1	1
dist	0	45	45	10	25	∞
path	-1	4	0	0	3	-1

6.4.3 All-Pairs Shortest Paths

Problem: How to find the shortest paths between **all pairs** of vertices u and v , $u \neq v$.

One solution:

- This problem can be solved as **n independent single-source/all-destinations problems** using each of the n vertices of G as a source vertex.
- If we use this approach on graphs with nonnegative edges, the total time taken would be **$O(n^3)$** .

Another solution:

- Using the **dynamic programming** approach, we can obtain a **conceptually simpler algorithm** that has complexity of $O(n^3)$ and works even when G has edges with **negative length** so long as G has **no cycles with negative length**.

Dynamic programming

A problem can be solved by dynamic programming has two key attributes: **optimal substructure** and **overlapping sub-problems**.

- ✓ **Optimal substructure** means that the solution to a given optimization problem can be obtained by the combination of optimal solutions to its sub-problems. Such optimal substructures are usually described by means of recursion.
- ✓ **Overlapping sub-problems** means that the problem should solve the same sub-problems over and over, rather than generating new sub-problems.

Overlapping sub-problem example

There are many subproblems which are solved again and again.

/* simple recursive program for Fibonacci numbers */

int fib(int n) //f(0)=0, f(1)=1, f(2)=1, f(3)=2,...

{

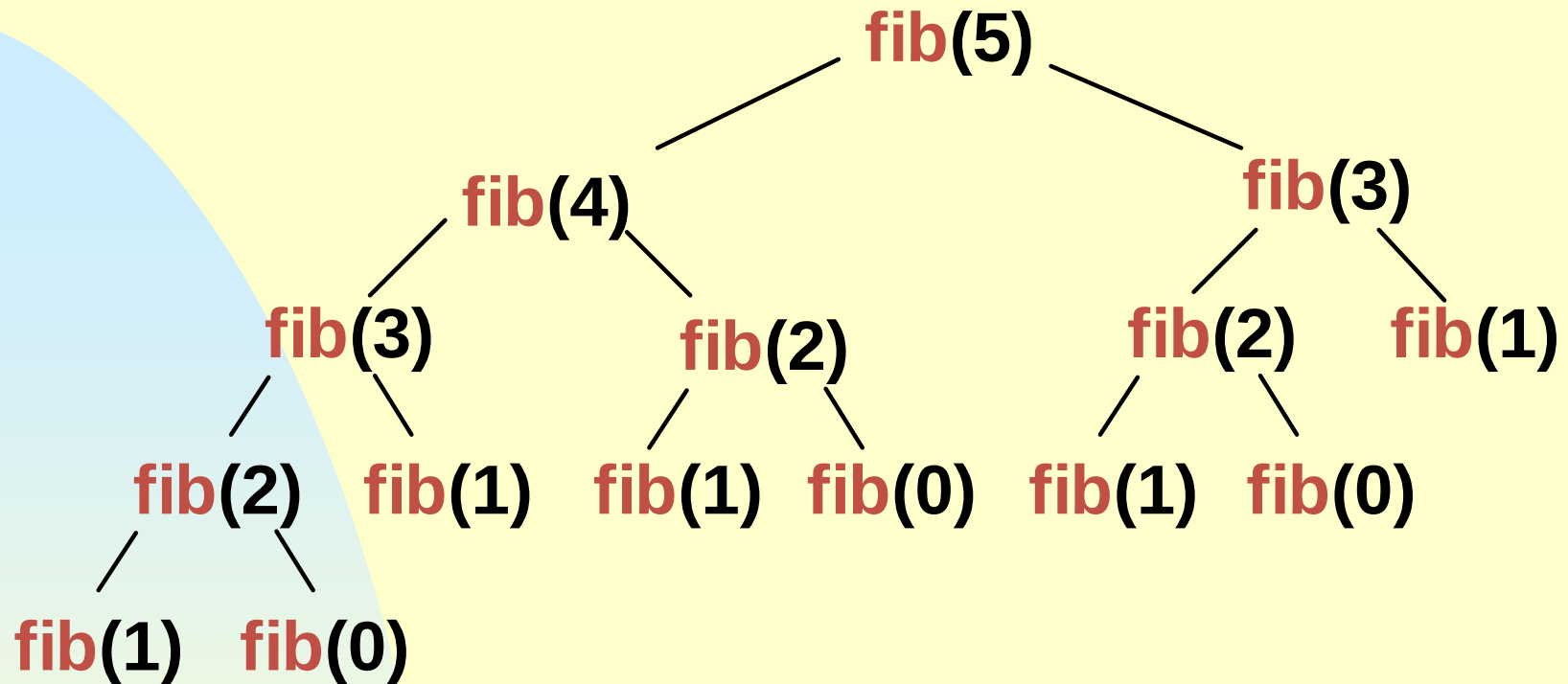
if (n <= 1)

return n;

return fib(n-1) + fib(n-2);

}

Recursion tree for execution of fib(5)



- ✓ The function fib(2) is being called **3 times**.
- ✓ If the value of fib(2) is **stored**, then instead of computing it again, we can **reuse the stored value**.

Two ways to solve the dynamic programming problems

Dynamic programming solves **each sub-problem only once**, that is, stores the results of sub-problems to avoid computing the same results again. There are two different ways:

- ✓ Top-down (Memorization) approach
- ✓ Bottom-up (Tabulation) approach

Top-down (Memorization) approach:

- ✓ If the solution to any problem can be **formulated recursively by using the solution to its sub-problems**, and if its sub-problems are overlapping, then one can easily **memorize or store the solutions to the sub-problems in a table**.
- ✓ Whenever we attempt to solve a new sub-problem, we **first check the table** to see if it is already solved. If a solution has been recorded, we can use it directly, otherwise we solve the sub-problem and add its solution to the table.

Bottom-up (Tabulation) approach:

- ✓ We try solving the small sub-problems first and stores the solutions in a table. Then we use these solutions to build-on and arrive at solutions to bigger sub-problems.
- ✓ So we usually **iteratively** generate solutions to bigger and bigger sub-problems **in a tabular form**.

Memorized version to compute Fibonacci numbers

```
int lookup[MAX]; //lookup stores the Fibonacci numbers
void Initialize() //initializing lookup before fib function is executed.
{
    for (int i = 0; i < MAX; i++) lookup[i] = NIL; //NIL may be -1.
}

int fib(int n) {
    if (lookup[n] == NIL) //check the lookup table first
    {
        if (n <= 1)
            lookup[n] = n;
        else
            lookup[n] = fib(n-1) + fib(n-2);
    }
    return lookup[n];
}
```

Tabulated version to compute Fibonacci numbers

```
/*Computing f(0), f(1), f(2),...orderly. */  
int fib(int n)  
{  
    int f[n+1]; //f stores the Fibonacci number  
    f[0] = 0;  
    f[1] = 1;  
    for (int i = 2; i <= n; i++)  
        f[i] = f[i-1] + f[i-2];  
    return f[n];  
}
```

Solution to All-Pairs Shortest-Path Problem

Assume:

The graph G is represented by **length-adjacency matrix**.

Define:

$A^k[i][j]$ --- the length of shortest path from i to j going through **no intermediate vertex of index greater than k** .

Then:

$A^{n-1}[i][j]$ --- the length of the shortest i -to- j path in G .

$A^{-1}[i][j]$ --- $\text{length}[i][j]$.

■ The basic idea of the algorithm:

Successively generate $A^{-1}, A^0, A^1, \dots, A^{n-1}$.

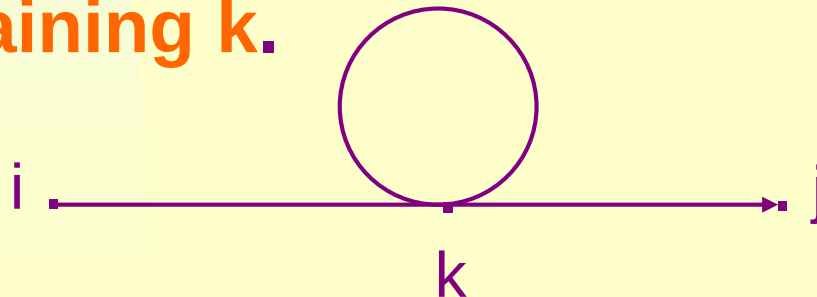
If we have already generated A^{k-1} , we can generate A^k by realizing that: for any pair of i and j , either of the following 2 cases applies:

(1) The shortest path from i to j going through no vertex with index greater than k **does not go through the vertex with index k** , so its length is $A^{k-1}[i][j]$.

(2) The shortest path from i to j going through no vertex with index greater than k **does go through k** .

In this case, the path consists of a subpath from i **to k** and another one from **k to j** . These subpaths must be the shortest paths from i to k and from k to j going through no vertex with index greater than $k-1$ (**not k , vertex k is visited once**), so their lengths are $A^{k-1}[i][k]$ and $A^{k-1}[k][j]$.

Note the above is true only if G has **no negative cycle containing k** .



From (1) and (2), we have:

$$A^k[i][j] = \min\{A^{k-1}[i][j], A^{k-1}[i][k] + A^{k-1}[k][j]\}, k \geq 0.$$

$$A^{-1}[i][j] = \text{length}[i][j]$$

Note: $A^k[i][k] = A^{k-1}[i][k]$ and $A^k[k][j] = A^{k-1}[k][j]$.

The function **Alllengths** in the next slide (Program 6.10) computes $A^{n-1}[i][j]$.

Assumptions:

- ✓ The two-dimensional array **a** is used to store A^k .
- ✓ **a** is the data member of MatrixWDigraph and the constructor will apply space for it.

Program 6.10

```
1 void MatrixWDigraph::AllLengths (const int n)
2 { //length[n][n] is the adjacency matrix of G with n vertices.
3   //a[i][j] is the length of shortest path from i to j
4   for (int i=0; i<n; i++)
5     for (int j=0; j<n; j++)
6       a[i][j] = length[i][j]; // copy length into a
7   for ( int k=0; k<n; k++) // for a path with highest index k
8     for (i=0; i<n; i++) // for all pairs
9       for (j=0; j<n; j++)
10        if (a[i][k]+a[k][j]<a[i][j]) a[i][j]=a[i][k]+a[k][j];
11 }
```

The time is obviously $O(n^3)$.

■ How to obtain the shortest **path** from vertex i to j ?

Inserting “**path[i][j]=path[k][j];**” after line 10 including in the if clause, where $\text{path}[i][j]$ stores **the preceding vertex of j** for the shortest path from vertex i to vertex j .

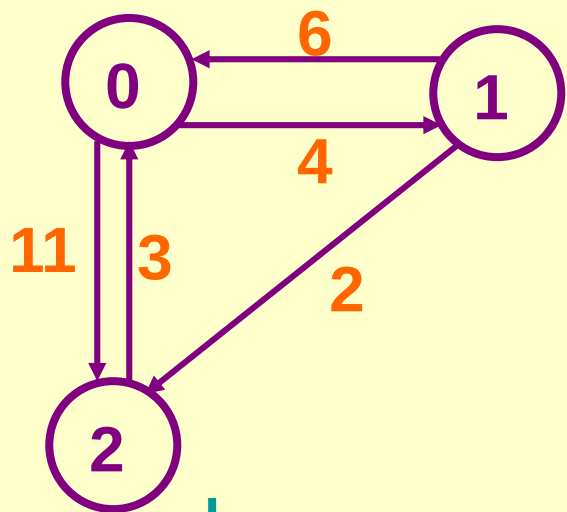
■ How to initialize path?

If there is a edge from the vertex i to vertex j , then $\text{path}[i][j]=i$; else $\text{path}[i][j]=-1$.

■ How to output a path from i to j reversely?

```
if (path[i][j]==-1)
    cout<<"there is no path from i to j"
else
{
    cout<<j;
    while(i!=path[i][j])
    {
        cout<<path[i][j];
        j=path[i][j];
    }
}
cout<<i;
```

Example 6.7:



A ⁻¹	0	1	2
0	0	4	11
1	6	0	2
2	3	∞	0

P	0	1	2
0	-1	0	0
1	1	-1	1
2	2	-1	-1

A ⁰	0	1	2
0	0	4	11
1	6	0	2
2	3	7	0

P	0	1	2
0	-1	0	0
1	1	-1	1
2	2	0	-1

A ¹	0	1	2
0	0	4	6
1	6	0	2
2	3	7	0

P	0	1	2
0	-1	0	1
1	1	-1	1
2	2	0	-1

A ²	0	1	2
0	0	4	6
1	5	0	2
2	3	7	0

P	0	1	2
0	-1	0	1
1	2	-1	1
2	2	0	156-1

6.5 Activity Networks

6.5.1 Activity-on-Vertex (AOV) Networks

Definition: A directed graph G in which the **vertices** represent **tasks or activities** and the **edges** represent **precedence relations between tasks** is an **activity-on-vertex network** or **AOV network**.

Example(in the next two slides): the AOV network about **courses learning** needed for a computer science major in a university.

Course-No.	Course-Name	Prerequisites
C1	Programming I	None
C2	Discrete Mathematics	none
C3	Data Structures	C1, C2
C4	Calculus I	none
C5	Calculus II	C4
C6	Linear Algebra	C5
C7	Analysis of Algorithms	C3, C6
C8	Assembly Language	C3
C9	Operating System	C7, C8
C10	Programming Languages	C7
C11	Compiler Design	C10
C12	Artificial Intelligence	C7
C13	Computational Theory	C7
C14	Parallel Algorithm	C13
C15	Numerical Analysis	C5

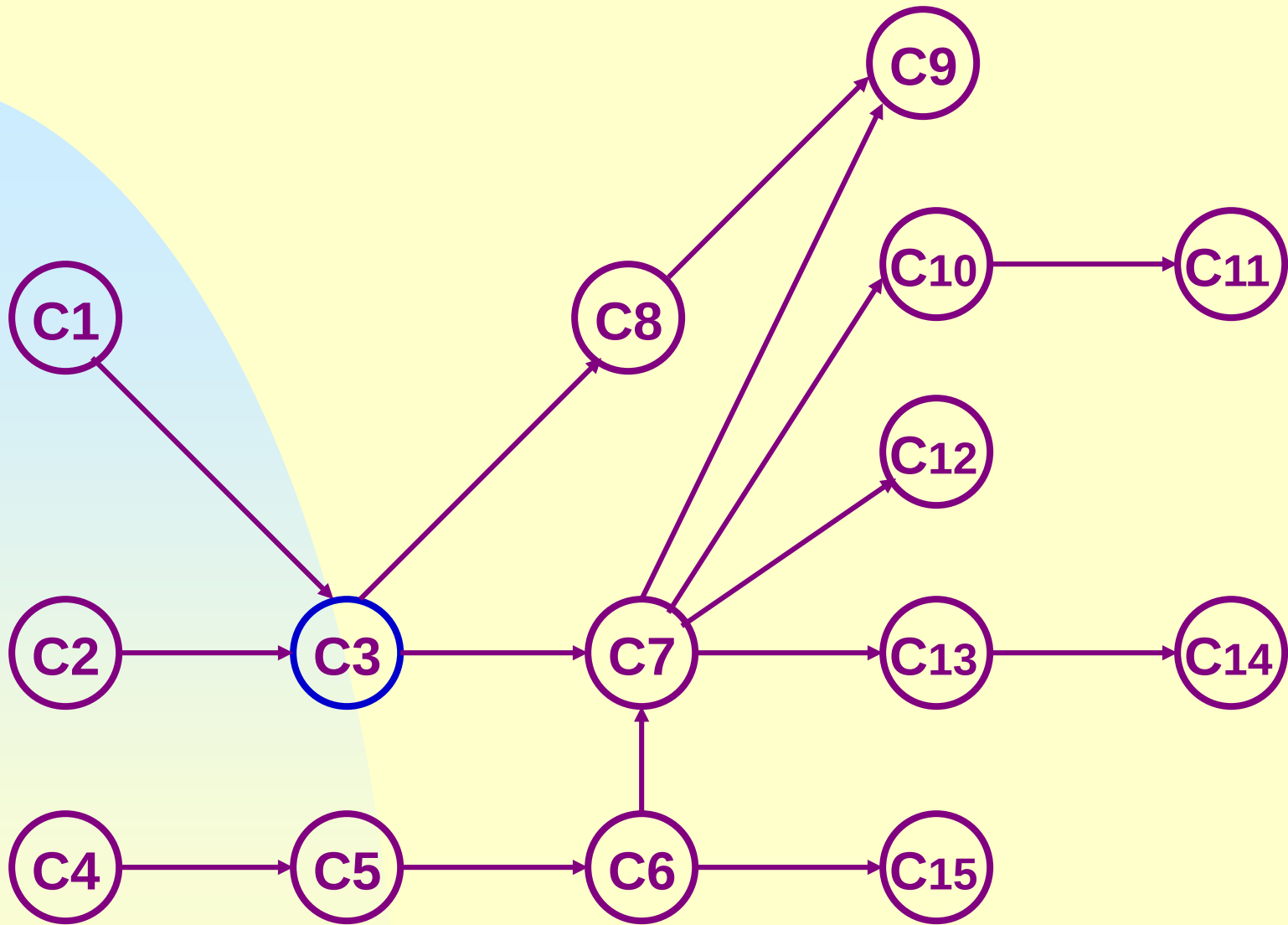


Fig. 6.36

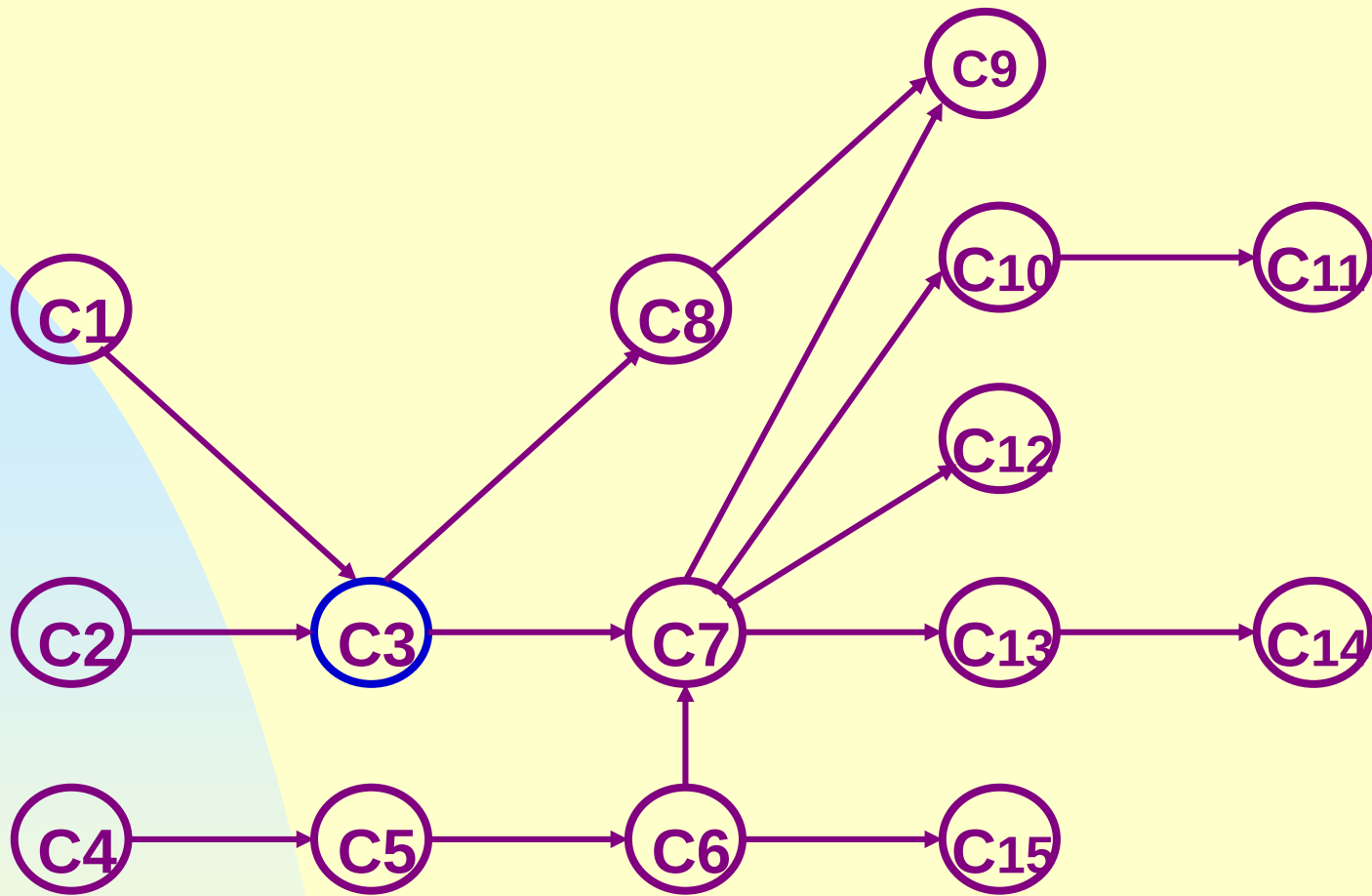
More definitions:

- ✓ In an AOV network G , vertex i is a predecessor of j iff there is a directed path from i to j .
- ✓ If $\langle i, j \rangle$ is an edge in G then i is an immediate predecessor of j and j immediate successor of i .
- ✓ A relation $\cdot$ is transitive iff it is the case that for all triples i, j, k , $i \cdot j$ and $j \cdot k \Rightarrow i \cdot k$.
- ✓ A relation $\cdot$ is irreflexive on a set S if there is no element x in S such that $x \cdot x$.
- ✓ A relation that is both transitive and irreflexive is a (strict) partial order.

Topological order

Definitions:

- In an directed graph G , vertex i is a predecessor of j iff there is a directed path from i to j .
- If $\langle i, j \rangle$ is an edge in G then i is an immediate predecessor of j and j immediate successor of i .
- A topological order is a linear ordering of vertices of a graph such that, for any two vertices i and j , if i is a predecessor of j in the network, then i precedes j in the linear ordering.



Two possible topological orders of Fig. 6.36 are:

C1,C2,C4,C5,C3,C6,C8,C7,C10,C13,C12,C14,C15,C11,C9

and

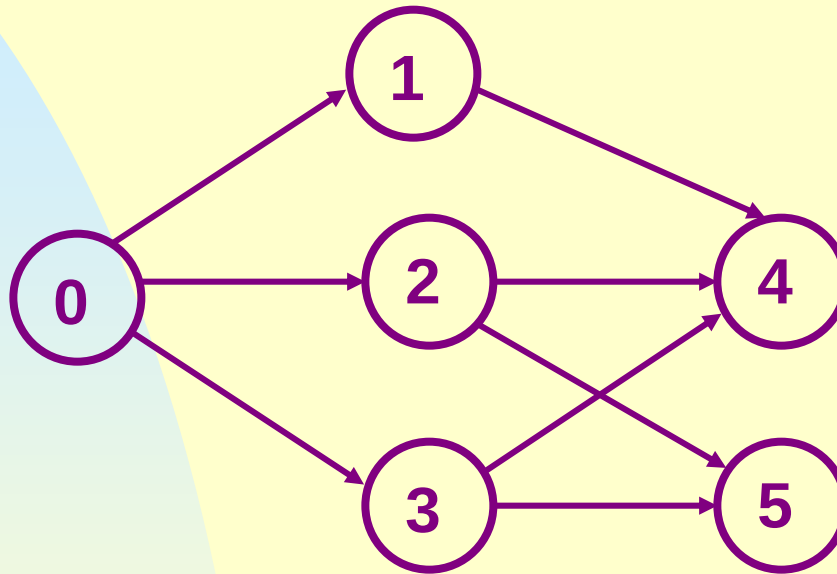
C4,C5,C2,C1,C6,C3,C8,C15,C7,C9,C10,C11,C12,C13,C14

Topological sorting algorithm

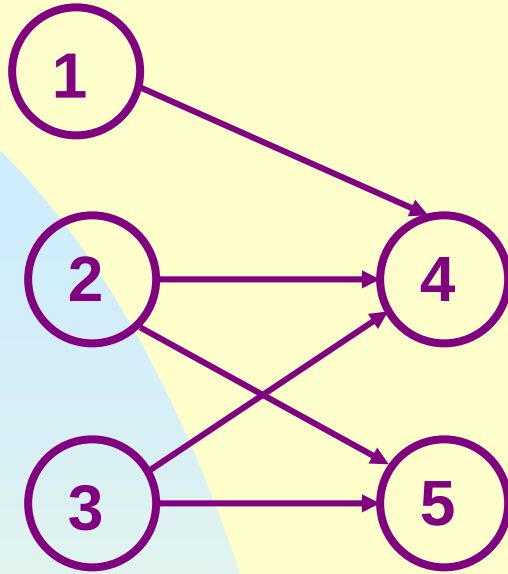
Idea:

- ✓ List a vertex with no predecessor.
- ✓ Then delete this vertex together with all edges leading out of it.
- ✓ Repeat the above two steps until all vertices have been listed or all remaining vertices have predecessors.

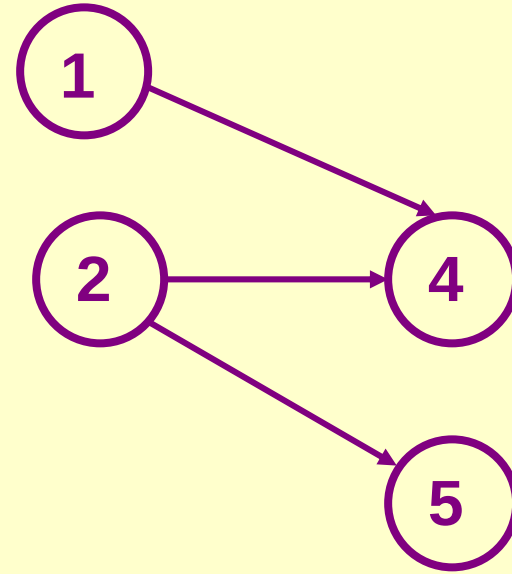
Example (Fig. 6.37):



(a) Initial

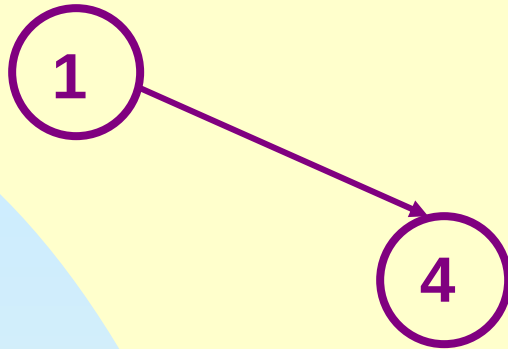


(b) Vertex 0 deleted

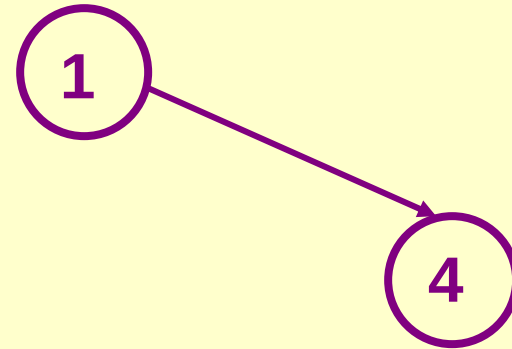


(c) Vertex 3 deleted

Topological order generated: 0, 3,



(d) Vertex 2 deleted



(e) Vertex 5 deleted



(f) Vertex 1 deleted

Topological order generated: 0, 3, 2, 5, 1, 4

■ **To obtain a complete algorithm** that can be easily translated into a computer program, we should consider the following questions, and thus **specify the data representation for the AOV network:**

(1) How to decide whether a vertex has a predecessor?

(2) How to delete a vertex together with all its incident edges?

■ To find a vertex that has no predecessors:

---We maintain a **count** for each vertex to represent the number of its **immediate predecessors**.

---Whenever the count of a vertex **drops to 0**, this vertex is **placed onto a list of vertices with 0 count**.

■ To delete a vertex together with all its incident edges:

--- It is easily implemented if the AOV network is represented by its **adjacency lists**.

---Then, the **deletion of all edges** leading out of vertex v can be carried out **by decreasing the count** of all vertices on its adjacency list. (The edges do not really be deleted.)

■ Based on the above analysis, we use **adjacency lists** to represent the **AOV network**, and we assume:

(1) We define **int *count** as a data member, and its space is allocated by the constructor.

- **count[i]**, $0 \leq i < n$, has been initialized to the in-degree of vertex **i**.
- When an edge **<i, j>** is newly inserted, the **count of j** is increased by 1.

(3) The list of vertices with zero count is maintained as a custom stack.

- The stack is linked through the **count field of the header nodes** since this field is of **no use** after a vertex's count has become zero.
- Note that: **the count fields of the header nodes change** after topological sorting.
- A queue can be used instead, but a stack is slightly simpler.

(4) We define `int *t` as a data member to store all the vertices in topological order for future use, and its space is allocated in the constructor.

Example:

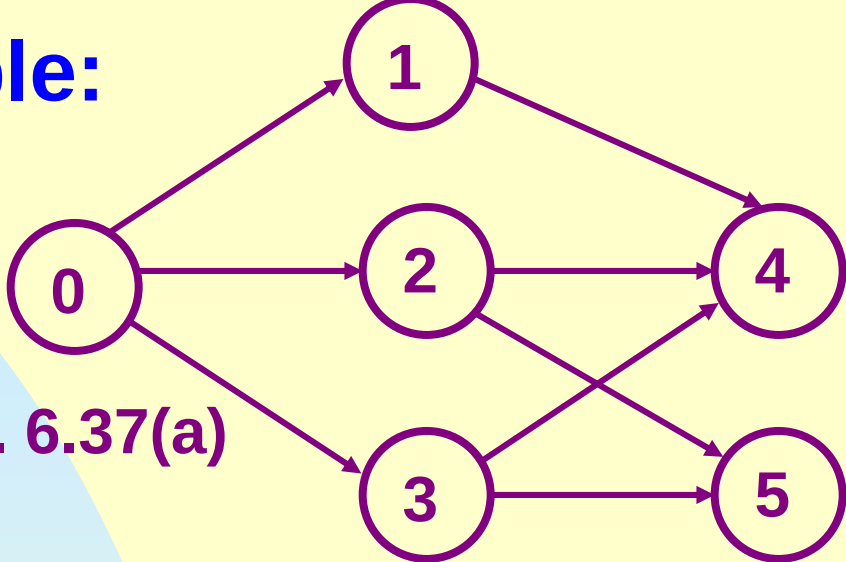
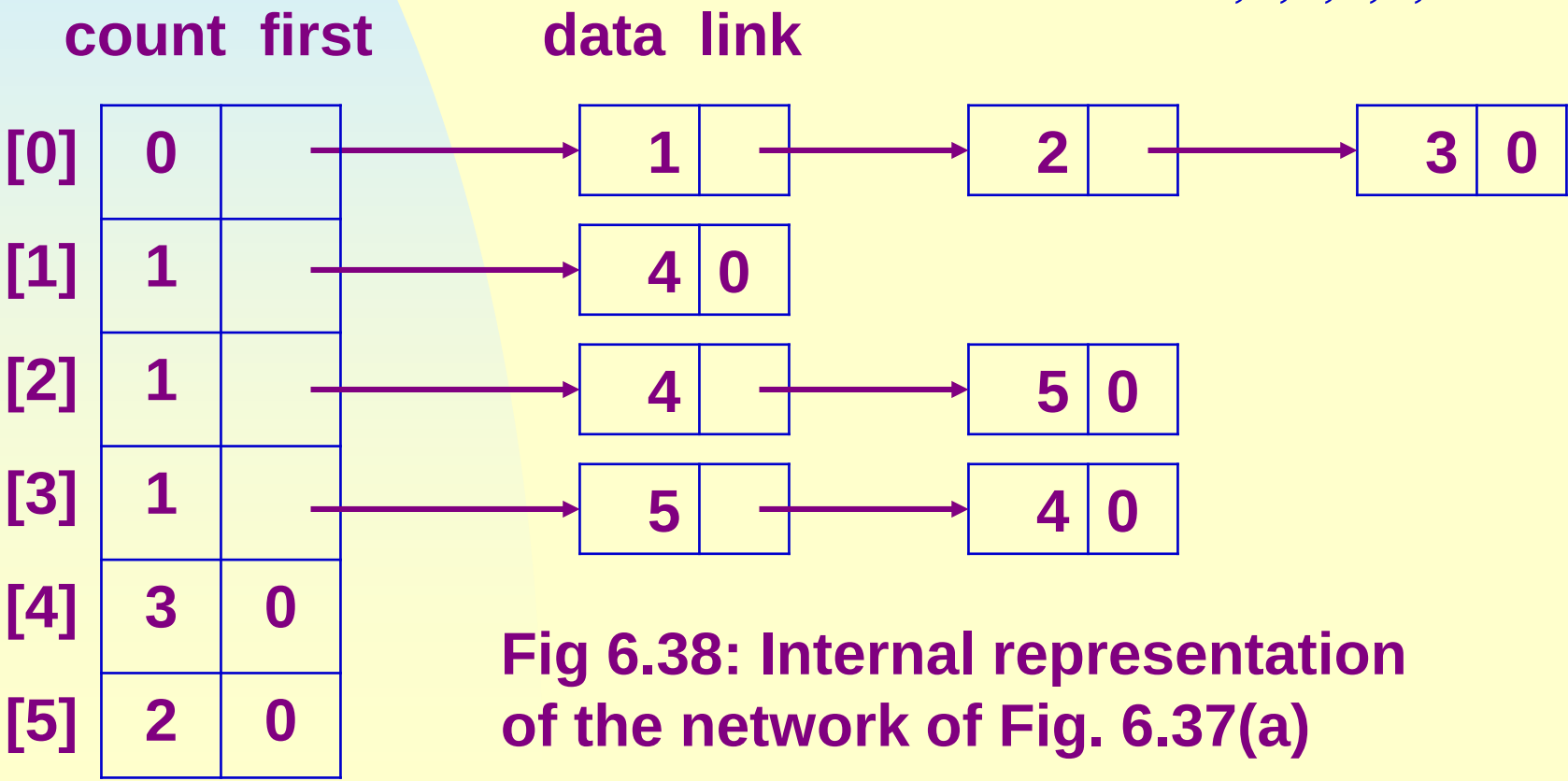


Fig. 6.37(a)

	3				
	2	2	5		
0	1	1	1	1	4

Topological order:
0,3,2,5,1,4



**Fig 6.38: Internal representation
of the network of Fig. 6.37(a)**

Program 6.12 Topological order

```
1 void LinkedDigraph::TopologicalOrder()
2 { //n vertices of a network will be stored into t in topological order
3   int top = -1, pos = 0;
4   for (int i=0; i<n; i++) //create a linked stack of vertices with
5     if (count[i]==0) { count[i]=top; top=i;} //no predecessors
6   for (int i=0; i<n; i++) { //n times of vertex output in possible
7     if (top== -1) throw "network has a cycle.";
8     int j=top; top=count[top]; //unstack a vertex
9     t[pos++] = j; // store vertex j in topological order
10    Chain<int>::ChainIterator ji=adjLists[j].begin();
11    while (ji != adjLists[j].end()) { // decrease the count of
12      count[*ji]--; // the successor vertices of j
13      if (count[*ji]==0) {count[*ji]=top; top=*ji;} //add to stack
14      ji++; // next successor
15    } //end of while
16  } //end of for starting at line 6
17}
```

Analysis of TopologicalOrder:

- The loop of lines 4-5: $O(n)$
- Lines 6-10 totally $O(n)$
- The **while** loop of 11-15 takes $O(d_i)$ for each vertex i , where d_i is the **out-degree** of i . The total time for this part is $O((\sum_{i=0}^{n-1} d_i) + n) = O(e + n)$.
- **The total time:** $O(e + n)$.
- It is linear in the size of the problem! (very efficient)

6.5.2 Activity-on-Edge (AOE) Networks

As for the activity-on-edge (AOE) network (see the example in the next slide):

- Directed edges --- tasks (or activities) to be performed
- Vertices --- events, which signal the completion of certain activities.

We can see in AOE network:

- An activity represented by an edge $\langle i, j \rangle$ cannot be started until the event at that vertex i has occurred.
- An event occurs only when all activities entering it have been completed.

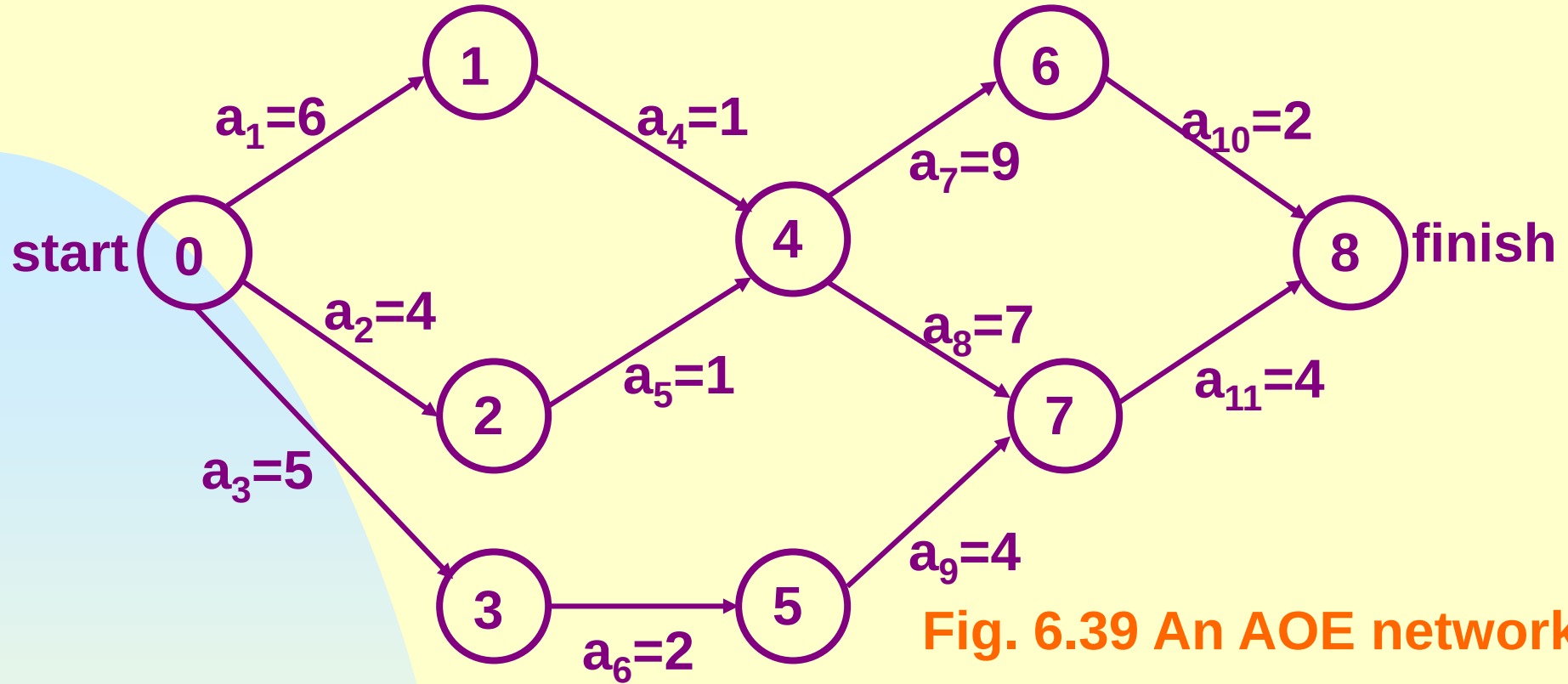


Fig. 6.39 An AOE network

- Since activities in an AOE network can be carried out in parallel, the **minimum time to complete the project is the length of the longest path** from the start vertex to the finish vertex. Specially, we call a path with **longest** length a **critical path**. e.g., the paths 0, 1, 4, 6, 8 and 0, 1, 4, 7, 8 (length 18).

Question: How to find a critical path?

To solve this problem, we introduce some relevant concepts about AOE network:

- ✓ $ee(i)$ --- the **earliest time** that an **event i** can occur, which is the length of the **longest path** from the start vertex to the vertex i .
- ✓ $le(i)$ --- the **latest time** that an **event i** can occur, without increasing the project duration, which is the length of the **longest path** from the start vertex to the finish vertex **minus** the length of the **longest path** from the vertex i to the finish vertex.
- ✓ $e(i)$ --- the **earliest start time** for activity a_i .
- ✓ $l(i)$ --- the **latest start time** for activity a_i without increasing the project duration.

- ✓ $l(i) - e(i)$ is called the **time margin** of the activity a_i . It is the measure of the criticality of a_i . It gives the time by which **an activity may be delayed or slowed without increasing the total time** needed to finish the project.
 - If $e(i) = l(i)$, then a_i is called a **critical activity**.
 - Once we get $e(i)$ and $l(i)$ for all activities, then **critical activities can be easily identified**.

Why we should identify the critical activities?

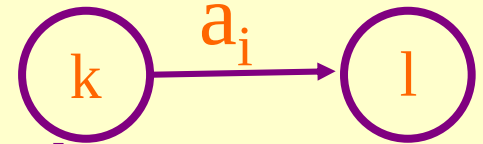
- ✓ Speeding up noncritical activities **will not reduce the project duration**. (For example, even if the noncritical activity a_6 is speeded up to take only 1 days, the total time to finish the project does not reduce.)
- ✓ So, if we attempt to **reduce project finish time**, we must speed up the critical activities.
- ✓ Here we should also note that: **speeding up a critical activity will not necessarily result in a reduced project length unless it is on all critical paths**. (e.g., speed up a_{11} will not reduce the project length, but a_1 will.)

How to obtain $e(i)$ and $l(i)$ and thus to measure the criticality of an activity?

- If activity a_i is represented by edge $\langle k, l \rangle$, then

$$e(i) = ee[k] \quad \text{and}$$

$$l(i) = le[l] - \text{duration of activity } a_i \quad \left. \vphantom{\begin{matrix} e(i) = ee[k] \\ l(i) = le[l] - \text{duration of activity } a_i \end{matrix}} \right\} (6.1)$$



Note: We can't use $le[k]$ to compute $l(i)$. In fact $l(i) \geq le[k]$.

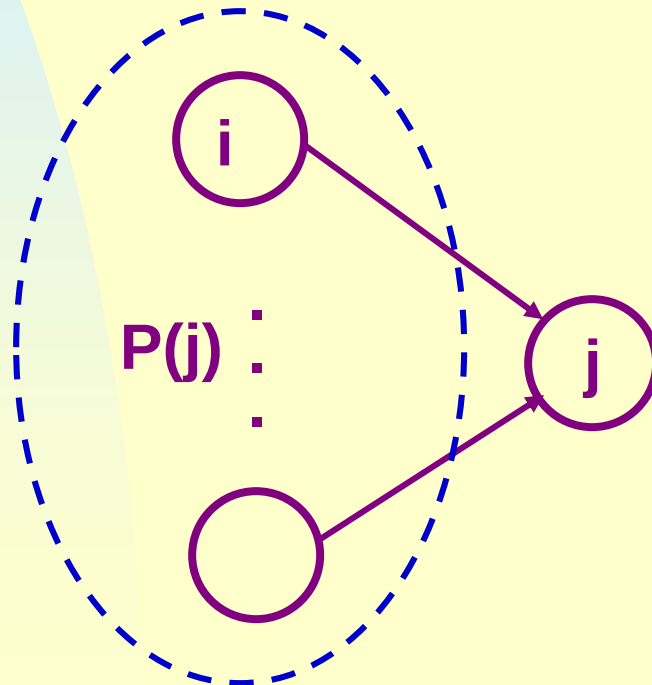
- So, to compute the earliest activity time and the latest activity time, we first obtain: the earliest event time $ee[j]$, and the latest event time $le[j]$, for every event j in the network.
- $ee[j]$ and $le[j]$ are computed in a **forward** stage and a **backward** stage respectively.

6.5.2.1 Calculation of Early Activity Times

A forward stage to calculate early event times:

$$\left. \begin{array}{l} ee[0]=0 \quad (\text{suppose } 0 \text{ is the start}) \\ ee[j]=\max_{i \in P(j)} \{ee[i]+\text{duration of } \langle i, j \rangle\} \end{array} \right\} \quad (6.2)$$

where $P(j)$ is the set of all vertices adjacent to j .



Note: $ee[i]$ ($i \in P(j)$) is computed **before** $ee[j]$, so we compute all the early event times in **topological order**.

Relevant Class definitions for AOV network

Before showing the codes for calculation of Early Activity Times, we give the class definitions:

- ✓ To represent the information of **duration for an activity**, we use the structure:

```
struct Pair
{
    int vertex; //the head (not the tail) of an activity
    int dur;    //activity duration
};
```

- ✓ **The ADT of LinkedDigraph:** //using adjacent lists

```
class LinkedDigraph {
private:
    Chain<Pair> *adjLists;
    int *count, *t, *ee, *le;
    int n;
```

public:

```
LinkedDigraph (const int vertices) {  
    if (vertices < 1)  
        throw "Number of vertices must be > 0";  
    n = vertices;  
    adjLists = new Chain<Pair>[n];  
    count = new int[n]; t = new int[n];  
    ee = new int[n]; le = new int[n];
```

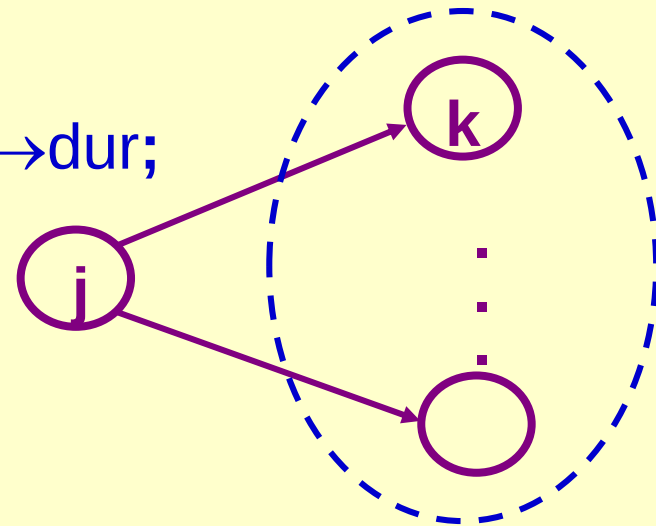
```
};
```

```
void TopologicalOrder();  
void EarliestEventTime();  
void LatestEventTime();  
void CriticalActivities();
```

```
};
```

Earliest Event Time

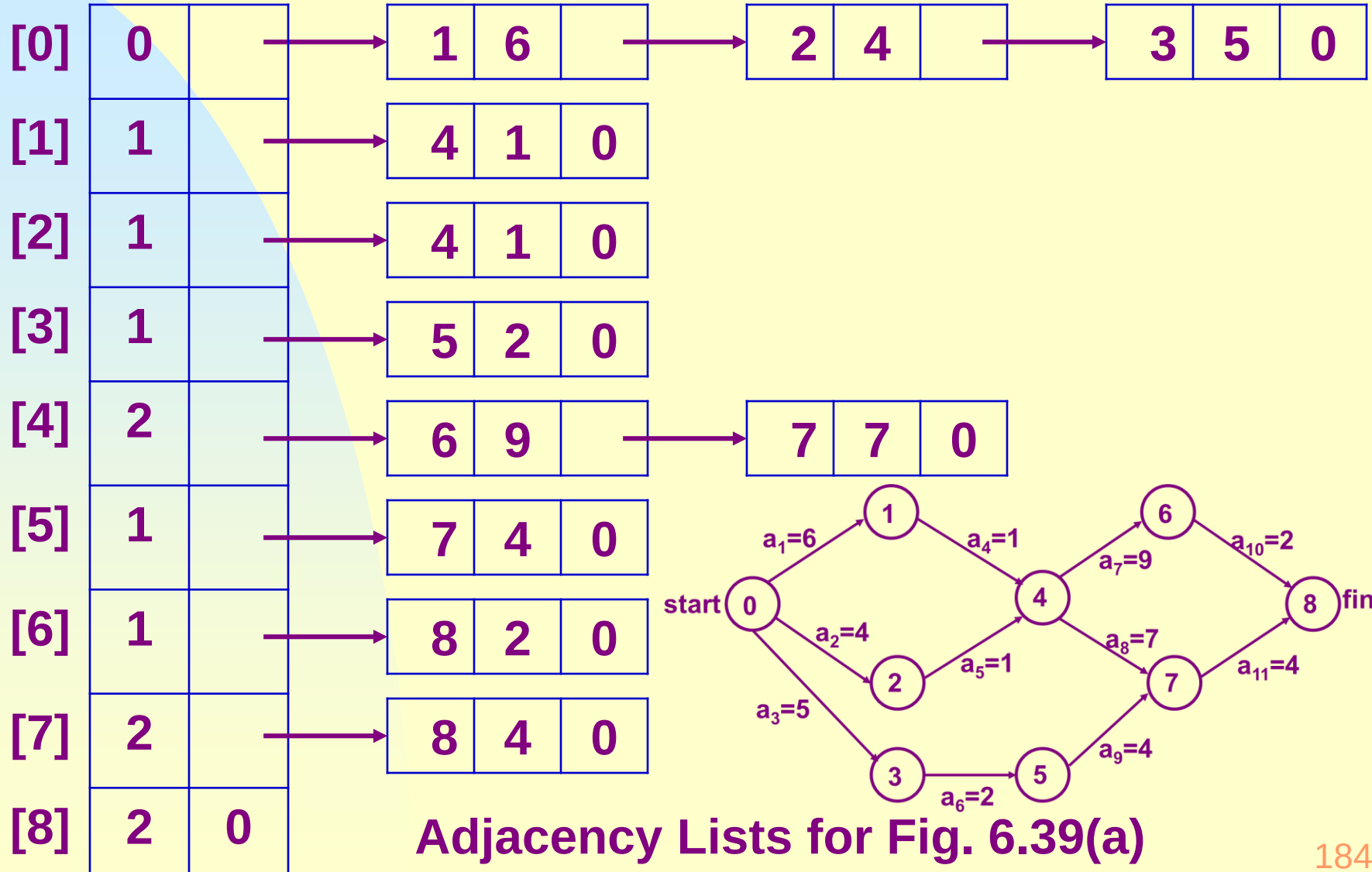
```
void LinkedDigraph::EarliestEventTime()
{ // assume a topological order has already been in t,
  // compute ee[j] according to t
  fill(ee, ee+n, 0); // initialize ee
  for (i=0; i<n; i++) { //processthe ith vertex in topological order
    int j=t[i];
    Chain<Pair>::ChainIterator ji=adjLists[j].begin();
    while (ji!=adjLists[j].end()) {
      int k=ji→vertex; //k is successor of j
      if (ee[k]<ee[j]+ji→dur) ee[k]=ee[j]+ji→dur;
      ji++;
    } //end of while
  } //end of for
}
```



The computing time is obvious $O(n+e)$.

Example:

count first vertex dur link



Computation of ee

Assume the topological order is: **0, 3, 5, 2, 1, 4, 7, 6, 8**

Output order	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
initial	0	0	0	0	0	0	0	0	0
output 0	0	6	4	5	0	0	0	0	0
output 3	0	6	4	5	0	7	0	0	0
output 5	0	6	4	5	0	7	0	11	0
output 2	0	6	4	5	5	7	0	11	0
output 1	0	6	4	5	7	7	0	11	0
output 4	0	6	4	5	7	7	16	14	0
output 7	0	6	4	5	7	7	16	14	18
output 6	0	6	4	5	7	7	16	14	18
output 8									

Program 6.12' Modified Topological order

```
void LinkedDigraph::ModifiedTopologicalOrder()
{
    //n vertices of a network will be stored into t in topological order
    //computing ee in parallel with generation of a topological order
    fill(ee, ee+n, 0); // initialize ee
    int top = -1, pos = 0;
    for (int i=0; i<n; i++) //create a linked stack of vertices with
        if (count[i]==0) { count[i]=top; top=i;} //no predecessors
    for (int i=0; i<n; i++) { //n times of vertex output in possible
        if (top== -1) throw "network has a cycle.";
        int j=top; top=count[top]; //unstack a vertex
        t[pos++] = j; // store vertex j in topological order
        Chain<Pair>::ChainIterator ji=adjLists[j].begin();
    }
```

```
while (ji != adjLists[j].end()) { // decrease the count of
    count[ji->vertex]--;           // the successor vertices of j
    ee[ji->vertex]=max(ee[ji->vertex], ee[j]+ji->dur)
    if (count[ji->vertex]==0)
        {count[ji->vertex]=top; top=ji->vertex;} //add to stack
    ji++; // next successor
} //end of while
} //end of for
}
```

Computation of ee and topological order in parallel

Output order	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	output stack
Initial	0	0	0	0	0	0	0	0	0	[0]
output 0	0	6	4	5	0	0	0	0	0	[3,2,1]
output 3	0	6	4	5	0	7	0	0	0	[5,2,1]
output 5	0	6	4	5	0	7	0	11	0	[2,1]
output 2	0	6	4	5	5	7	0	11	0	[1]
output 1	0	6	4	5	7	7	0	11	0	[4]
output 4	0	6	4	5	7	7	16	14	0	[7,6]
output 7	0	6	4	5	7	7	16	14	18	[6]
output 6	0	6	4	5	7	7	16	14	18	[8]
output 8										

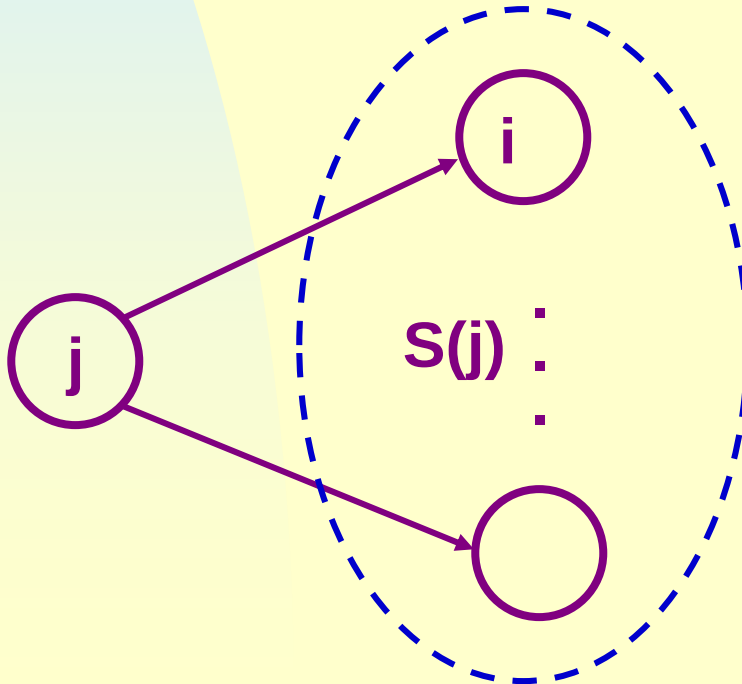
The topological order is: 0, 3, 5, 2, 1, 4, 7, 6, 8

6.5.2.2 Calculation of Late Activity Times

A backward stage to calculate late event times:

$$\left. \begin{aligned} le[n-1] &= ee[n-1] \quad (n-1 \text{ is the finish vertex}) \\ le[j] &= \min_{i \in S(j)} \{le[i] - \text{duration of } \langle j, i \rangle\} \end{aligned} \right\} \quad (6.3)$$

where $S(j)$ is the set of all vertices adjacent from j .

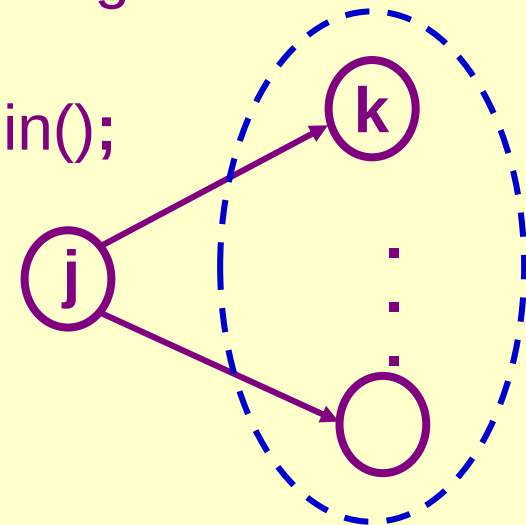


$le[i]$ ($i \in S(j)$) is computed **before** $le[j]$, so we compute all the late event times in **reverse topological order**.

Latest Event Time

$le[i]$ ($0 \leq i < n$) is computed in the **reverse topological order**. Assume we have gotten $ee[n-1]$.

```
void LinkedDigraph::LatestEventTime()
{ // assume a topological order has already been in t, ee has
  // been computed, compute  $le[j]$  in the reverse order of t
  fill(le, le+n, ee[n-1]); // initialize le
  for (i=n-2; i>=0; i--) { //compute in reverse topological order
    int j=t[i];
    Chain<Pair>::ChainIterator ji=adjLists[j].begin();
    while (ji!=adjLists[j].end()) {
      int k=ji->vertex; //k is successor of j
      if ( $le[k] - ji->dur < le[j]$ )  $le[j] = le[k] - ji->dur$ ;
      ji++;
    }
  }
} //The computing time is obvious  $O(n+e)$ .
```



Example:

Assume the topological order of Fig. 6.39 is:

0, 3, 5, 2, 1, 4, 7, 6, 8

we compute $le[i]$ for $i=8, 6, 7, 4, 1, 2, 5, 3, 0$.

$$le[8] = ee[8] = 18$$

$$le[6] = \min \{le[8] - 2\} = 16$$

$$le[7] = \min \{le[8] - 4\} = 14$$

$$le[4] = \min \{le[6] - 9, le[7] - 7\} = 7$$

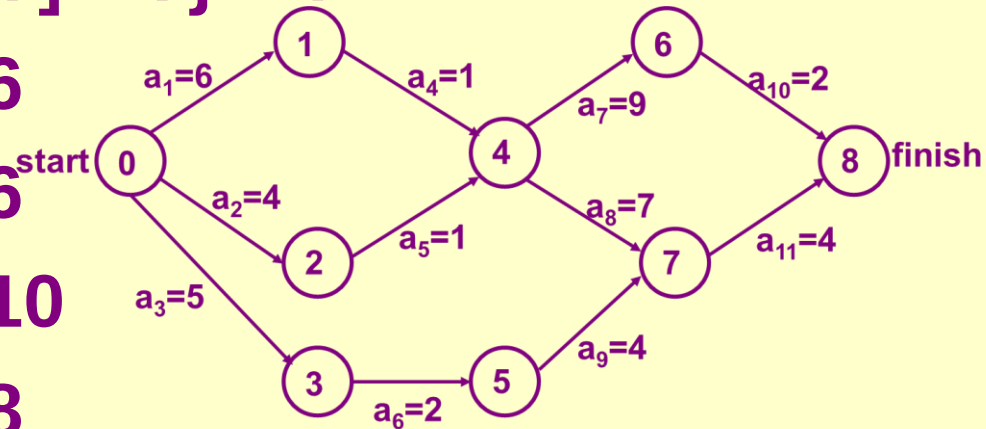
$$le[1] = \min \{le[4] - 1\} = 6$$

$$le[2] = \min \{le[4] - 1\} = 6$$

$$le[5] = \min \{le[7] - 4\} = 10$$

$$le[3] = \min \{le[5] - 2\} = 8$$

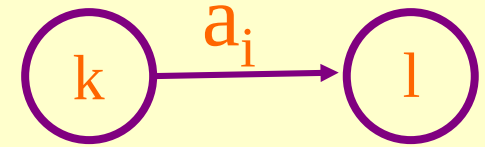
$$le[0] = \min \{le[1] - 6, le[2] - 4, le[3] - 6\} = 0$$



Compute $e[i]$ and $l[i]$, output critical activities

$e(i) = ee[k]$ and

$l(i) = le[l] - \text{duration of activity } a_i <k,l>$ (6.1)



```
void LinkedDigraph::CriticalActivities()
{ // compute  $e[i]$  and  $l[i]$ , output critical activities
  int i=1; // the ith activity
  int u, v, e, l; // e, l are the earliest, latest start time of  $<u, v>$ 
  for (u=0; u<n; u++) { // scan the adjacency lists.
    Chain<Pair>::ChainIterator ui=adjLists[u].begin();
    while (ui!=adjLists[u].end()) {
      v=ui->vertex; //  $<u, v>$  is an edge for activity i
      e=ee[u]; l=le[v]-ui->dur;
      if (l==e) cout <<"a"<<i<<"<"<<u<<","<<v<<">"
        <<"is a critical activity"<<endl;
      ui++; i++;
    } //end of while
  } //end of for
} //The computing time is obvious  $O(e)$ .
```


For Fig. 6.39, the $e(i)$ and $l(i)$ are:

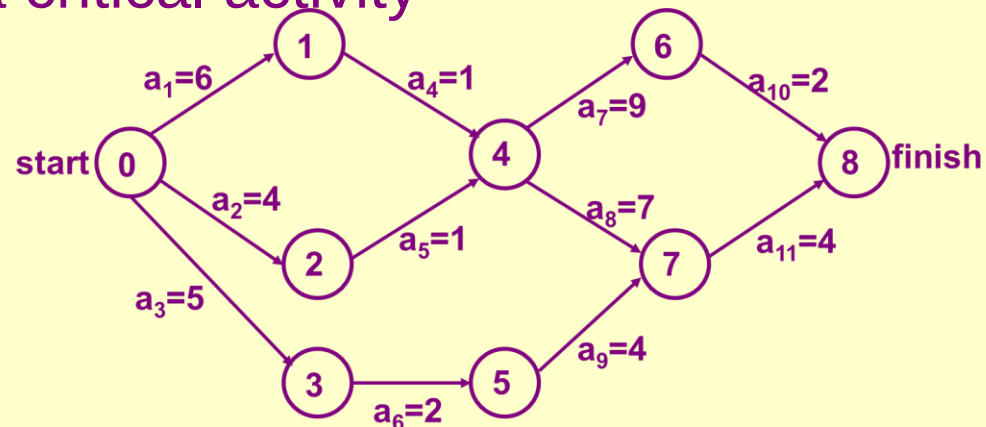
activity	e	l	l-e	l-e=0?
a_1 <0, 1>	0	0	0	Y
a_2 <0, 2>	0	2	2	N
a_3 <0, 3>	0	3	3	N
a_4 <1, 4>	6	6	0	Y
a_5 <2, 4>	4	6	2	N
a_6 <3, 5>	5	8	3	N
a_7 <4, 6>	7	7	0	Y
a_8 <4, 7>	7	7	0	Y
a_9 <5, 7>	7	10	3	N
a_{10} <6, 8>	16	16	0	Y
a_{11} <7, 8>	14	14	0	Y

The critical activities: a_1 , a_4 , a_7 , a_8 , a_{10} , and a_{11} .

Output a critical path

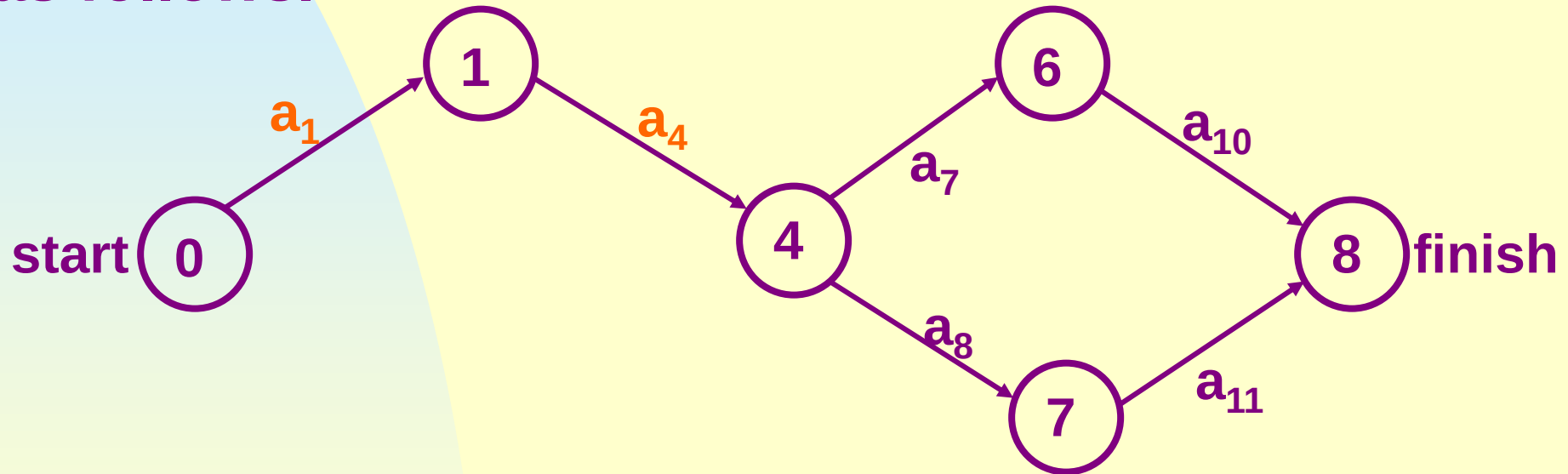
//Note that any critical activity is inevitably on a critical path.

```
void LinkedDigraph::OutputCriticalPath() //edges are activities
{ // 0 is the start vertex, and n-1 is the finish vertex
  int u, v, e, l; // e, l are the earliest, latest start time of <u, v>
  u=0; cout<<u;
  while(u!=n-1) {
    Chain<Pair>::ChainIterator ui=adjLists[u].begin();
    while (ui!=adjLists[u].end()) {
      v=ui->vertex; // <u, v> is an edge
      e=ee[u]; l=le[v]-ui->dur;
      if (l==e) { // <u, v> is a critical activity
        cout<<" "<<v;
        u=v;
        break;
      } //end of if
    } //end of inner while
  } //end of inner while
} //end of outer while
} //The computing time is obvious  $O(e)$ .
```



How to do if we try to **reduce the critical path length**?

Idea: We output all the critical activities (i.e., delete all non-critical activities) to get the **critical network** as follows.



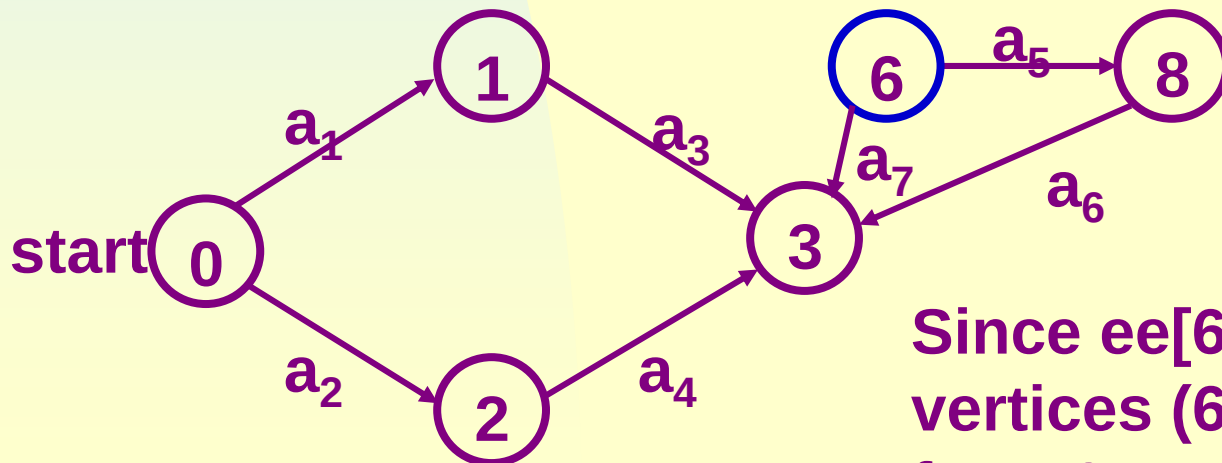
Then, we can see: a_1 and a_4 are on all the critical paths, so speeding up a_1 and a_4 will **reduce the project length**, but a_7 , a_8 , a_{10} or a_{11} will not.

Question: How do we know an activity are **on all the critical paths in the critical network?**

- ✓ We can see in the above example: If we delete a_1 or a_4 , there is **no path** from the start to finish. If we delete a_7 , a_8 , a_{10} or a_{11} , there is still a path from the start to finish.
- ✓ So, we can delete a critical activity (edge) and then do **DFS or BFS from the start vertex** to determine whether there is a path from the start to the finish. If so, this critical activity is the one on all the critical paths.

Another use for critical-path analysis

- ✓ **Topological order** can be used to detect whether there is a directed cycle. But it **cannot** be used to detect whether there is any vertex not reachable from the start vertex.
- ✓ **But Critical-path analysis** can do this task.
 - If finally $ee[i] == 0$ ($i \neq 0$), there are some vertices **not reachable** from the start vertex 0.



Since $ee[6] == 0$, there are some vertices (6 and 8) not reachable from 0.

Exercise(s) and Experiment(s)

Exercise(s):

- **P339:9; P372:1,5**

Experiment(s):

- **P352:3,5; P359:1,3**